

图像检测/分割

计算机视觉任务

语义分割



GRASS, CAT,
TREE, SKY

No objects, just pixels

分类+ 定位



CAT

Single Object

目标检测



DOG, DOG, CAT

Multiple Object

示例分割



DOG, DOG, CAT

This image is CC0 public domain

语义分割



GRASS, CAT,
TREE, SKY

No objects, just pixels



CAT

Single Object



DOG, DOG, CAT

Multiple Object



DOG, DOG, CAT

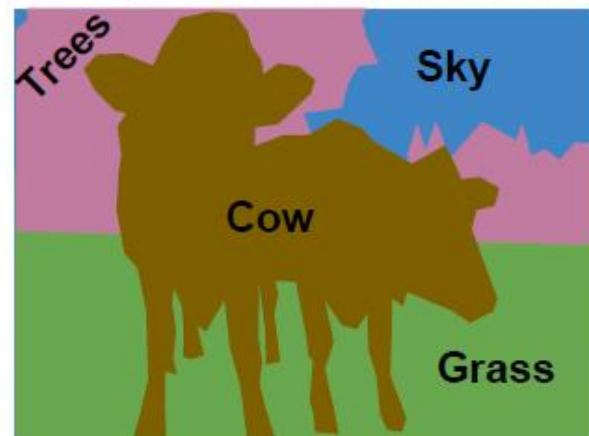
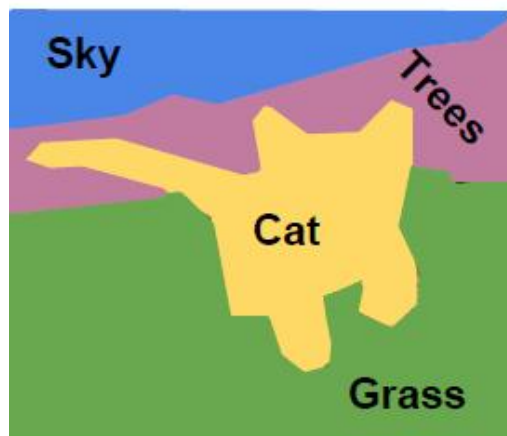
This image is CC0 public domain

语义分割

给每个像素都输出一个值，来代表这个像素属于某一个分类

使用分类标签标记图像中的每个像素

不要区分实例，只关心像素



语义分割

将图像分割成几组具有某种特定语义含义的像素区域，并识别出每个区域的类别，最终获得一幅具有像素语义标注的图像，使图像更容易理解和分析。



语义分割

图像分类

v.s.

目标检测

v.s.

语义分割



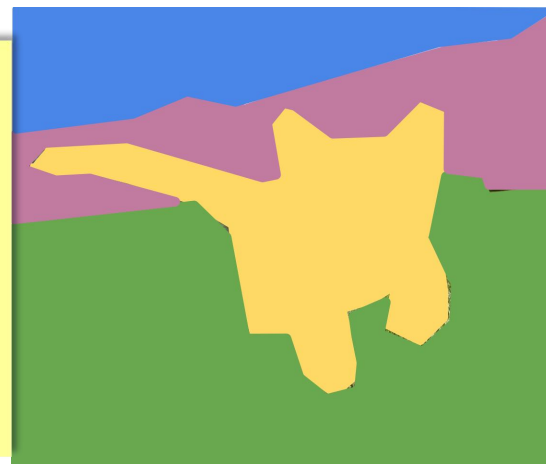
图像主体是猫

区别于图像识别

侧重于语义理解

计算机视觉顶层任务

猫的定位(图像级)



猫的定位(像素级)

语义分割

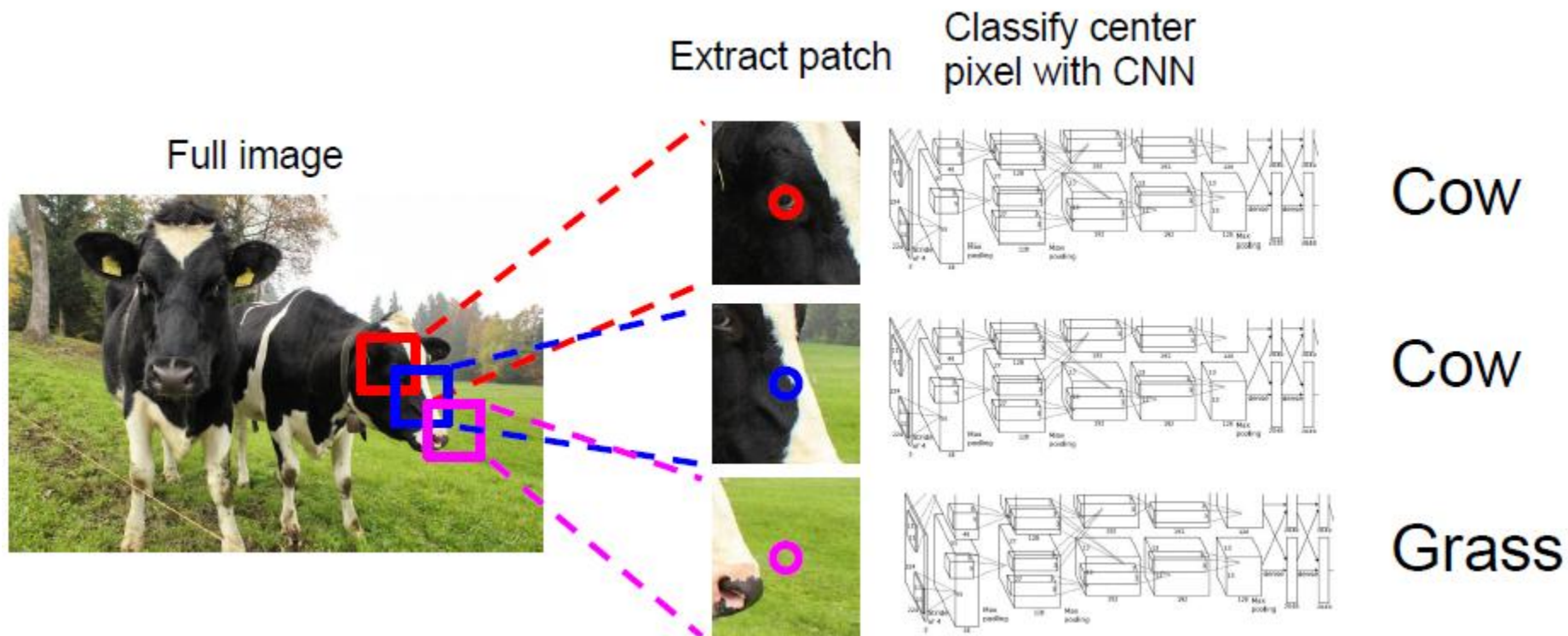
语义分割是图像理解的基石

- 自动驾驶
- 街景识别
- 自动抠图
- ...

精确定位道路、车体、行人、物体信息



语义分割思想：滑动窗口



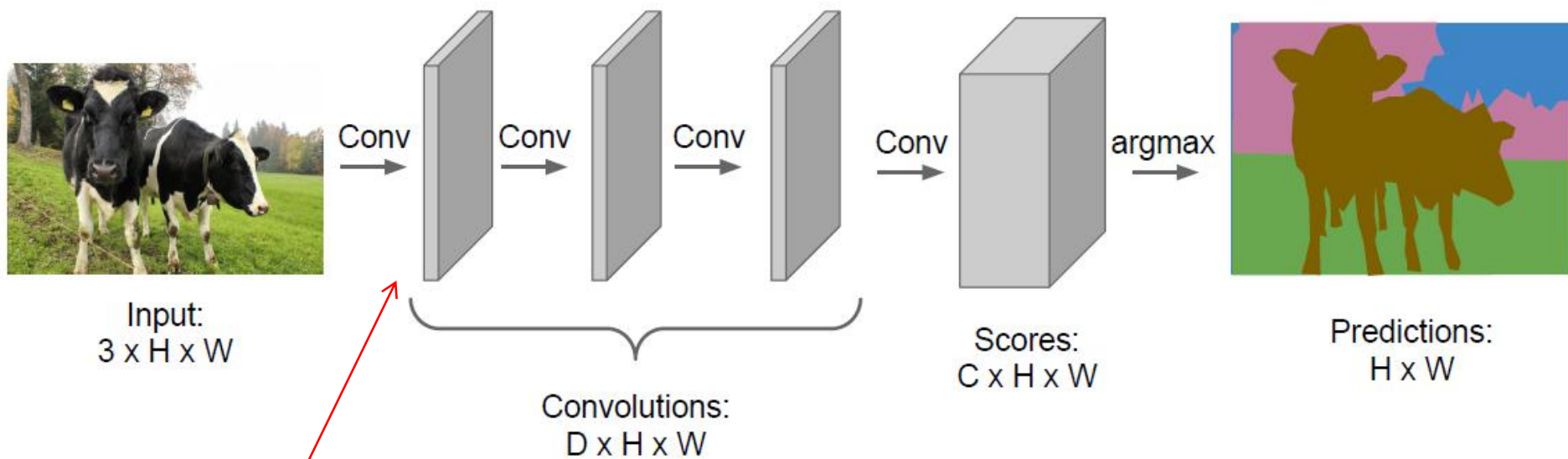
问题：非常低效！不重复使用重叠窗口之间的共享特征

两张非常接近的块，没有利用到他们的相关信息，这样导致的结果就是计算效率十分低

使用滑动窗方法，每次取图像的一部分，使用神经网络判断此部分的中心像素属于哪一类。

语义分割思想：全卷积

将网络设计为一堆卷积图层，一次对像素进行预测！



问题：原始图像分辨率（无采样）的卷积计算复杂度高

训练样本非常**Expensive**

直接输出这张图片每个像素在每个分类上的打分，然后取分数最大的类作为当前像素的分类，

语义分割思想：完全卷积

使用全卷积网络，同时使用降采样与升采样技术

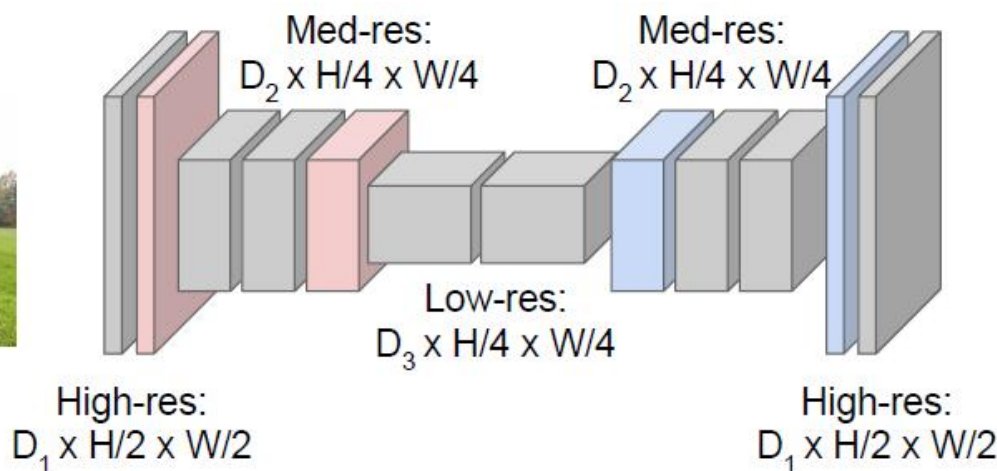
下采样：池化，
步长卷积

将网络设计为一堆卷积层，并在
网络内进行下采样和上采样！

上采样



Input:
 $3 \times H \times W$



Predictions:
 $H \times W$

直接输出这张图片每个像素在每个分类上的
打分，然后取分数最大的类作为当前像素的
分类，

上采样

网络上采样：“Unpooling”

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

“Bed of Nails”

1	2
3	4



1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input: 2 x 2

Output: 4 x 4

上采样有3种常见的方法：双线性插值(bilinear)，反卷积(Transposed Convolution)，反池化(Unpooling)

网络上采样：“Max Unpooling”

Max Pooling

Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4



5	6
7	8

Output: 2 x 2



Rest of the network

Max Unpooling

Use positions from pooling layer

1	2
3	4

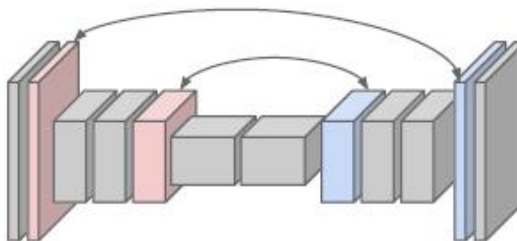
Input: 2 x 2



0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

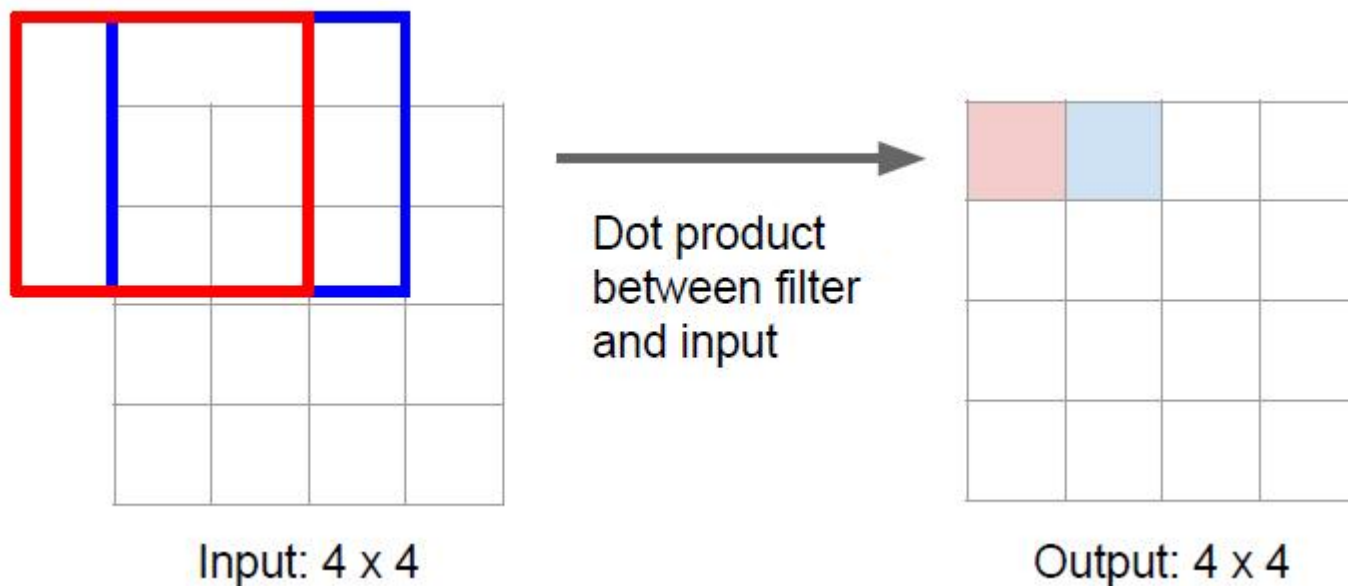
Output: 4 x 4

Corresponding pairs of
downsampling and
upsampling layers



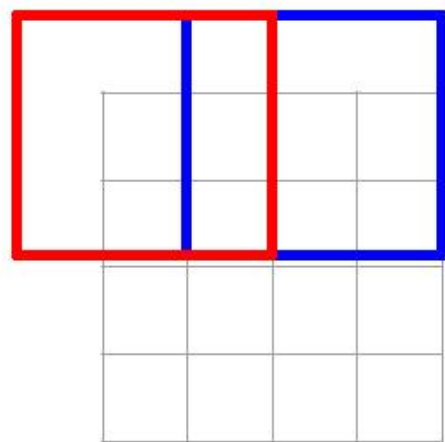
基于学习的上采样：转置卷积

Recall: 标准的 3×3 卷积，步长1，pad1



基于学习的上采样：转置卷积

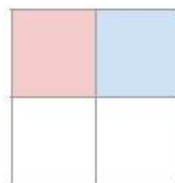
Recall:标准的 3×3 卷积，步长2，pad1



Input: 4 x 4



Dot product
between filter
and input



Output: 2 x 2

滤波器在每个输入像素的输出中移动2个像素

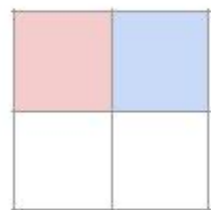
步长给出输入和输出之间的比例

基于学习的上采样：转置卷积

3×3 转置卷积，步长2，pad1

其他名称：

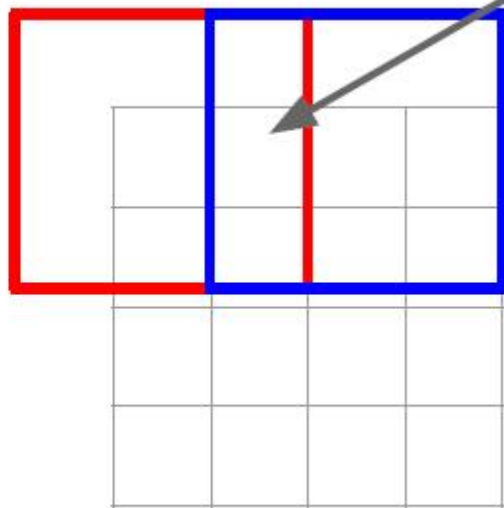
- Deconvolution
- Upconvolution
- Fractionally strided convolution
- Backward strided convolution



Input: 2 x 2



Input gives
weight for
filter



Output: 4 x 4

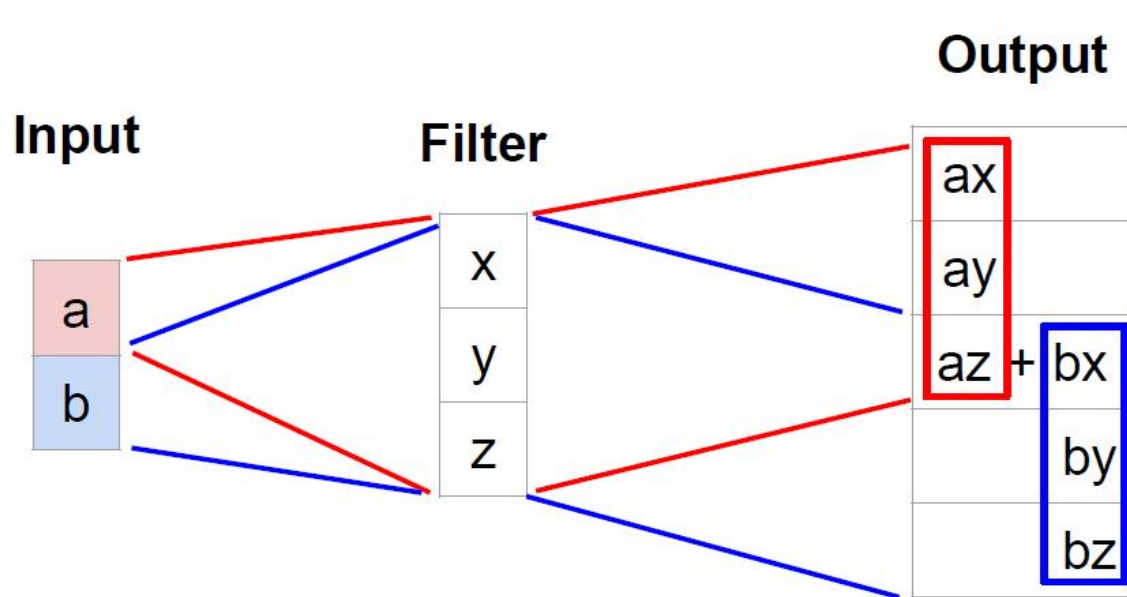
输出重叠部分之和

滤波器在每个输入像素的
输出中移动2个像素

步长给出输入和输出之间
移动的比例

转置卷积：一维的例子

卷积总可以表示成矩阵相乘的形式。而转置卷积，就是将其系数矩阵进行转置



输出包含由输入加权的滤波器的副本，在输出中重叠的地方求和

需要从输出中裁剪一个像素以使输出精确到输入的2倍

3	3	2	1
0	0	1	3
3	1	2	2
2	0	0	2

*

0	1	2
2	2	0
0	1	2

=

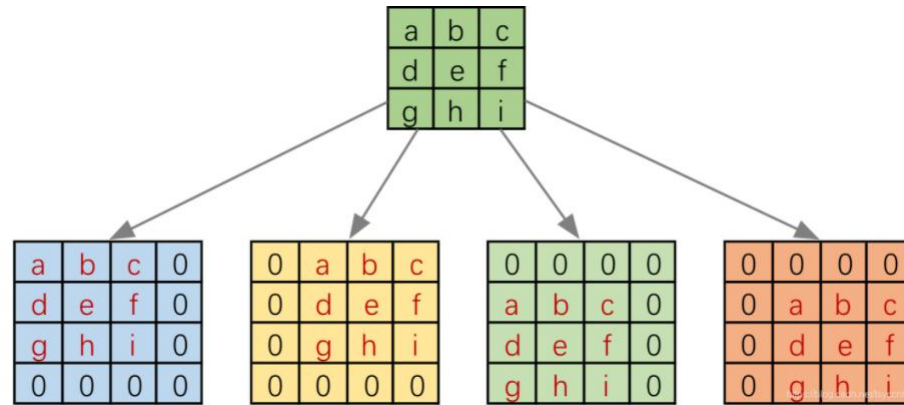
12	12
10	17

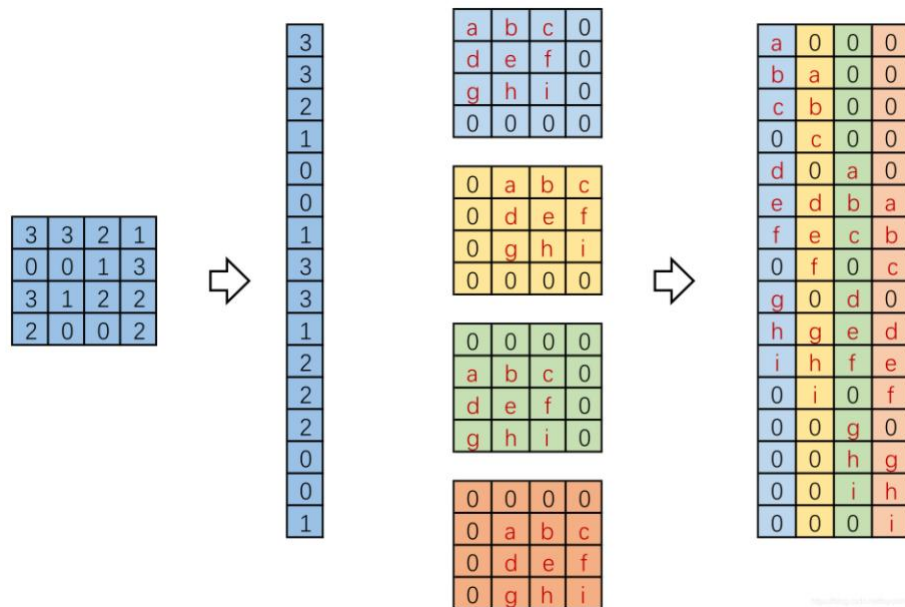
输入

卷积核

输出

实际计算时，并不是像这样一个位置一个位置的进行滑动计算，效率太低。计算机会将卷积核转换成等效的矩阵，将输入转换为向量。通过输入向量和卷积核矩阵的相乘获得输出向量。输出的向量经过整形便可得到我们的二维输出特征。3x3卷积核要在输入上不同的位置卷积4次，所以通过补零的方法将卷积核分别置于一个4x4矩阵的四个角落。输入可以直接和这四个4x4的矩阵进行卷积，舍去了滑动操作。





将输入拉成长向量，四个4x4卷积核也拉成长向量并进行拼接

记向量化的图像为 I ,向量化的卷积矩阵为 C , 输出特征向量为 O

$$I^T * C = O^T$$

3	3	2	1	0	0	1	3	3	1	2	2	2	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

x

a	0	0	0
b	a	0	0
c	b	0	0
0	c	0	0
d	0	a	0
e	d	b	a
f	e	c	b
0	f	0	c
g	0	d	0
h	g	e	d
i	h	f	e
0	i	0	f
0	0	g	0
0	0	h	g
0	0	i	h
0	0	0	i

=

12	12	10	17
----	----	----	----

- 将一个1x16的行向量乘以16x4的矩阵，得到了1x4的行向量。那么反过来将一个1x4的向量乘以一个4x16的矩阵是不是就能得到一个1x16的行向量呢？ **转置卷积**
- 转置卷积的公式

$$O^T * C^T = I^T$$

12	12	10	17
----	----	----	----

×

a	b	c	0	d	e	f	0	g	h	i	0	0	0	0	0
0	a	b	c	0	d	e	f	0	g	h	i	0	0	0	0
0	0	0	0	a	b	c	0	d	e	f	0	g	h	i	0
0	0	0	0	0	a	b	c	0	d	e	f	0	g	h	i

=

-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

两个操作并不是可逆的，对于同一个卷积核，经过转置卷积操作之后并不能恢复到原始的数值，保留的只有原始的形状。

作为矩阵乘法的卷积(一维)

我们可以用矩阵乘法来表示卷积
 x 核, a 输入向量

$$\vec{x} * \vec{a} = X\vec{a}$$

$$\begin{bmatrix} x & y & x & 0 & 0 & 0 \\ 0 & x & y & x & 0 & 0 \\ 0 & 0 & x & y & x & 0 \\ 0 & 0 & 0 & x & y & x \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix}$$

例如：一维卷积，内核
size=3, stride=1, padding=1

作为矩阵乘法的卷积(一维)

我们可以用矩阵乘法来表示卷积
 x 核, a 输入向量

$$\vec{x} * \vec{a} = X \vec{a}$$

$$\begin{bmatrix} x & y & x & 0 & 0 & 0 \\ 0 & x & y & x & 0 & 0 \\ 0 & 0 & x & y & x & 0 \\ 0 & 0 & 0 & x & y & x \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix}$$

例如：一维卷积，内核
size=3, stride=1, padding=1

卷积转置乘以相同矩阵的转置：

$$\vec{x} *^T \vec{a} = X^T \vec{a}$$

$$\begin{bmatrix} x & 0 & 0 & 0 \\ y & x & 0 & 0 \\ z & y & x & 0 \\ 0 & z & y & x \\ 0 & 0 & z & y \\ 0 & 0 & 0 & z \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} ax \\ ay + bx \\ az + by + cx \\ bz + cy + dx \\ cz + dy \\ dz \end{bmatrix}$$

当步长 = 1时，卷积转置只是一个正则卷积（具有不同的填充规则）

语义分割思想：完全卷积

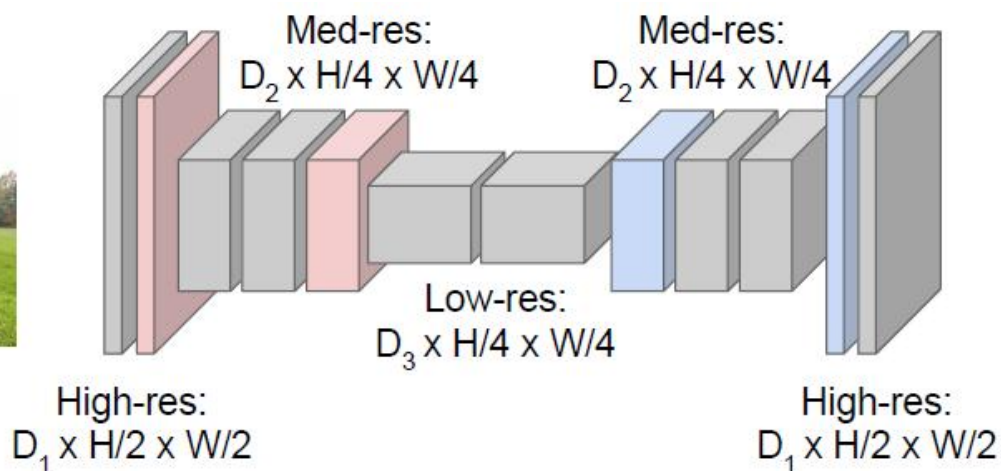
下采样：池化，
步长卷积

将网络设计为一堆卷积层，并在
网络内进行下采样和上采样！

采样：
解卷或步长的
转置卷积



Input:
 $3 \times H \times W$



Predictions:
 $H \times W$

分类 + 定位



GRASS, CAT,
TREE, SKY

No objects, just pixels



CAT

Single Object



DOG, DOG, CAT

Multiple Object



DOG, DOG, CAT

This image is CC0 public domain

分类 + 定位

图像识别 (classification) :

输入: 图片

输出: 物体的类别

评估方法: 准确率



CAT

定位 (localization) :

输入: 图片

输出: 方框在图片中的位置 (x, y, w, h)

评估方法: 检测评价函数 intersection-over-union (IOU)

(x, y)



$h, height,$
高度



(x, y, w, h)

分类 + 定位

卷积神经网络CNN可实现图像识别（判定是猫还是狗）的任务，只需要添加一些额外的功能来完成定位任务即可。

定位的问题的解决思路有哪些？



CAT

(x, y)



$h, height,$
高度

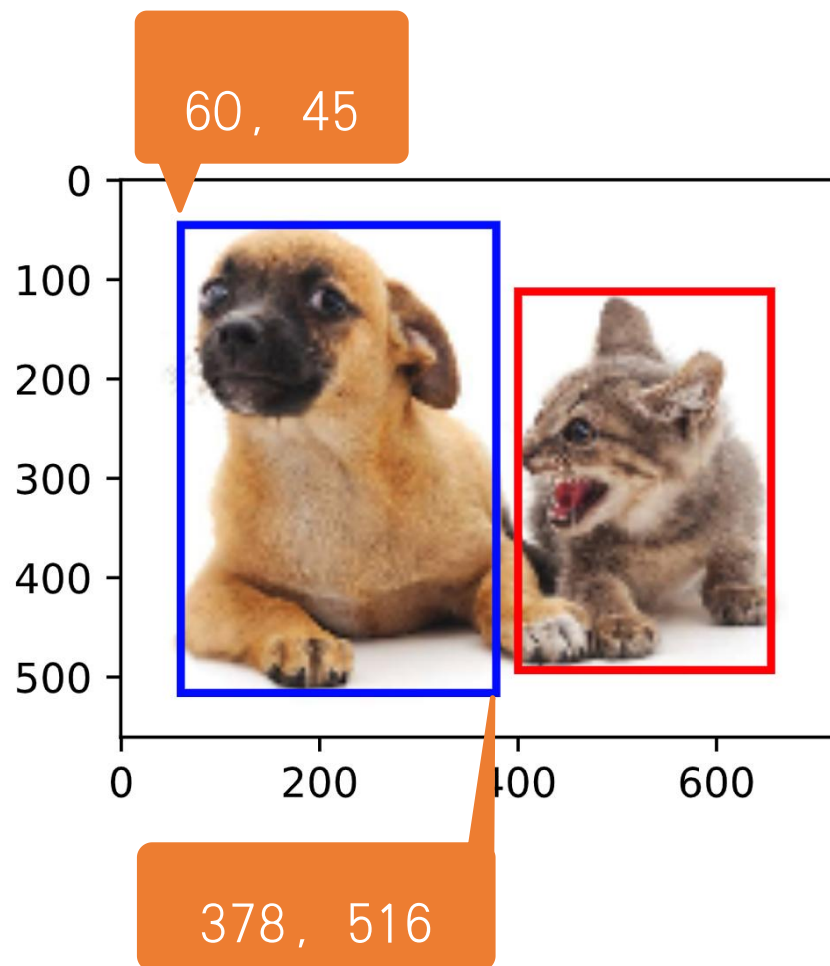


(x, y, w, h)

$w, width,$ 宽度

边界框

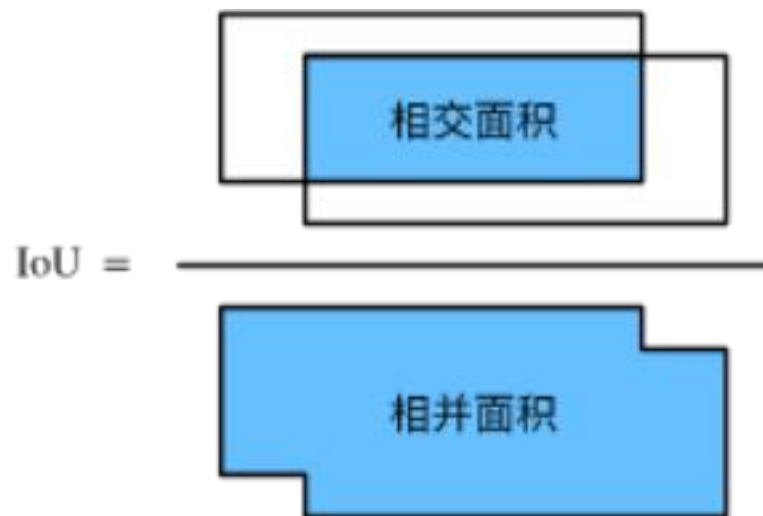
- 一个边界框可以由 4 个数字定义：
 - (左上角 x , 左上角 y , 右下角 x , 右下角 y)
 - (左上角 x , 右上角 y , 宽度, 高度)



交并比

- 交并比 (IoU) 测量两个框之间的相似性:
 - 0 表示不重叠
 - 1 表示完全相同
- Jaccard 系数, 给定集A和B:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$



定位

定位的问题的解决思路

思路一：看做回归问题

需要预测出 (x,y,w,h) 四个参数的值，从而得出方框的位置

Input: image



Neural Net



Output:

Box coordinates
(4 numbers)

Correct output:
box coordinates
(4 numbers)



Loss:

L2 distance

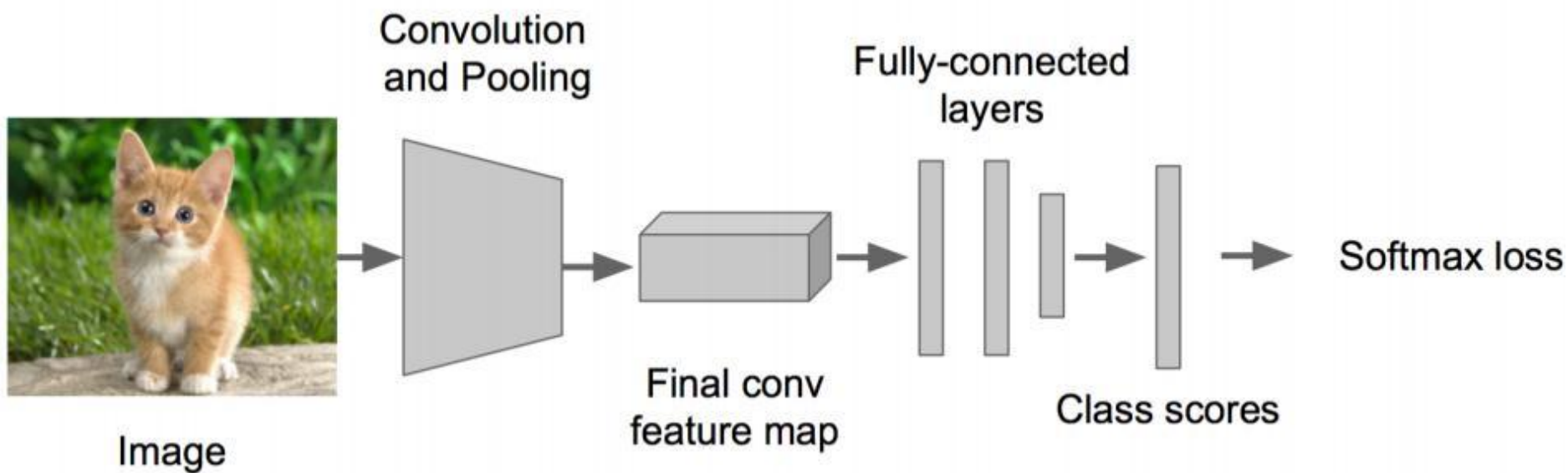
用欧氏距离作损失函数

Only one object,
simpler than detection

定位

步骤1:

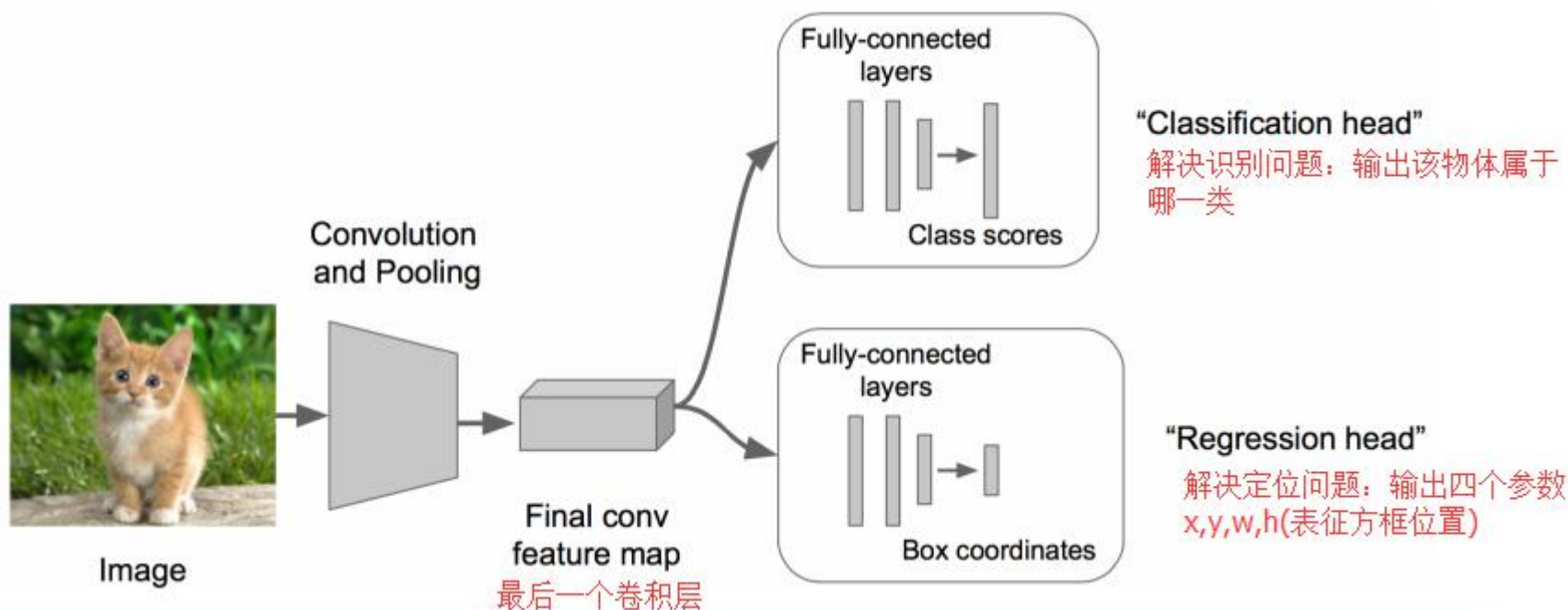
- 先解决简单问题，搭一个识别图像的神经网络
- 在AlexNet VGG GoogleLenet上fine-tuning一下



定位

步骤2:

- 在上述神经网络的尾部展开（也就说CNN前面保持不变，对CNN的结尾处作出改进：加了两个头：“分类头”和“回归头”）
- 成为classification + regression模式



定位

步骤3:

- Regression那个部分用欧氏距离损失
- 使用SGD训练

步骤4:

- 预测阶段把2个头部拼上
- 完成不同的功能

这里需要进行两次fine-tuning

第一次在ALexNet上做，第二次将头部改成regression head，前面不变，做一次fine-tuning

定位

Regression的部分加在哪?

有两种处理方法:

- 加在最后一个卷积层后面 (如VGG)
- 加在最后一个全连接层后面 (如R-CNN)

Regression不好做, 应想方设法转换为classification问题。
regression的训练参数收敛的时间要长得多, 所以上面的网络采取了用classification的网络来计算出网络共同部分的连接权值。

定位

定位的问题的解决思路

思路二：取图像窗口

- classification + regression思路
- 取不同的大小的“框”
- 让框出现在不同的位置，得出这个框的判定得分
- 取得分最高的那个框

Input: image



Neural Net



Output:

Box coordinates
(4 numbers)

Correct output:
box coordinates
(4 numbers)



Loss:

L2 distance

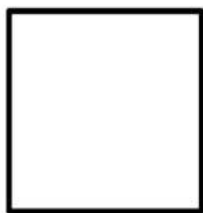
用欧氏距离作损失函数

Only one object,
simpler than detection

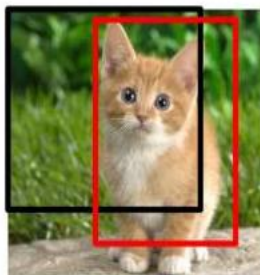
定位

定位的问题的解决思路

思路二：取图像窗口



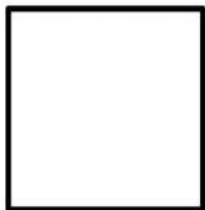
Network input:
 $3 \times 221 \times 221$



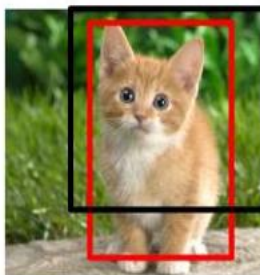
Larger image:
 $3 \times 257 \times 257$

0.5	

Classification scores:
 $P(\text{cat})$



Network input:
 $3 \times 221 \times 221$



Larger image:
 $3 \times 257 \times 257$

0.5	0.75

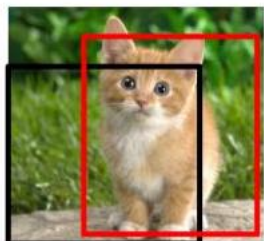
Classification scores:
 $P(\text{cat})$

定位

定位的问题的解决思路



Network input:
3 x 221 x 221



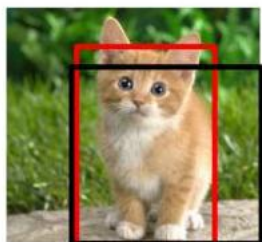
Larger image:
3 x 257 x 257

0.5	0.75
0.6	

Classification scores:
P(cat)



Network input:
3 x 221 x 221



Larger image:
3 x 257 x 257

0.5	0.75
0.6	0.8

Classification scores:
P(cat)

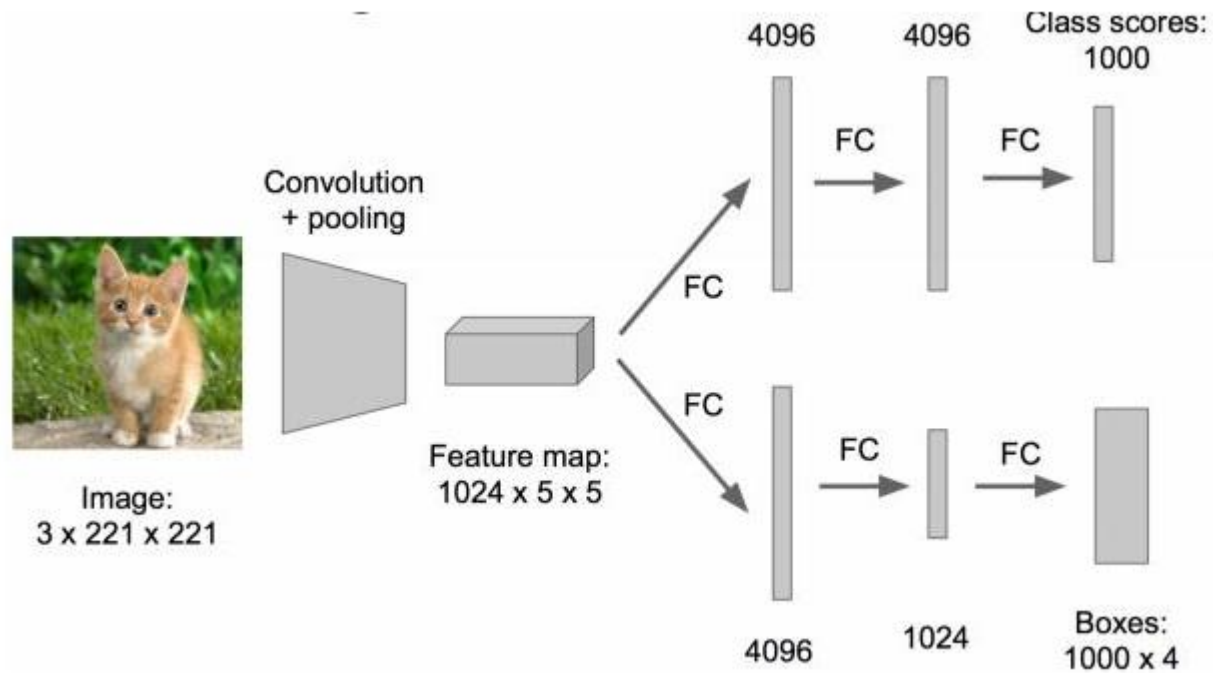
思路二：取图像窗口

根据得分的高低，选择了右下角的黑框作为目标位置的预测。
注：有的时候也会选择得分最高的两个框，然后取两框的交集作为最终的位置预测。

Q：框要取多大？
取不同的框，依次从左上角扫到右下角。

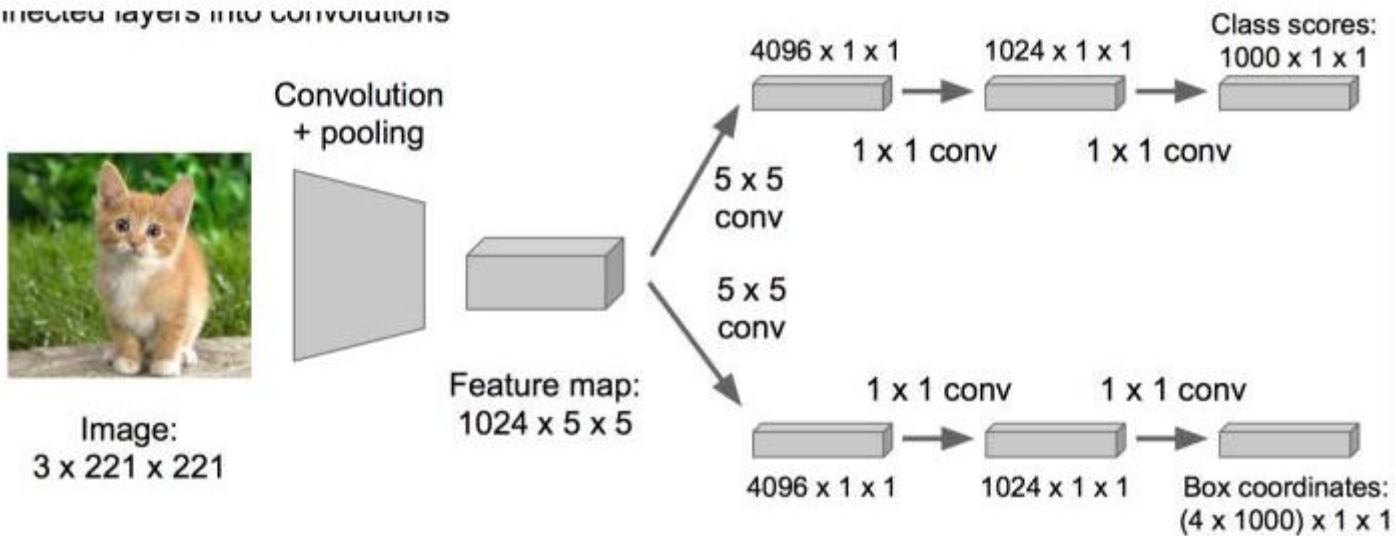
小结：
对一张图片，用各种大小的框（遍历整张图片）将图片截取出来，输入到CNN，然后CNN会输出这个框的得分（classification）以及这个框图片对应的x,y,h,w（regression）。

分类+定位



分类+定位

connected layers into convolutions

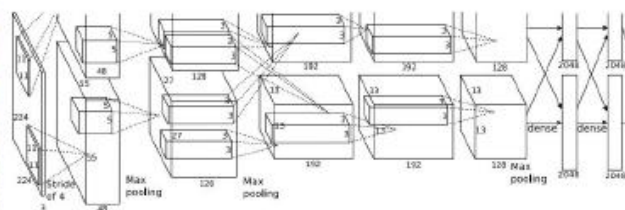


Single Object

分类 + 定位



This image is CC0 public domain



Fully
Connected:
4096 to 1000

Class Scores

Cat: 0.9
Dog: 0.05
Car: 0.01
...

Correct label:
Cat

Softmax
Loss

Multitask Loss

Vector:
4096

Fully
Connected:
4096 to 4

Box
Coordinates
(x, y, w, h)

L2 Loss

Correct box:
(x', y', w', h')

Treat localization as a
regression problem!

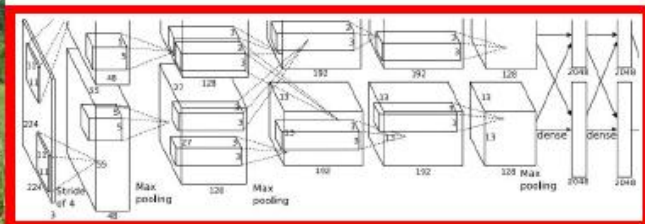
+

Loss

分类 + 定位



This image is CC0 public domain



Often pretrained on ImageNet
(Transfer learning)

Vector:
4096

Fully
Connected:
4096 to 1000

Class Scores

Cat: 0.9
Dog: 0.05
Car: 0.01
...

Correct label:
Cat

**Softmax
Loss**

+ → **Loss**

Fully
Connected:
4096 to 4

**Box
Coordinates**
(x, y, w, h)

L2 Loss

Correct box:
(x', y', w', h')

Treat localization as a
regression problem!

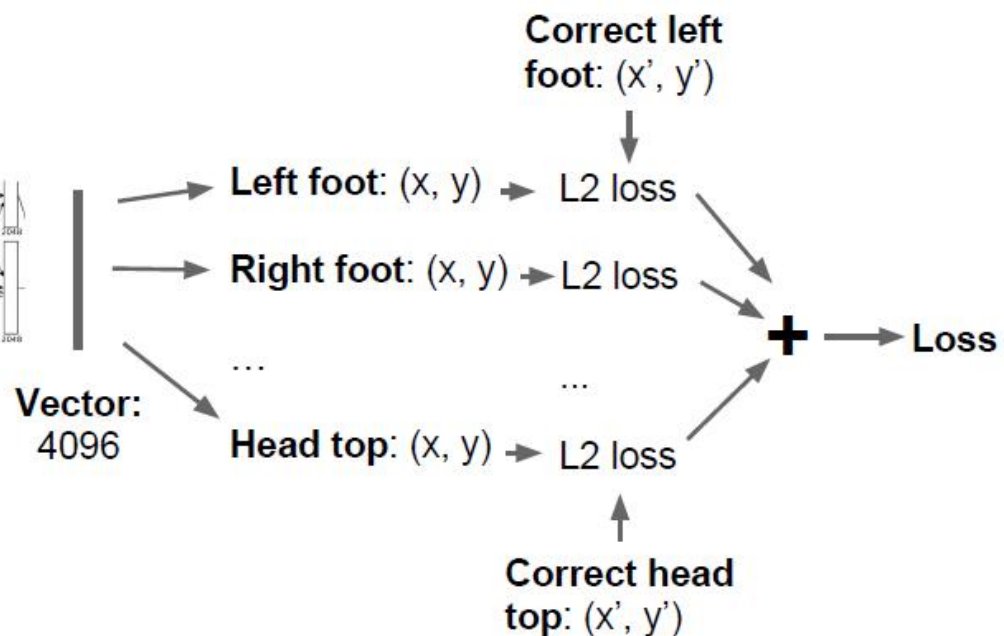
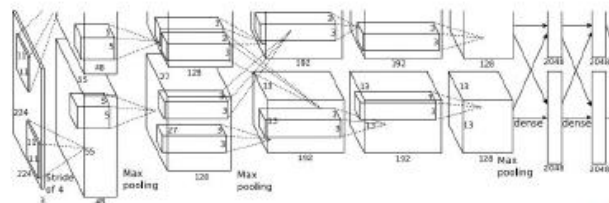
Aside: 人体姿势评估



将姿势表示为一组14个
关节位置:

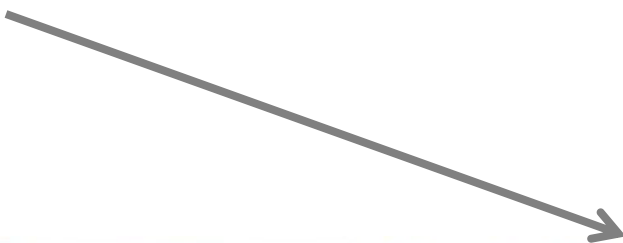
- Left / right foot
- Left / right knee
- Left / right hip
- Left / right shoulder
- Left / right elbow
- Left / right hand
- Neck
- Head top

Aside: 人体姿势评估



Toshev and Szegedy, "DeepPose: Human Pose Estimation via Deep Neural Networks", CVPR 2014

目标检测



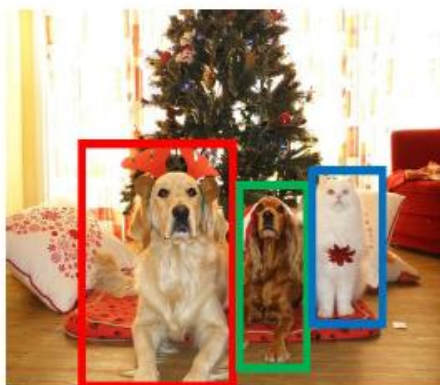
GRASS, CAT,
TREE, SKY

No objects, just pixels



CAT

Single Object



DOG, DOG, CAT

Multiple Object



DOG, DOG, CAT

This image is CC0 public domain

目标检测：深度学习的影响

传统的目标检测方法大概分为区域选择、特征提取（SIFT、HOG等）、分类器（SVM等）三个部分，其主要问题有两方面：一是区域选择策略没有针对性、时间复杂度高，窗口冗余；另一方面手工设计的特征鲁棒性较差。

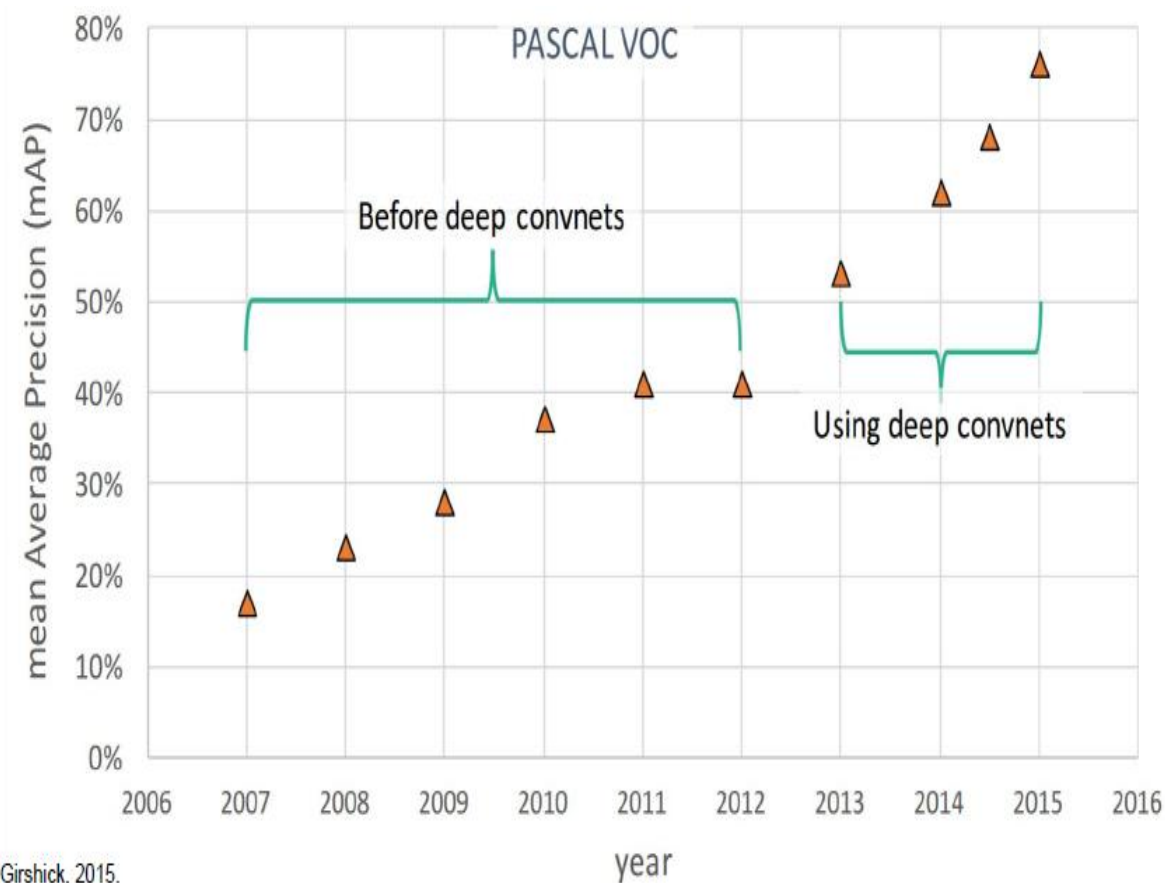
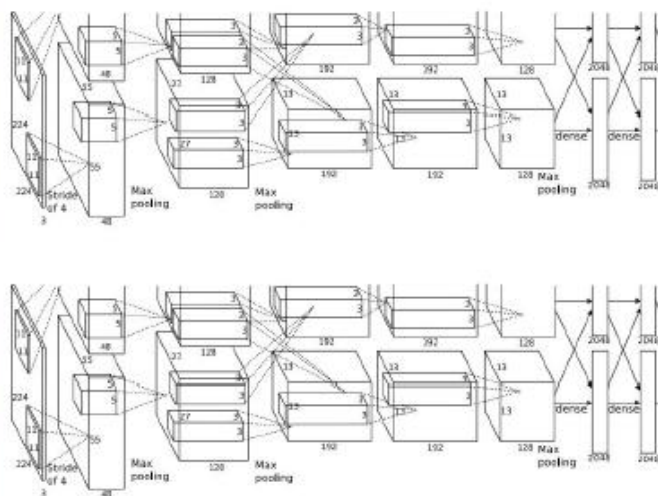


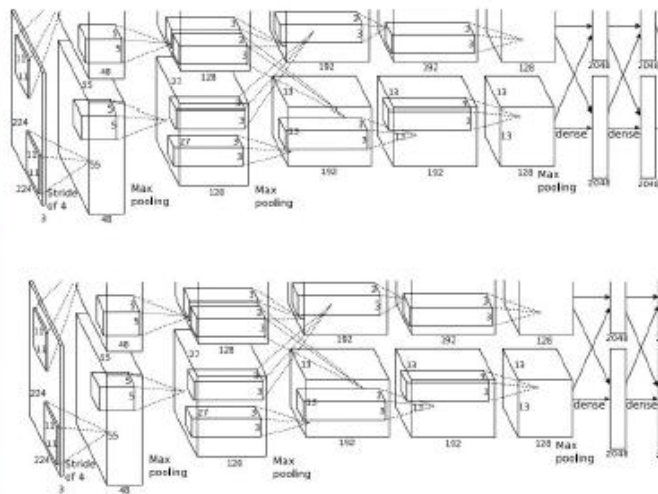
Figure copyright Ross Girshick, 2015.
Reproduced with permission.

回归的目标检测?

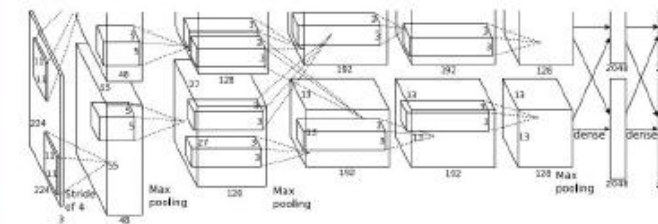
每个图像需要不同数量的输出!



CAT: (x, y, w, h) 4 numbers



DOG: (x, y, w, h)
DOG: (x, y, w, h) 16
CAT: (x, y, w, h) numbers



DUCK: (x, y, w, h) Many
DUCK: (x, y, w, h) numbers!

....

回归的目标检测?

多物体识别+定位多个物体



CAT? NO

DOG? NO



CAT? YES!

DOG? NO



CAT? NO

DOG? NO

看成分类问题有何不妥?

- 需要找很多位置, 给很多个不同大小的框
- 还需要对框内的图像分类

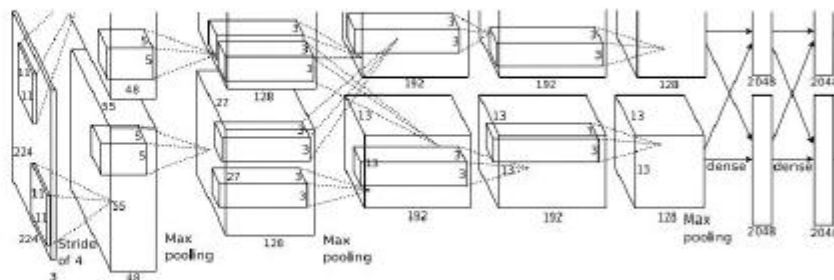
看做classification

框和位置很多!

方法: 找出可能含有物体的框 (也就是候选框, 比如选1000个候选框), 这些框之间是可以互相重叠互相包含的

分类的目标检测：滑动窗口

将CNN应用于许多不同的图像分割，CNN将每块分割分类为目标或背景



Dog? NO
Cat? NO
Background? YES

分类的目标检测：滑动窗口

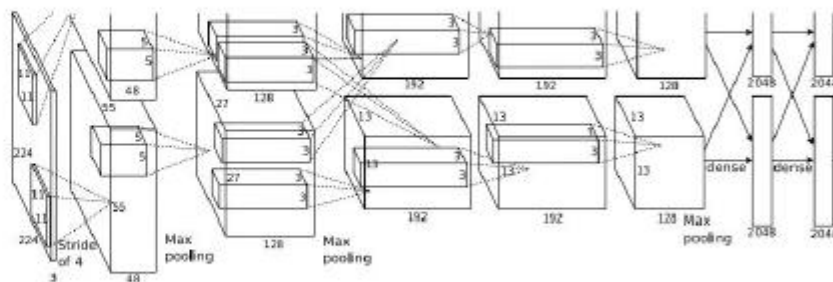
将CNN应用于许多不同的图像分割，CNN将每块分割分类为目标或背景



Dog? YES

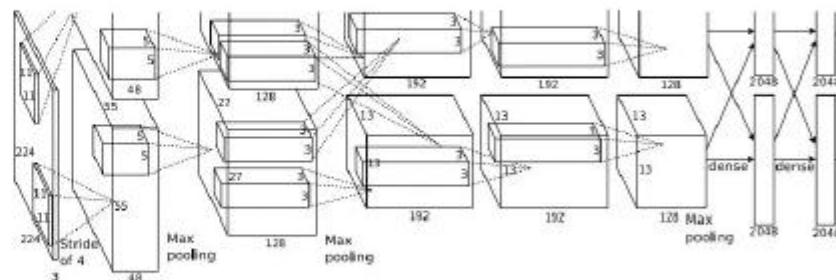
Cat? NO

Background? NO



分类的目标检测：滑动窗口

将CNN应用于许多不同的图像分割，CNN将每块分割分类为目标或背景



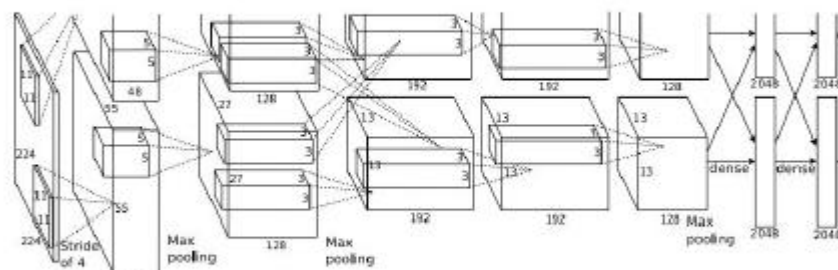
Dog? YES
Cat? NO
Background? NO

多个目标检测：滑动窗口

将CNN应用于许多不同的图像分割，CNN将每块分割分类为目标或背景



Dog? NO
Cat? YES
Background? NO

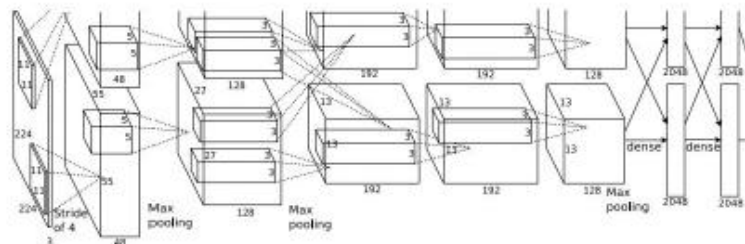
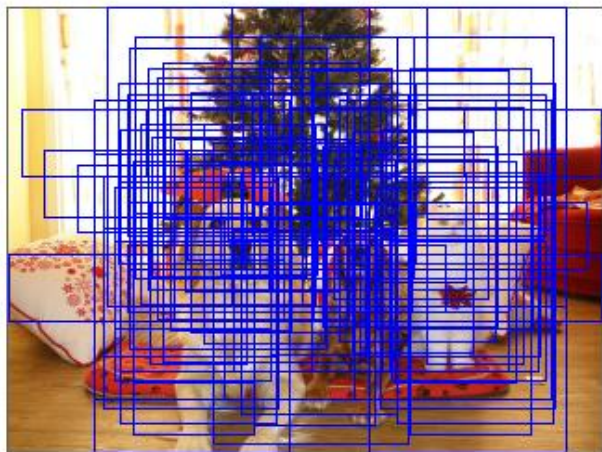


问题：计算量非常大！

但是由于物体大小不同，我们需要不同位置、不同长宽比、不同大小的窗口，计算量难以接受

多个目标检测：滑动窗口

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

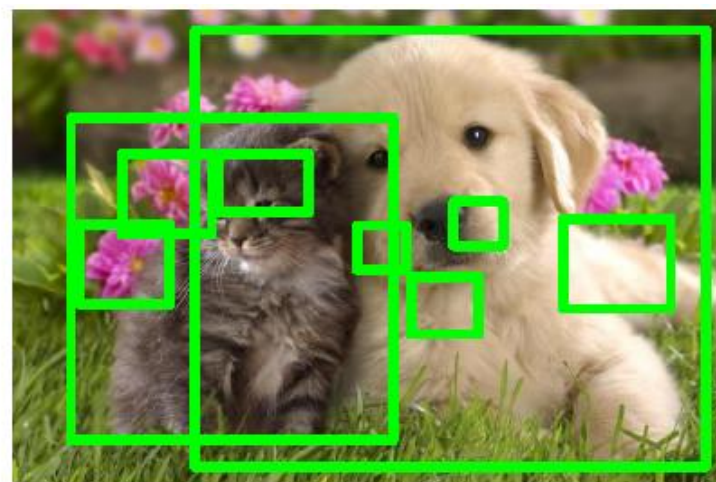


Problem: Need to apply CNN to huge number of locations, scales, and aspect ratios, very computationally expensive!

Region Proposals

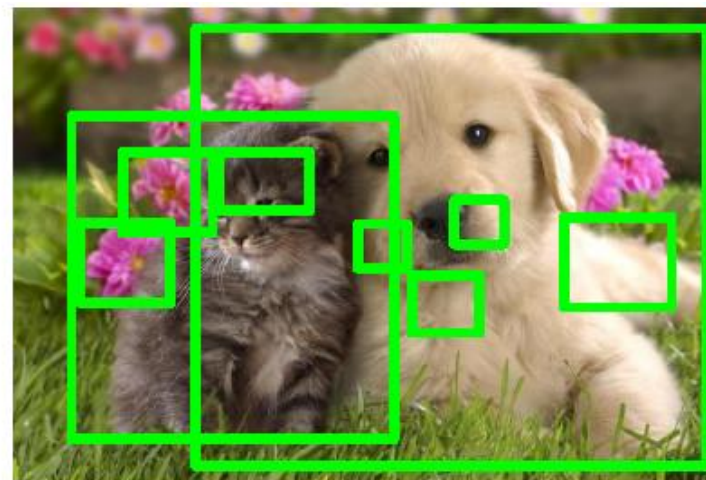
找到可能包含目标“blobby”的图像区域

运行相对快，例如，选择性搜索可在几秒钟内在CPU上提供1000个region proposals



计算机视觉中，采用候选区域，输入一张图，输出1000个区域，尽管这1000个区域中不一定都含有物体，但是图片中的物体大部分都会在这1000个区域中的某一个区域中。这种方法没用到神经网络，但是相对来说耗时非常短。

Region Proposals

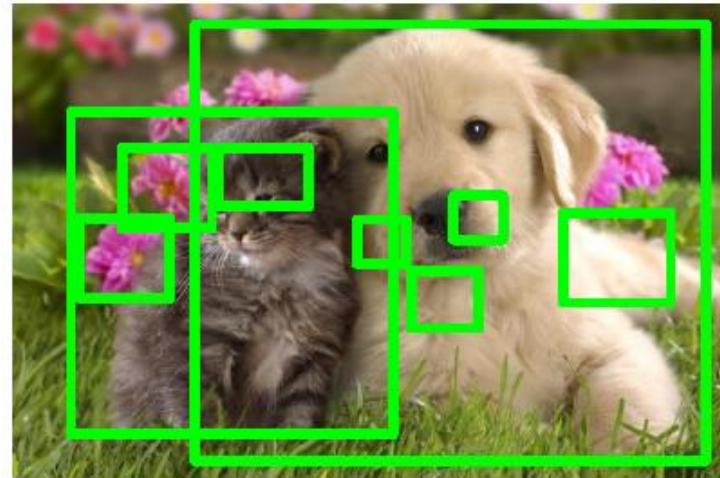


Method	Approach	Outputs Segments	Outputs Score	Control #proposals	Time (sec.)	Repeatability	Recall Results	Detection Results
Bing [18]	Window scoring		✓	✓	0.2	***	*	.
CPMC [19]	Grouping	✓	✓	✓	250	-	**	*
EdgeBoxes [20]	Window scoring		✓	✓	0.3	**	***	***
Endres [21]	Grouping	✓	✓	✓	100	-	***	**
Geodesic [22]	Grouping	✓		✓	1	*	***	**
MCG [23]	Grouping	✓	✓	✓	30	*	***	***
Objectness [24]	Window scoring		✓	✓	3	.	*	.
Rahtu [25]	Window scoring		✓	✓	3	.	.	*
RandomizedPrim's [26]	Grouping	✓		✓	1	*	*	**
Rantalankila [27]	Grouping	✓		✓	10	**	.	**
Rigor [28]	Grouping	✓		✓	10	*	**	**
SelectiveSearch [29]	Grouping	✓	✓	✓	10	**	***	***
Gaussian				✓	0	.	.	*
SlidingWindow				✓	0	***	.	.
Superpixels		✓			1	*	.	.
Uniform				✓	0	.	.	.

Region Proposals: Selective Search

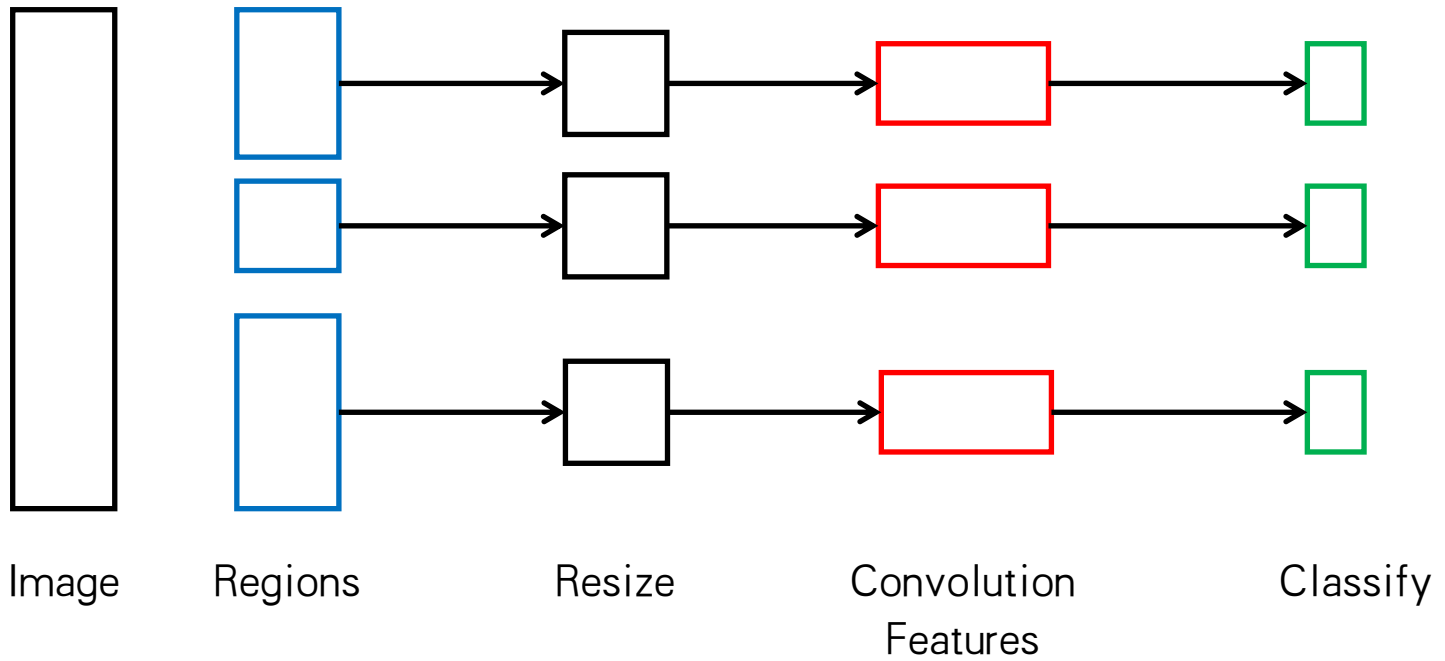
找到可能包含目标“blobby”的图像区域

运行相对快，例如，选择性搜索可在几秒钟内在CPU上提供1000个region proposals



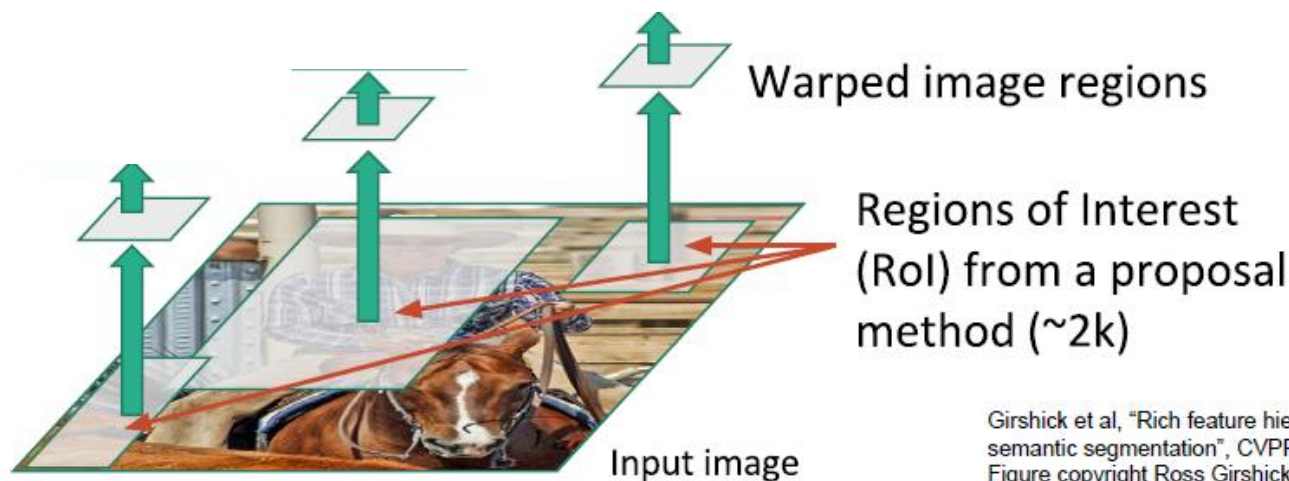
计算机视觉中，采用候选区域，输入一张图，输出1000个区域，尽管这1000个区域中不一定都含有物体，但是图片中的物体大部分都会在这1000个区域中的某一个区域中。这种方法没用到神经网络，但是相对来说耗时非常短。

R-CNN



R-CNN

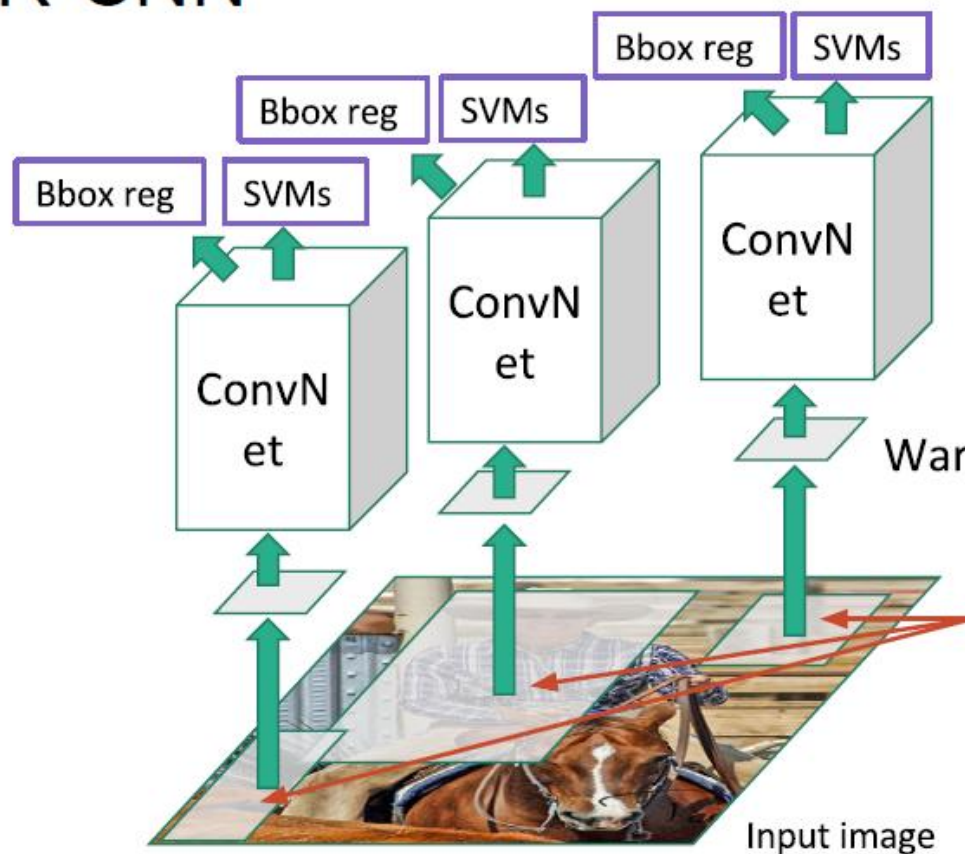
- ✓ 采用CNN网络提取图像特征，从经验驱动的人造特征范式HOG、SIFT到数据驱动的代表学习范式，提高特征对样本的表示能力；
- ✓ 采用大样本下有监督预训练+小样本微调的方式解决小样本难以训练甚至过拟合等问题（迁移学习）



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

R-CNN

R-CNN



Linear Regression for bounding box offsets

Classify regions with SVMs

Forward each region through ConvNet

Warped image regions

Regions of Interest (RoI) from a proposal method (~2k)

先把这1000个区域变为同样的大小，然后输入到CNN中，CNN中有两个输出，一个是候选区域的分类（这其中加入了一个背景类，代表什么物体都没有），另外一个个bounding box坐标的矫正参数，来使bounding box更好地选中物体

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.



Selective Search算法

1. 使用 Efficient Graph-Based Image Segmentation的方法获取原始分割区域

$$R = \{r_1, r_2, \dots, r_n\}。$$

区域内间距区域为对应最小生成树中权重最大的边的权重值。区域间间距即在所有分别属于两个区域且有边连接的点对中，寻找权重最小的那对（若两个区域内的点没有边相连，则定义间距为正无穷大）。

2. 初始化相似度集合 $S = \emptyset$ 。

3. 计算两两相邻区域之间的相似度将其添加到相似度集合 S 中。

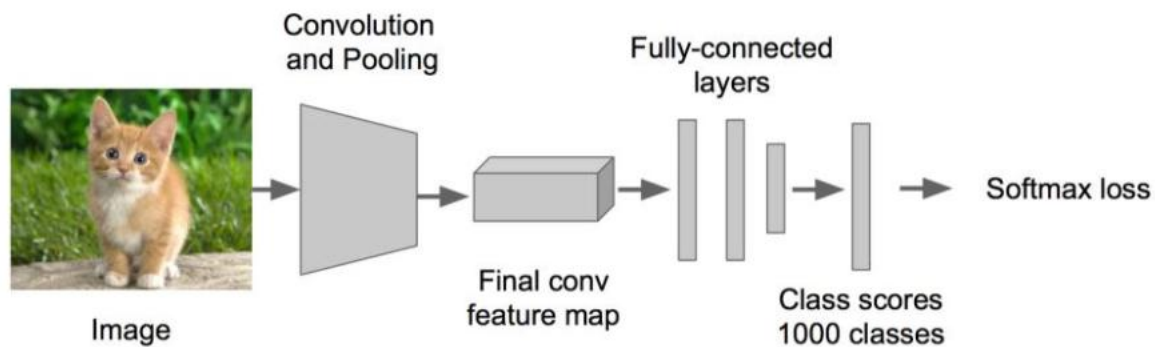
$$s(r_i, r_j) = a_1 S_{colour}(r_i, r_j) + a_2 S_{texture}(r_i, r_j) + a_3 S_{size}(r_i, r_j) + a_4 S_{fill}(r_i, r_j)$$

4. 从相似度集合 S 中找出，相似度最大的两个区域 r_i 和 r_j ，将其合并成为一个区域 r_t ，从相似度集合中除去原先与 r_i 和 r_j 相邻区域之间计算的相似度，计算 r_t 与其相邻区域（原先与 r_i 或 r_j 相邻的区域）的相似度，将其结果添加的到相似度集合 S 中。同时将新区域 r_t 添加到区域集合 R 中。

5. 获取每个区域的 Bounding Boxes，这个结果就是物体位置的可能结果 L 。

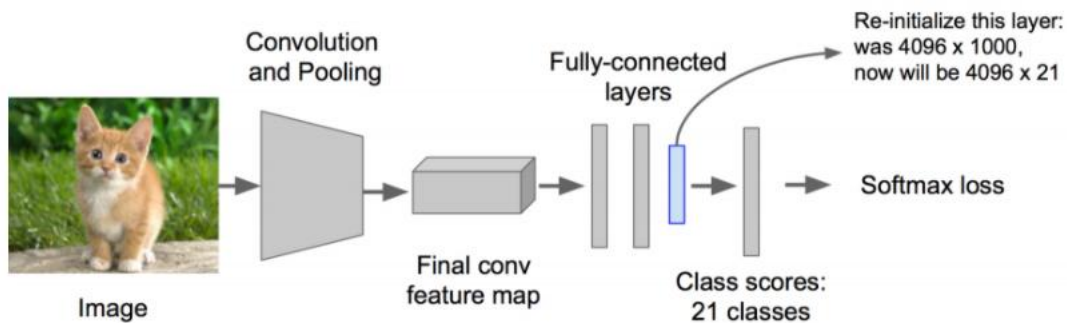
R-CNN

步骤一：训练（或者下载）一个分类模型（比如AlexNet）



步骤二：对该模型做fine-tuning

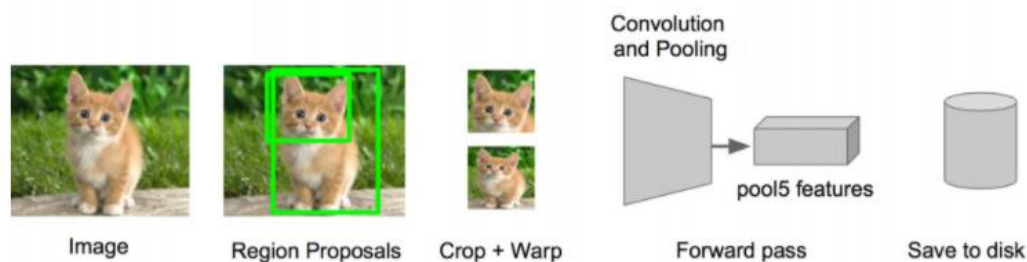
- 将分类数从1000改为20
- 去掉最后一个全连接层



R-CNN

步骤三：特征提取

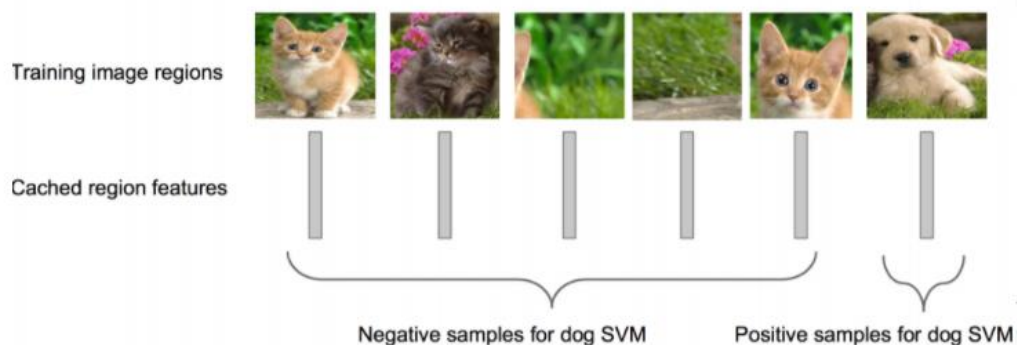
- 提取图像的所有候选框（选择性搜索）
- 对于每一个区域：修正区域大小以适合CNN的输入，做一次前向运算，将第五个池化层的输出（就是对候选框提取到的特征）存到硬盘



步骤四：训练一个SVM分类器（二分类）来判断这个候选框里物体的类别

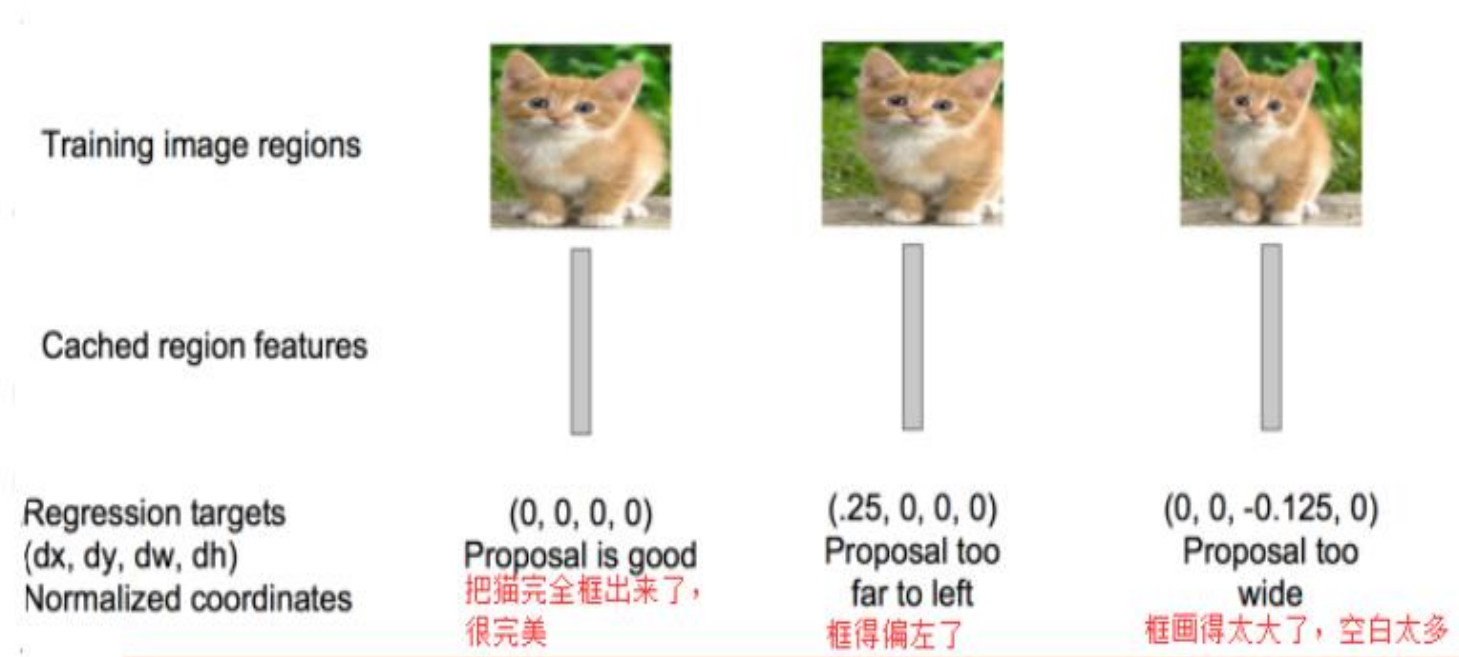
每个类别对应一个SVM，判断是不是属于这个类别，是就是positive，反之negative

比如下图，就是狗分类的SVM



R-CNN

步骤五：使用回归器精细修正候选框位置：对于每一个类，训练一个线性回归模型去判定这个框是否框得完美。



R-CNN

R-CNN存在的问题

- 1、基于R-CNN目标检测算法只能输入固定尺寸的图片，样本输入受限
- 2、经人工处理过的图片，易降低网络识别检测精度
- 3、R-CNN需对各候选区域进行一次卷积操作，计算量大，耗时长



R-CNN: 问题与挑战

- 训练目标

用softmax分类器微调网络（对数损失）

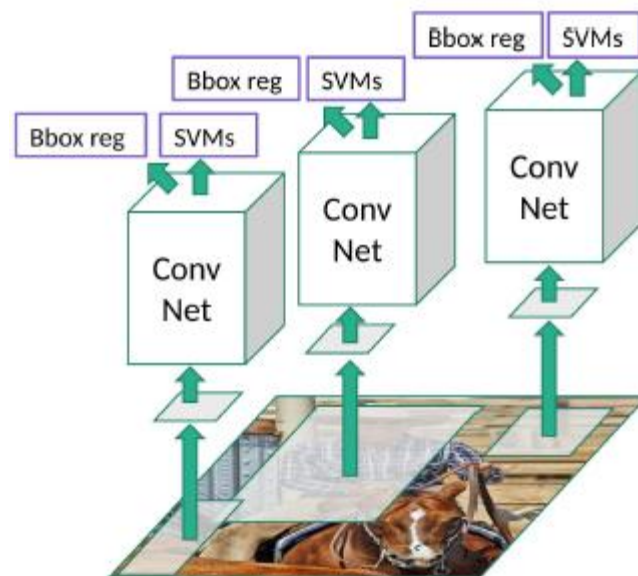
训练线性支持向量机（回归损失）

训练边界框回归（最小二乘）

- 训练速度慢（84小时） 占用大量磁盘空间

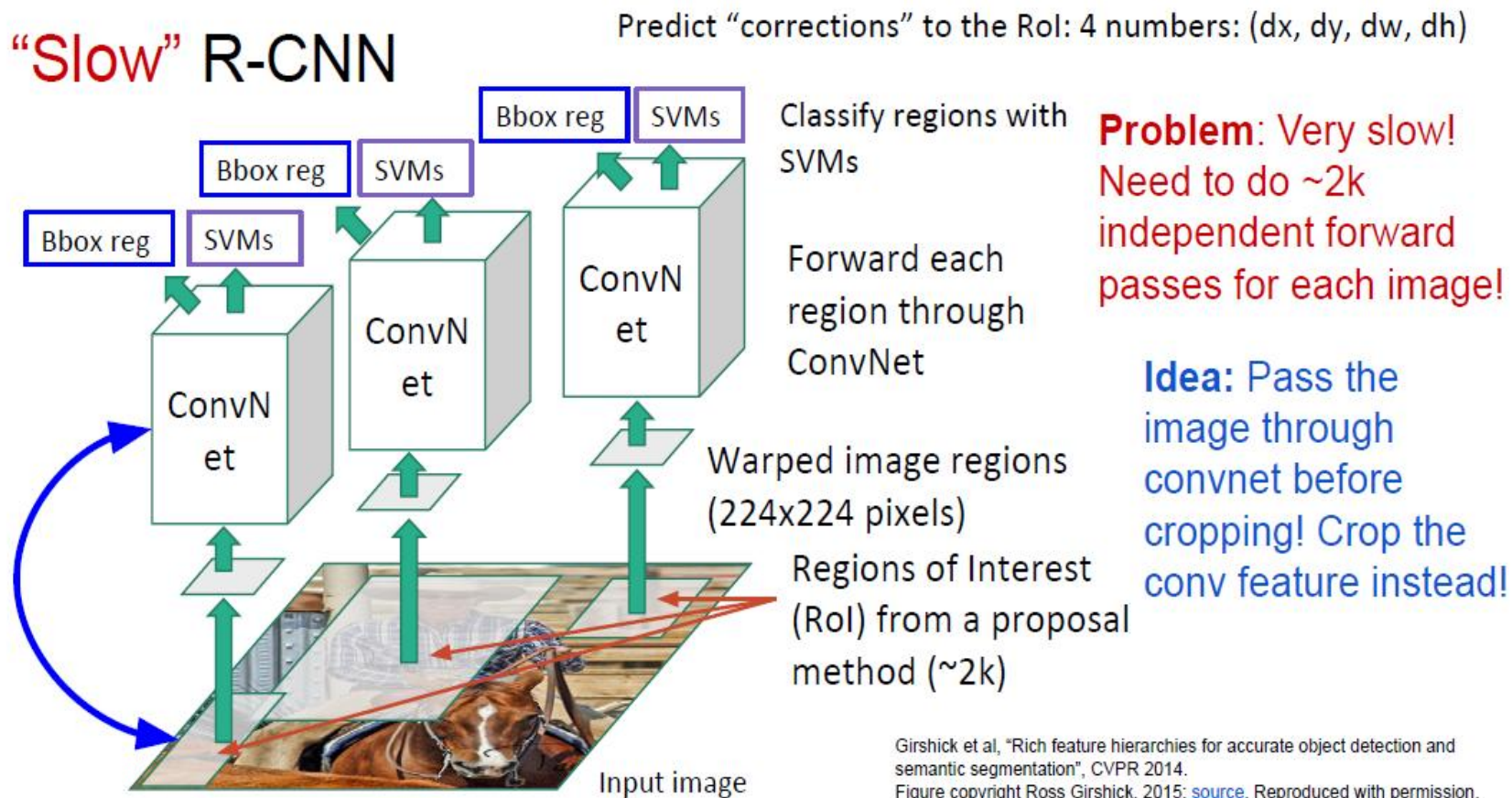
- 推理（检测）很慢47s

问题: Very slow!
Need to do ~2k
independent forward
passes for each image!



R-CNN: 问题与挑战

“Slow” R-CNN





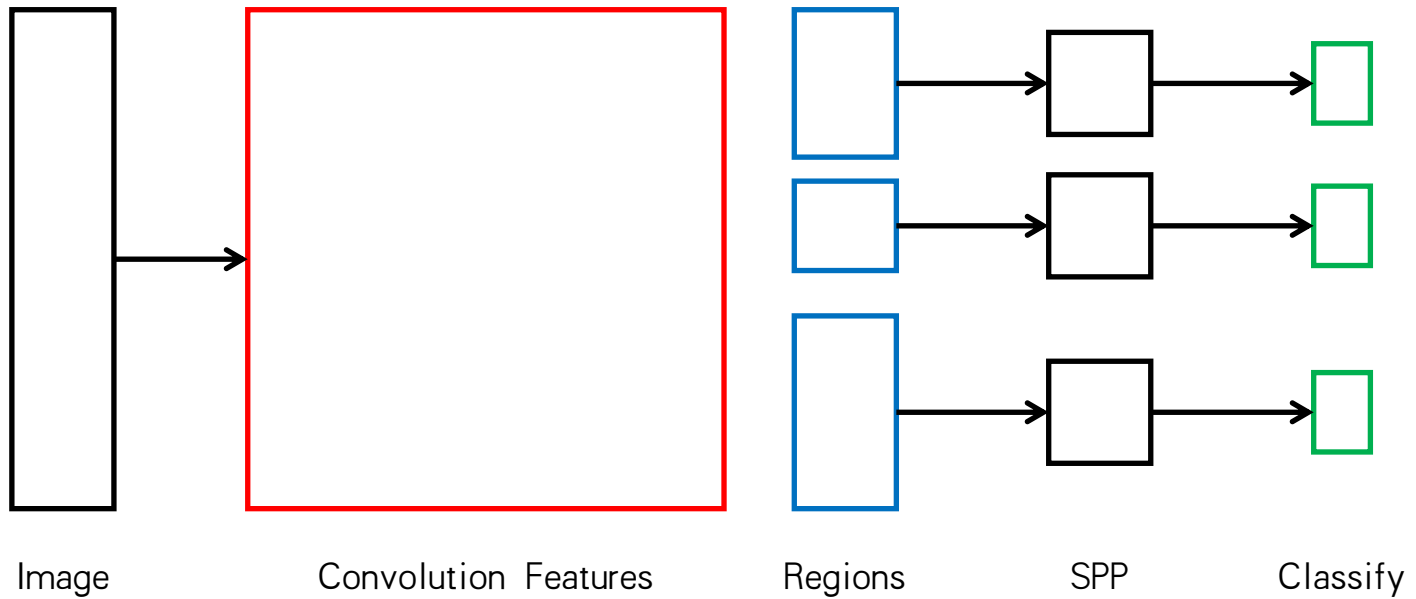
SPP NET

SPP-Net (Spatial Pyramid Pooling) 是何凯明2014年提出的方法，通过解决传统CNN无法处理不同尺寸输入的问题对同年的R-CNN算法做改进，实验结果表明SPP方法比R-CNN快了近100倍

从算法架构上，SPP-Net与R-CNN相似：通过Selective Search获取候选区域，最后也是使用SVM做分类。

但不再将每个候选区域过一次CNN，而是将原始图过一次CNN，在CNN的全连接层前添加新提出的SPP层，根据候选区域位置crop的图像卷积结果通过SPP层来确保输入全连接层的尺寸满足要求。最后在全连接层的输出一性获得所有候选区域的特征向量。

SPP net



SPP—Net: Spatial Pyramid Pooling (空间金字塔池化)

R—CNN中单幅图片大约有2000个region proposal，重复用CNN为每个region proposal提取特征是极其费时的，因此SPP—net提出：能否在feature map上提取ROI（即region proposal）特征，这样就只需要在整幅图像上做一次卷积，大大提高了速度。需解决两个问题：

1. 原始图像的ROI如何映射到特征图（一系列卷积层的最后输出）
2. ROI在特征图上的对应的特征区域的维度不满足全连接层的输入要求怎么办

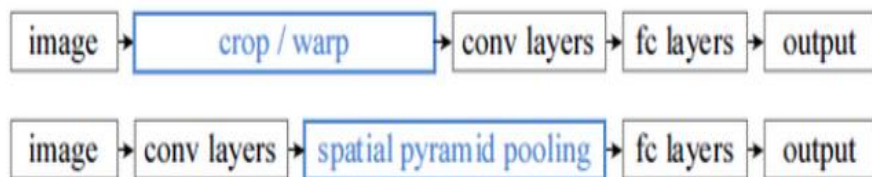
CNN的卷积层不需要固定尺寸的图像，全连接层是需要固定大小输入的，可把SPP层放到卷积层的后面：

SPP NET

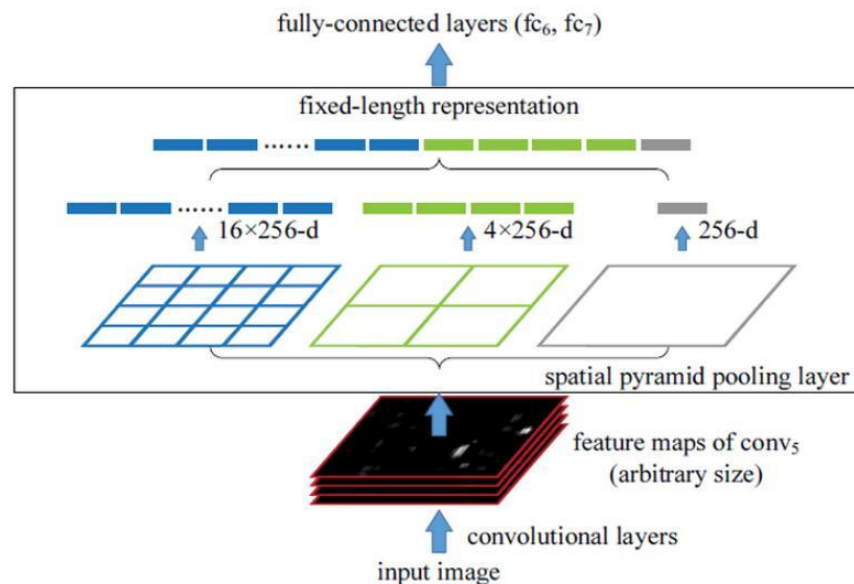
可允许不同尺寸图片输入，将R-CNN最后一个卷积层后的池化层替换为SPP层，生成长度固定的特征，并输入到最后的全连接层中。

特点：

- 1、可实现CNN多尺度图像的输入；
- 2、只对原图进行一次卷积特征提取。



SPP Net vs R-CNN

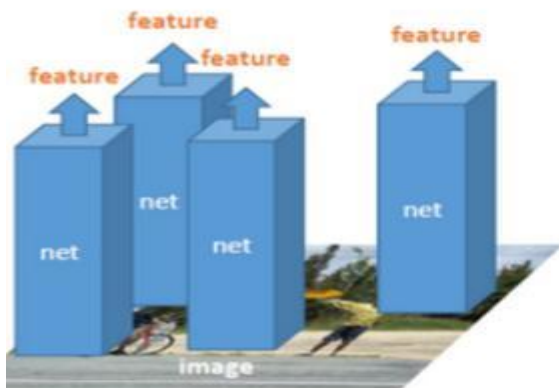


SPP Net结构

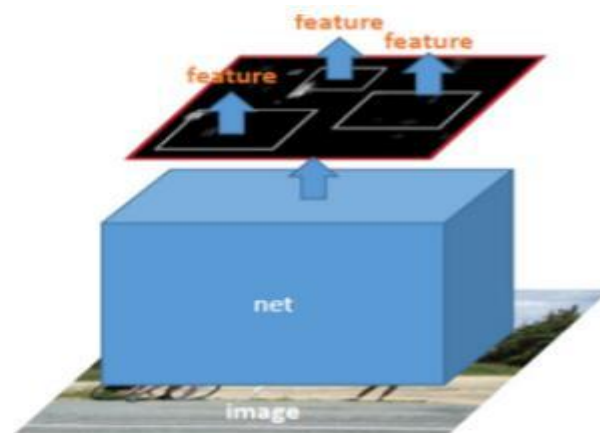
SPP-Net: Spatial Pyramid Pooling (空间金字塔池化)

1. 原始图像的ROI如何映射到特征图

output field size = (input field size - kernel size + 2*padding) / stride + 1
input field size = (output field size - 1) * stride - 2*padding + kernel size



R-CNN
2000 nets on image regions



SPP-net
1 net on full image

SPP—Net: Spatial Pyramid Pooling (空间金字塔池化)

感受野上面的坐标映射

① 计算公式

对于Convolution/Pooling Layer:

$$p_i = s_i \cdot p_{i+1} + [(k_i - 1)/2 - padding]$$

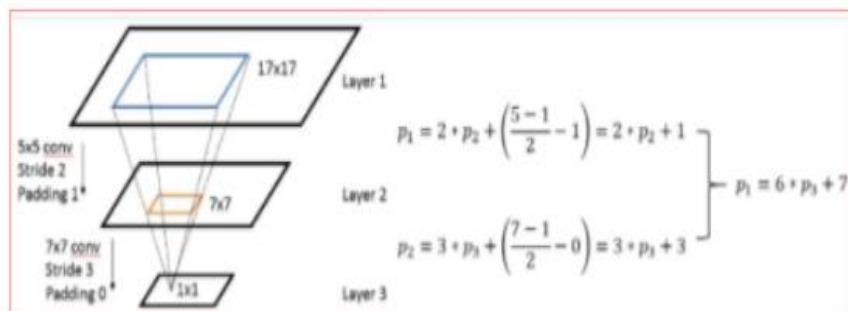
对于Neuronlayer (ReLU/Sigmoid/...) :

$$p_i = p_{i+1}$$

其中 p_i 为第 i 层感受野上的坐标, s_i 为Stride的大小, k_i 为感受野的大小。

② 例子

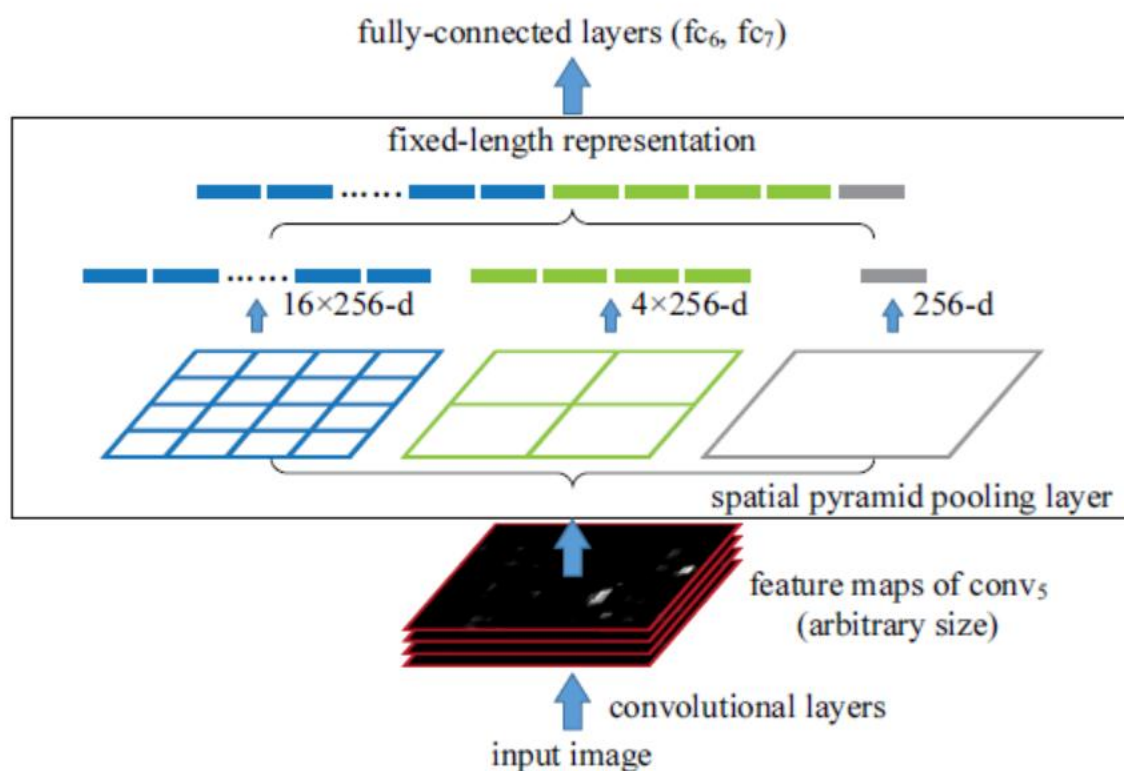
上面是计算任意一个Layer输入输出的坐标映射关系, 如果是计算任意Feature Map之间的关系, 只需要用简单的组合就可以得到,



SPP—NET是把原始ROI的左上角和右下角 映射到 Feature Map上的两个对应点。 有了Feature Map上的两个角点就确定了对应的Feature Map 区域

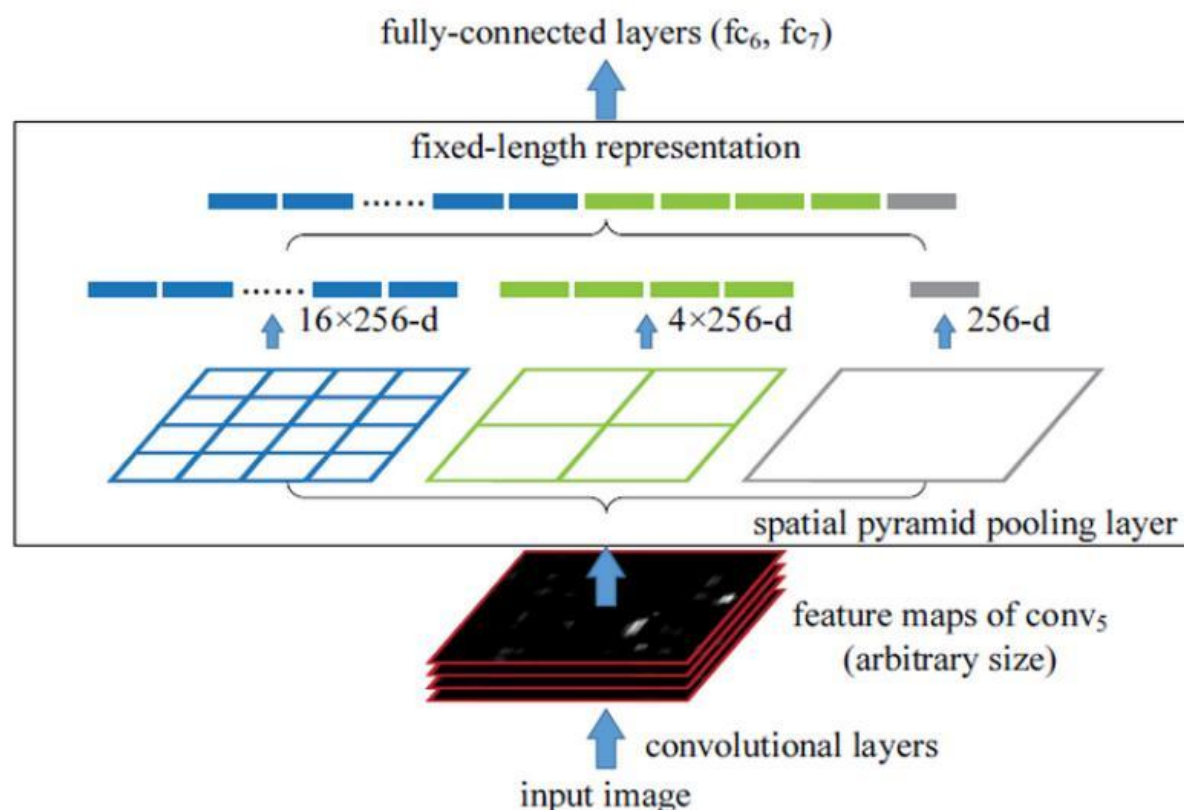
SPP-Net: Spatial Pyramid Pooling (空间金字塔池化)

2. 如何使feature map上的ROI对应的维度满足全连接层固定输入维度的要求



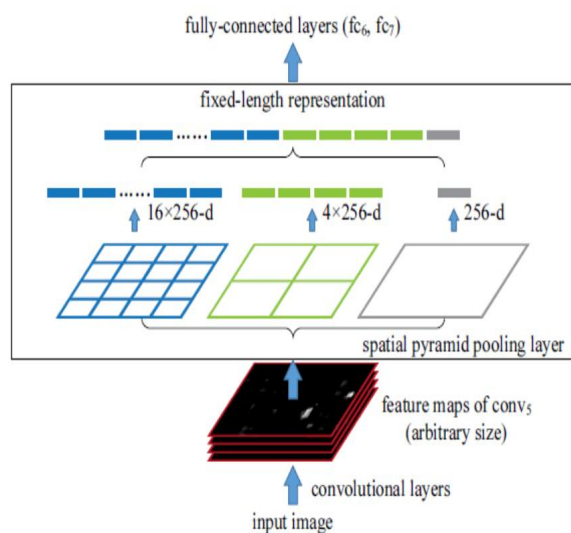
SPP-Net: Spatial Pyramid Pooling (空间金字塔池化)

2. 如何使feature map上的ROI对应的维度满足全连接层固定输入维度的要求



SPP—Net: Spatial Pyramid Pooling (空间金字塔池化)

2. 如何使feature map上的ROI对应的维度满足全连接层固定输入维度的要求



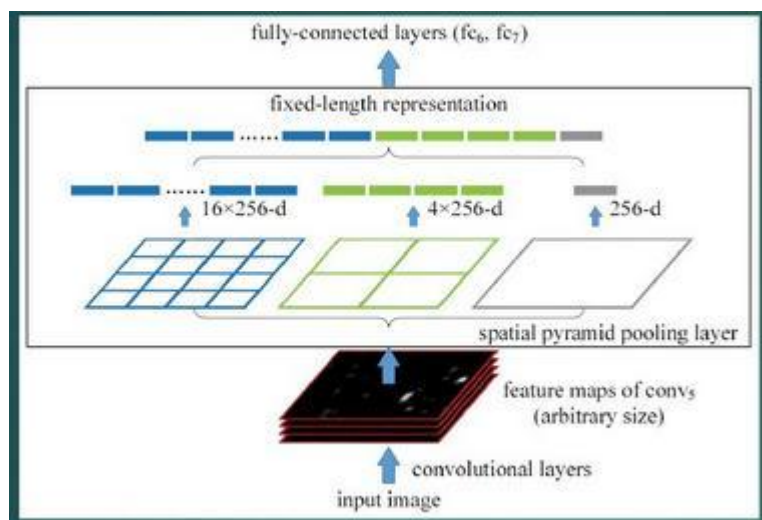
对于任意大小的feature map，首先将其分成16、4、1个块(window size)，然后在每个块上最大池化，池化后的特征拼接得到一个固定维度的输出，以满足全连接层的需要。假设原图输入是224x224，对于conv5出来后的输出是13x13x256的。如果像上图那样将response map分成1x1，2x2，4x4三张子图，分别做max pooling后，出来的特征就是(16+4+1)x256 维度。如果原图的输入不是224x224，出来的特征依然是(16+4+1)x256 维度。这样就实现了不管图像尺寸如何，SPP池化的输出永远是 (16+4+1) x256 维度。

SPP-Net: Spatial Pyramid Pooling (空间金字塔池化)

1. 结合空间金字塔方法实现CNNs的对尺度输入。

一般CNN后接全连接层或者分类器，他们都需要固定的输入尺寸，因此不得不
对输入数据进行crop或者warp，这些预处理会造成数据的丢失或几何的失真。SPP
Net的第一个贡献就是将金字塔思想加入到CNN，实现了数据的多尺度输入。

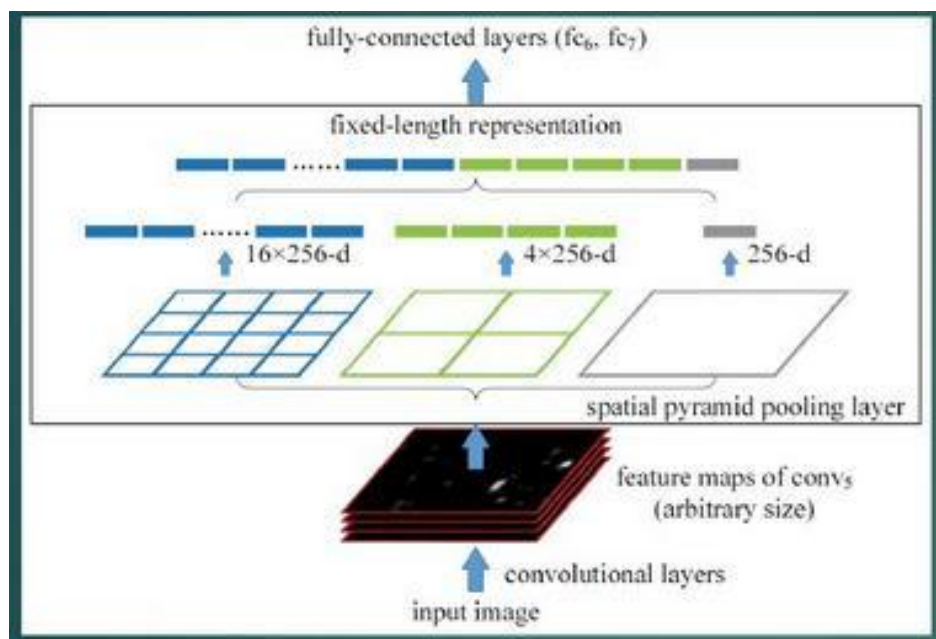
在卷积层和全连接层之间加入了**SPP layer**。此时网络的输入可以是任意尺度的
的，在**SPP layer**中每一个pooling的filter会根据输入调整大小，而SPP的输出尺度
始终是固定的。



SPP-Net: Spatial Pyramid Pooling (空间金字塔池化)

2. 只对原图提取一次卷积特征

在R-CNN中，每个候选框先resize到统一大小，然后分别作为CNN的输入，这样是很低效的。SPP Net根据这个缺点做了优化：只对原图进行一次卷积得到整张图的feature map，然后找到每个候选框在feature map上的映射patch，将此patch作为每个候选框的卷积特征输入到SPP layer和之后的层。节省了大量的计算时间，比R-CNN有一百倍左右的提速。





SPP NET

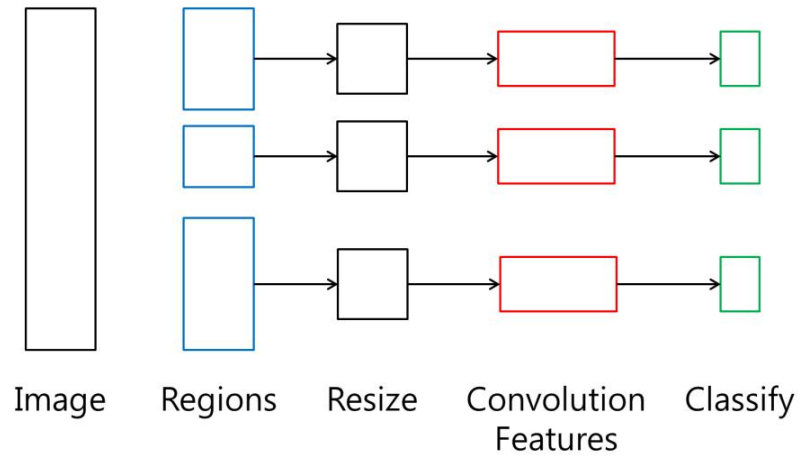
SPP NET存在的问题

经SPP NET改进后的R-CNN虽能有效提高检测速度，但仍存在局限性：

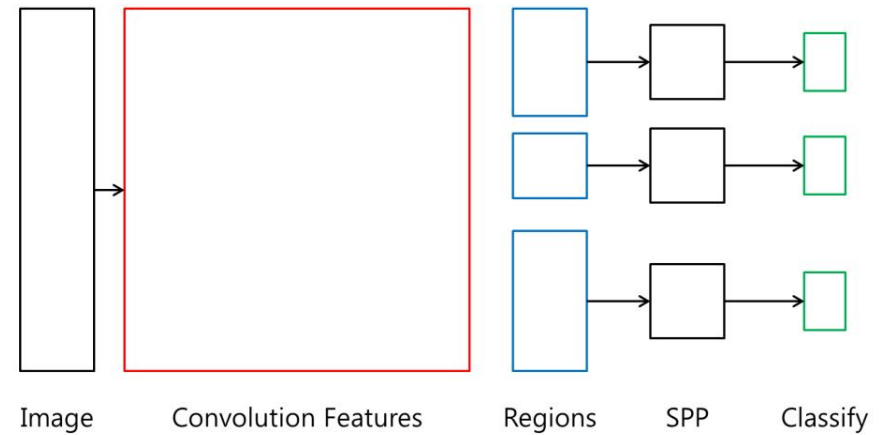
在SPP NET采用selective search对原始图片进行候选区域提取时，由于候选区域数量较多，存在候选区域特征重复提取计算问题，限制了SPP NET的检测速度。

另外，对SPP NET，虽然ROI特征在最后一个卷积层才提取，省去了多次前向CNN。但由于SVM，ROI特征仍需存储。此外，SPP NET中的tunning无法更新SPP层之前的所有权重，因此对于比较深的网络无能为力。

R-CNN vs. SPP net



R-CNN



SPP net

RCNN

Too slow

Run CNN once



SPP-net

Multi stage training/ No CNN fine-tuning



Fast
RCNN

Single stage training



- Better performance
- Single stage training
- All nets are training

Fast R-CNN

RCNN缺点：由于每一个候选框都要独自经过CNN，这使得花费的时间非常多。

解决：共享卷积层，现在不是每一个候选框都当做输入进入CNN了，而是输入一张完整的图片，在第五个卷积层再得到每个候选框的特征

原来的方法：许多候选框（比如两千个） $\rightarrow$ CNN $\rightarrow$ 得到每个候选框的特征 $\rightarrow$ 分类+回归

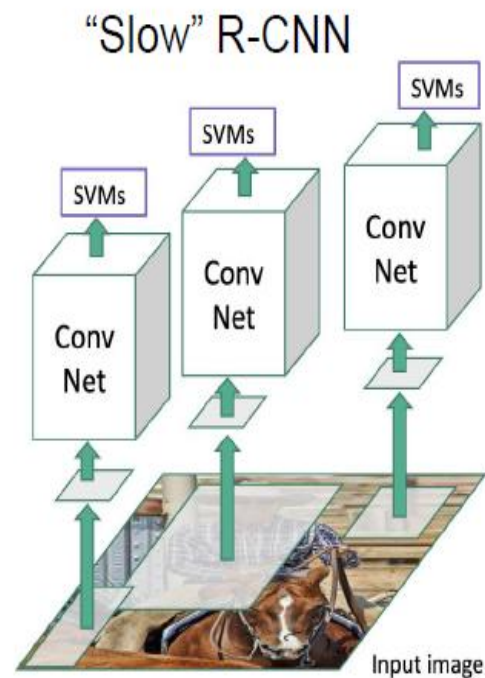
现在的方法：一张完整图片 $\rightarrow$ CNN $\rightarrow$ 得到每张候选框的特征 $\rightarrow$ 分类+回归

Fast R-CNN

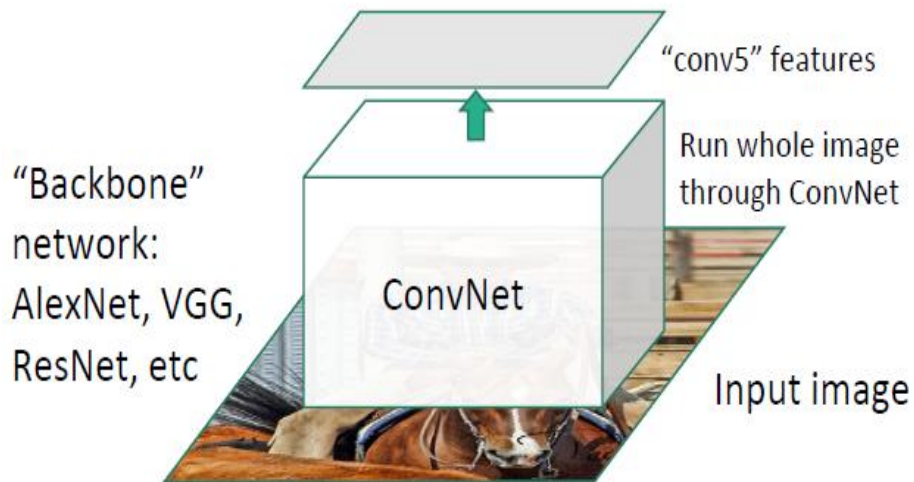
直接使用Softmax替代了R-CNN中SVM进行分类，同时在网络中加入了多任务函数边框回归，实现了端到端的训练（除SS Region Proposal阶段）



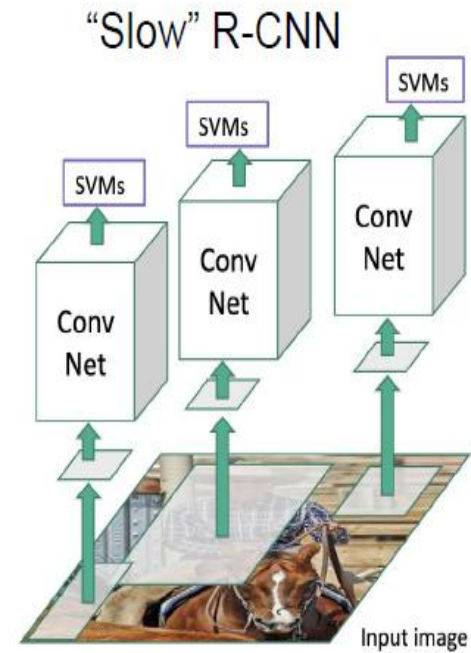
Input image



Fast R-CNN



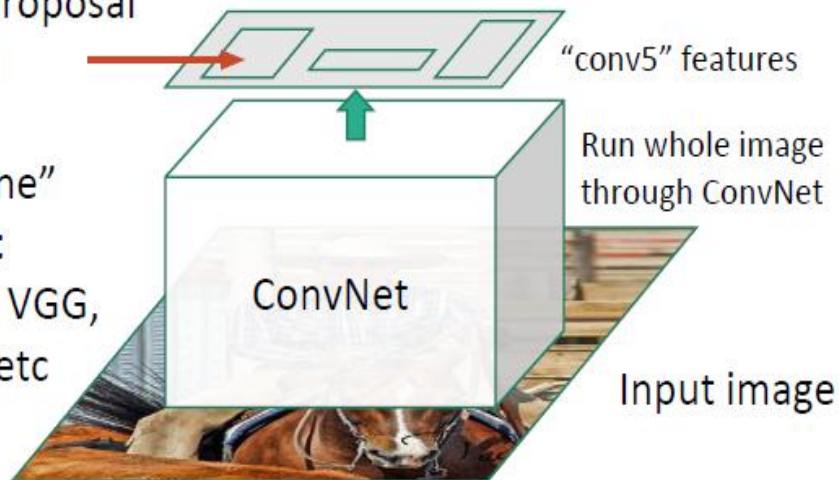
Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.



Fast R-CNN

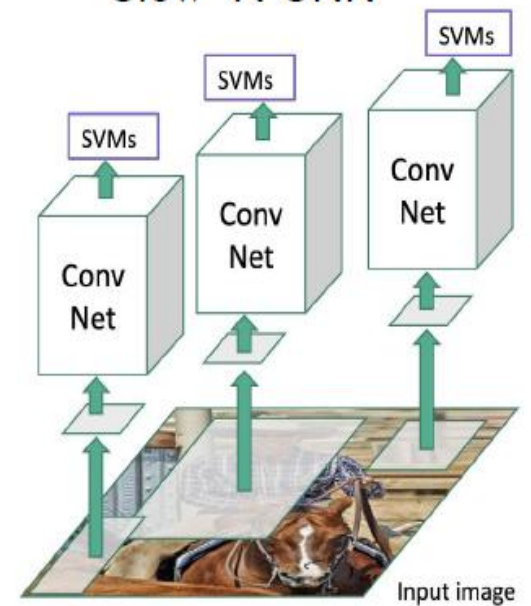
Regions of Interest (RoIs) from a proposal method

“Backbone” network:
AlexNet, VGG,
ResNet, etc

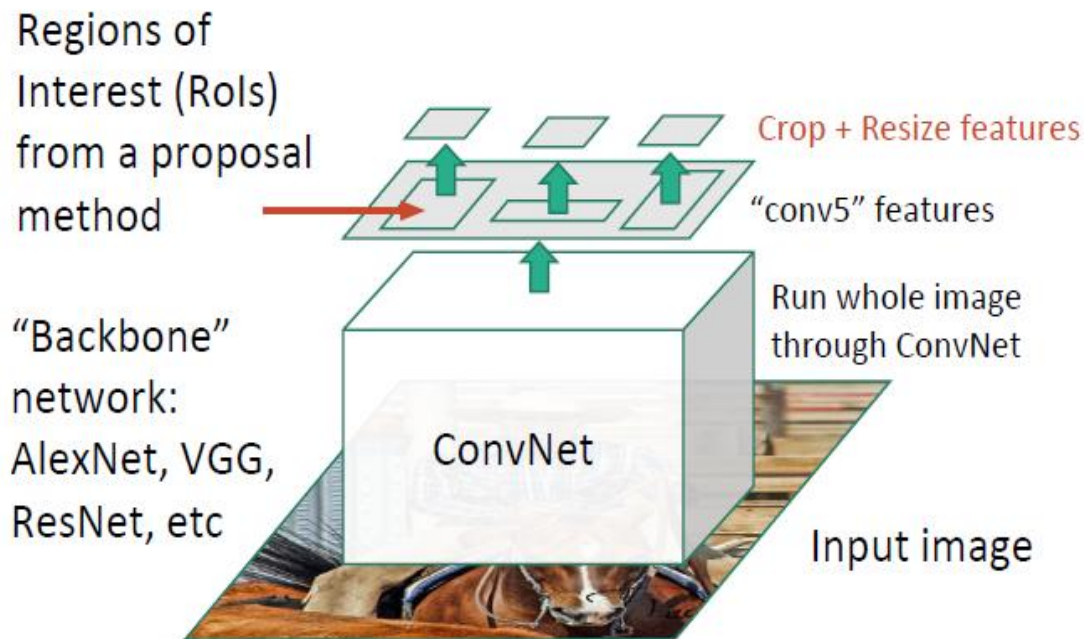


Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

“Slow” R-CNN

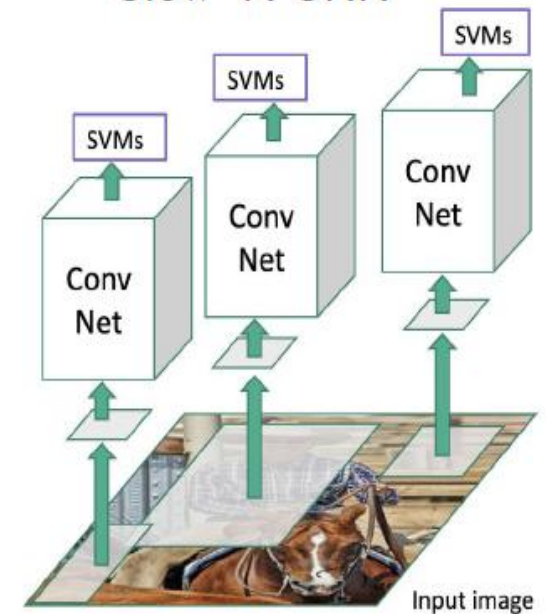


Fast R-CNN



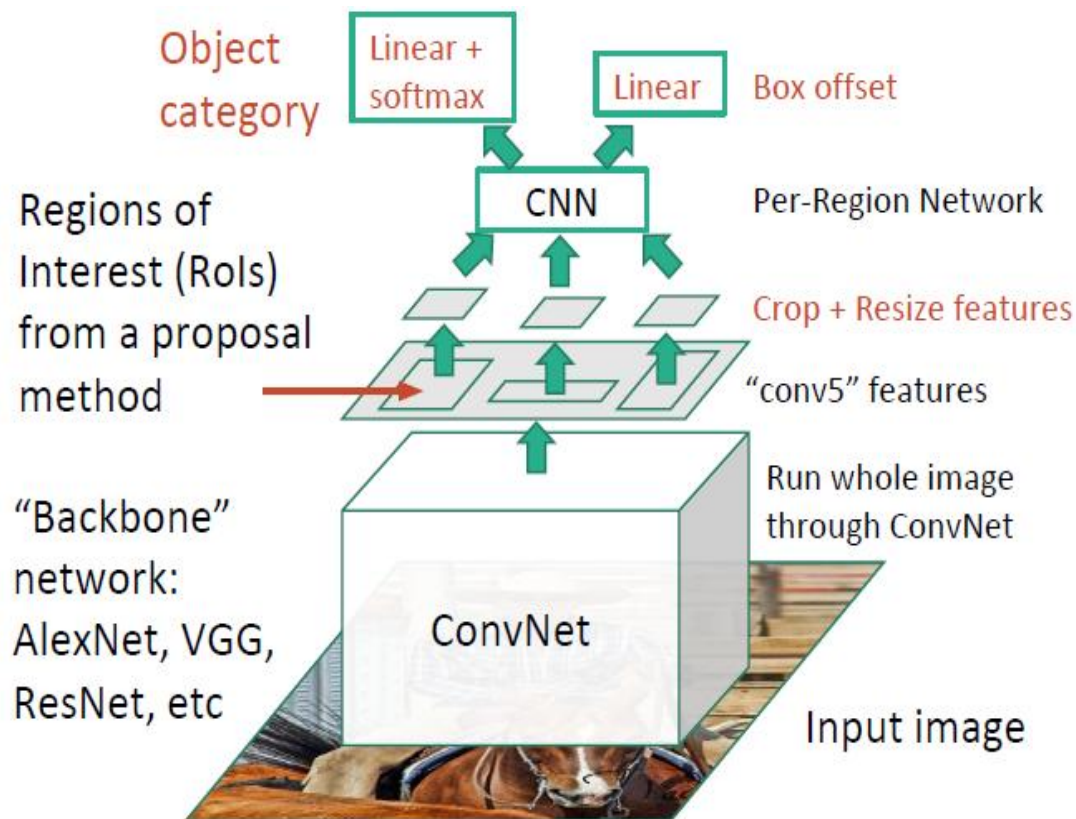
Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

"Slow" R-CNN

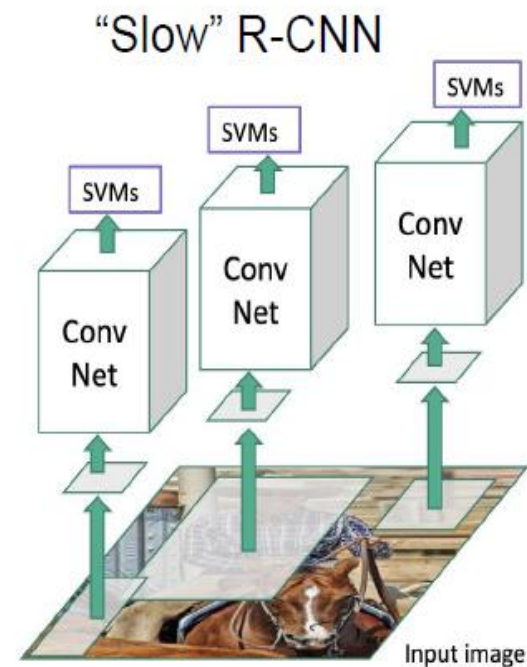


Fast R-CNN

Fast R-CNN，它不是从原图中提取候选区域，而是先让图片经过卷积神经网络，然后在特征图上选取感兴趣区域（ROI），然后通过全连接层，得到两个输出

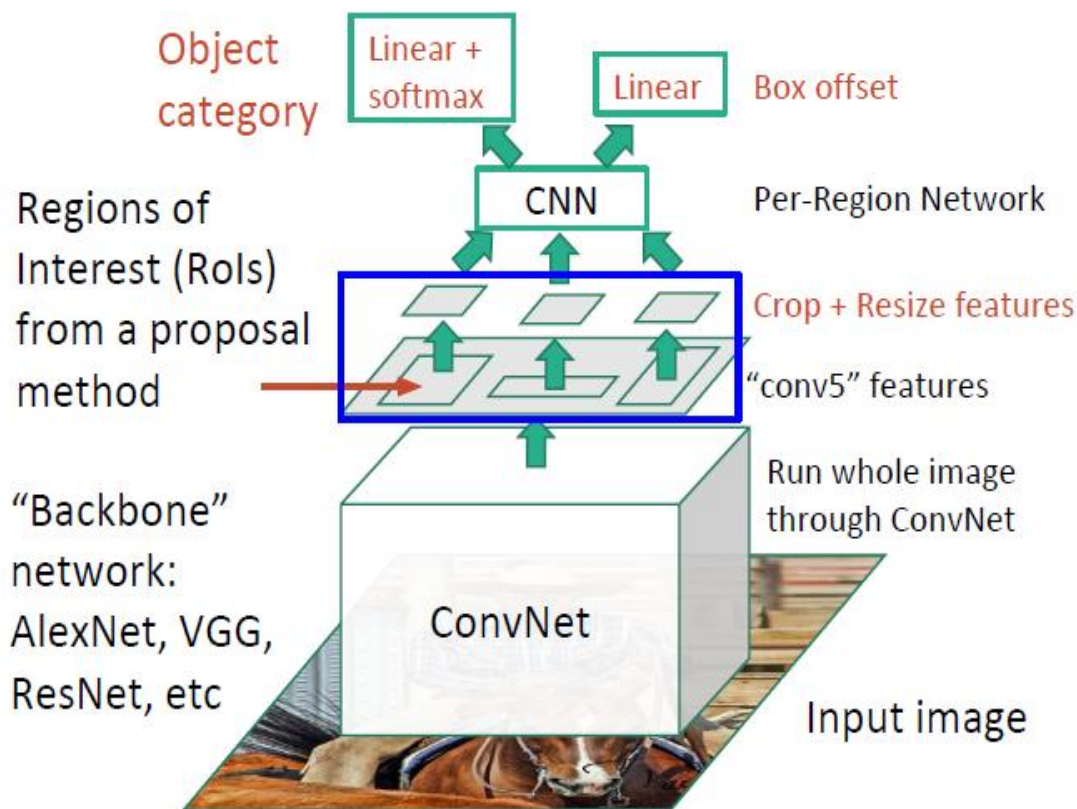


Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

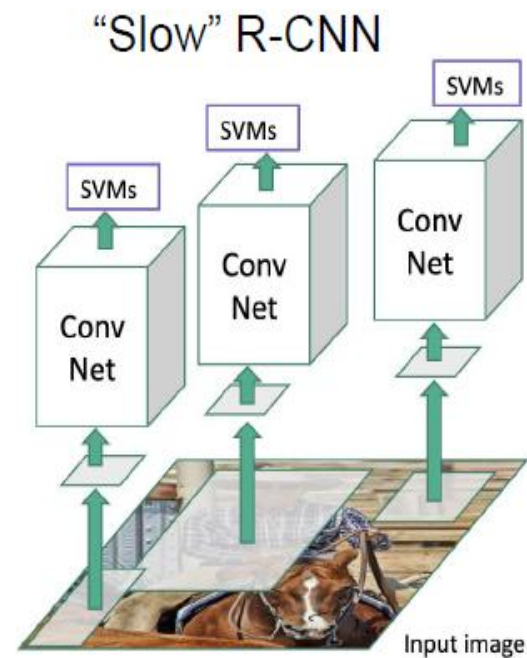


重用图像上的卷积结果

Fast R-CNN

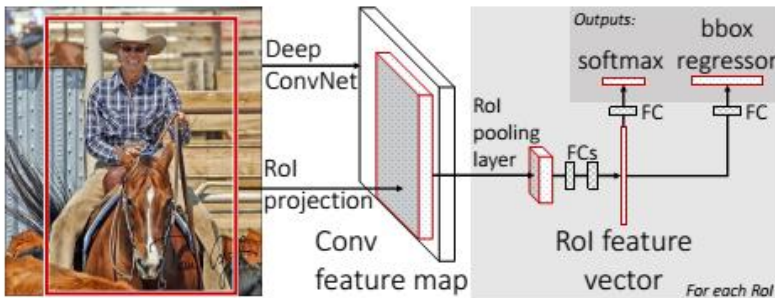
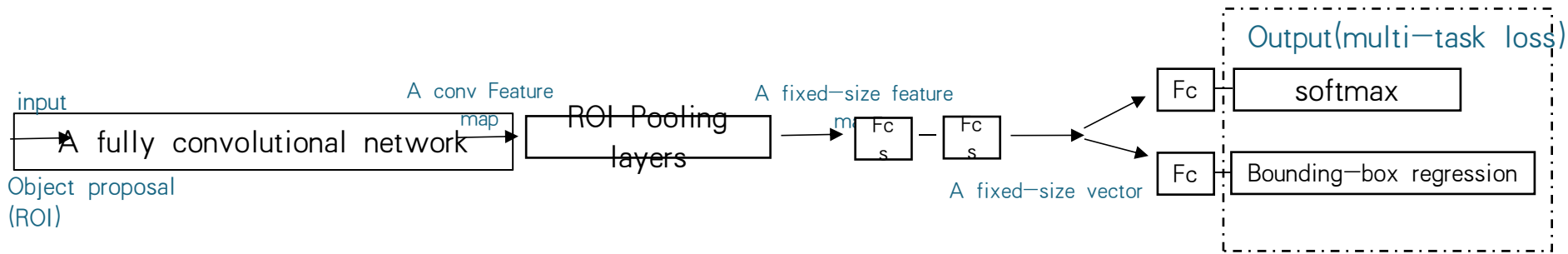


Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.



重用图像上的卷积结果

Fast RCNN

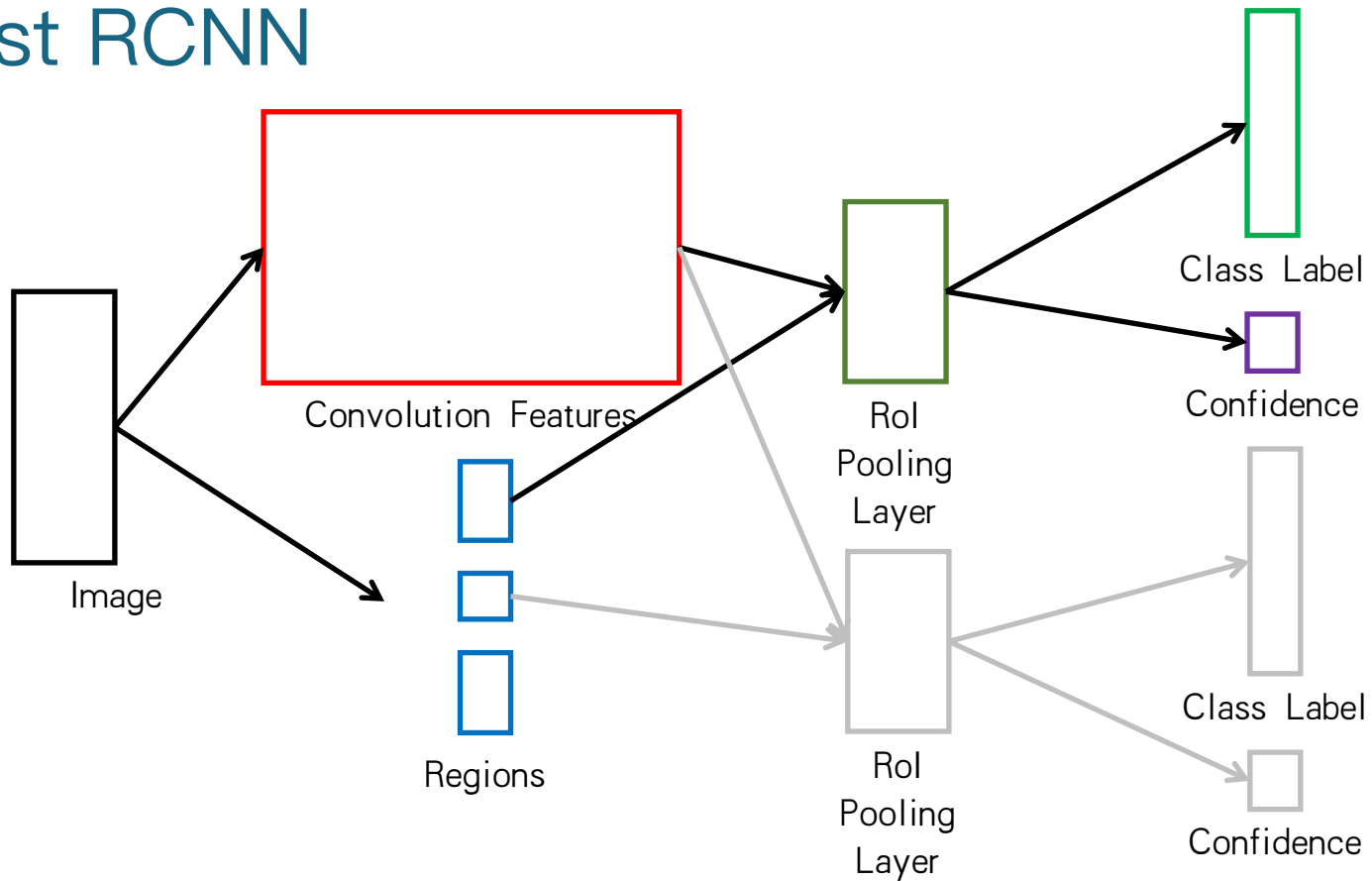


$$LOSS = L_{cls}(p, u) + \lambda[u \geq 1]L_{loc}(t^u, v)$$

Pred location true location

pred target Not background

Fast RCNN





Fast R-CNN

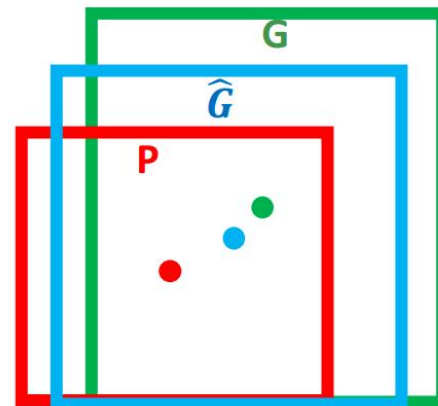
(1) ROI pooling layer

实际上是SPP-NET的一个精简版，SPP-NET对每个proposal使用了不同大小的金字塔映射，而ROI pooling layer只需要下采样到一个7x7的特征图。对于VGG16网络conv5_3有512个特征图，这样所有region proposal对应了一个7*7*512维度的特征向量作为全连接层的输入。换言之，这个网络层可以把不同大小的输入映射到一个固定尺度的特征向量。

(2) 边框回归

对于窗口一般使用四维向量 (x,y,w,h) ，分别表示窗口的中心点坐标和宽高。红色的框 P 代表原始的Proposal, 绿色的框 G 代表目标的Ground Truth,

我们的目标是寻找一种关系使得输入原始的窗口 P 经过映射得到一个跟真实窗口 G 更接近的回归窗口 $\hat{G}$ 。所以，边框回归的目的即是：给定 (P_x, P_y, P_w, P_h) 寻找一种映射 f ，使得 $f(P_x, P_y, P_w, P_h) = (G_x^{\wedge}, G_y^{\wedge}, G_w^{\wedge}, G_h^{\wedge})$ 并且 $(G_x^{\wedge}, G_y^{\wedge}, G_w^{\wedge}, G_h^{\wedge}) \approx (G_x, G_y, G_w, G_h)$



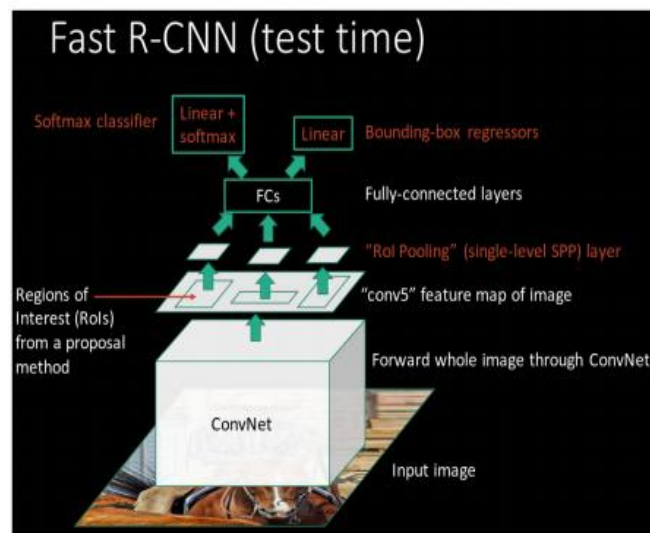
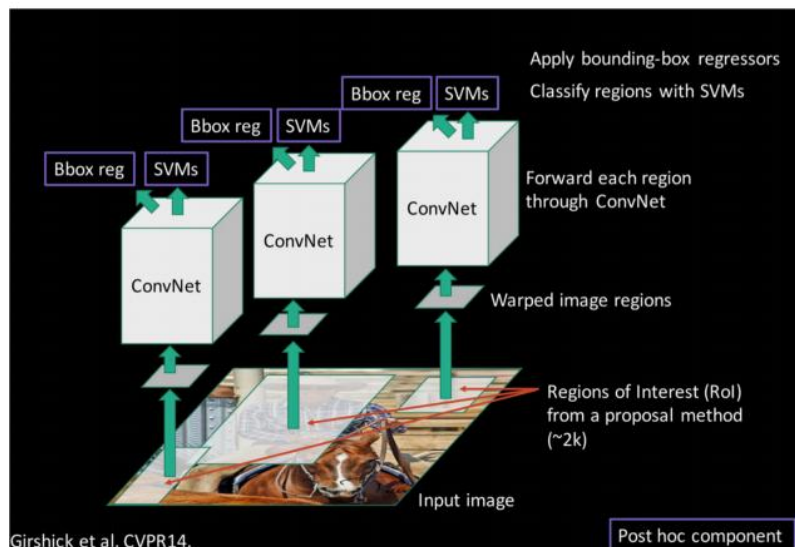
Fast R-CNN

Fast R-CNN vs R-CNN

与R-CNN框架图对比, 可以发现主要有两处不同:

一是最后一个卷积层后加了一个ROI pooling layer;

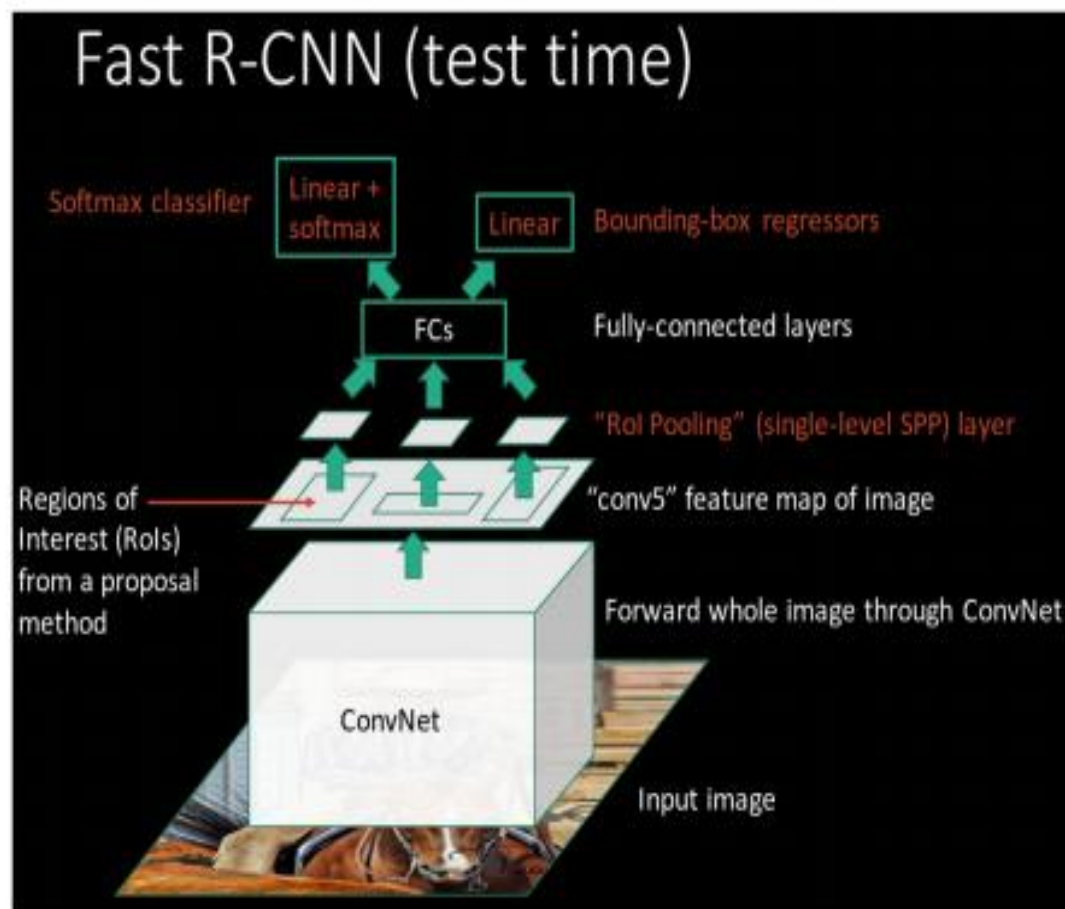
二是损失函数使用了多任务损失函数, 将边框回归Bounding Box Regression 直接加入到CNN网络中训练。



R-CNN Problem #1:
Slow at test-time due to independent forward passes of the CNN

Solution:
Share computation of convolutional layers between proposals for an image

Fast R-CNN

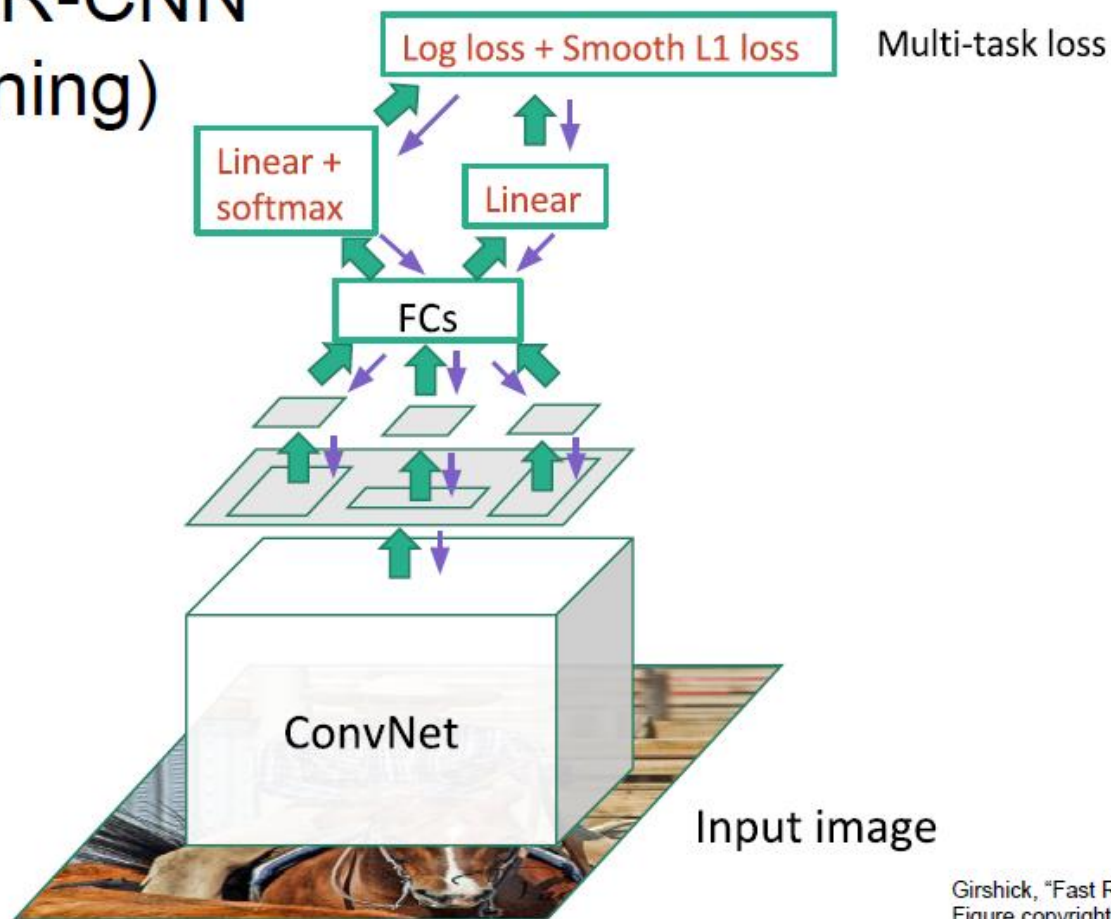


R-CNN Problem #1:
Slow at test-time due to independent forward passes of the CNN

Solution:
Share computation of convolutional layers between proposals for an image

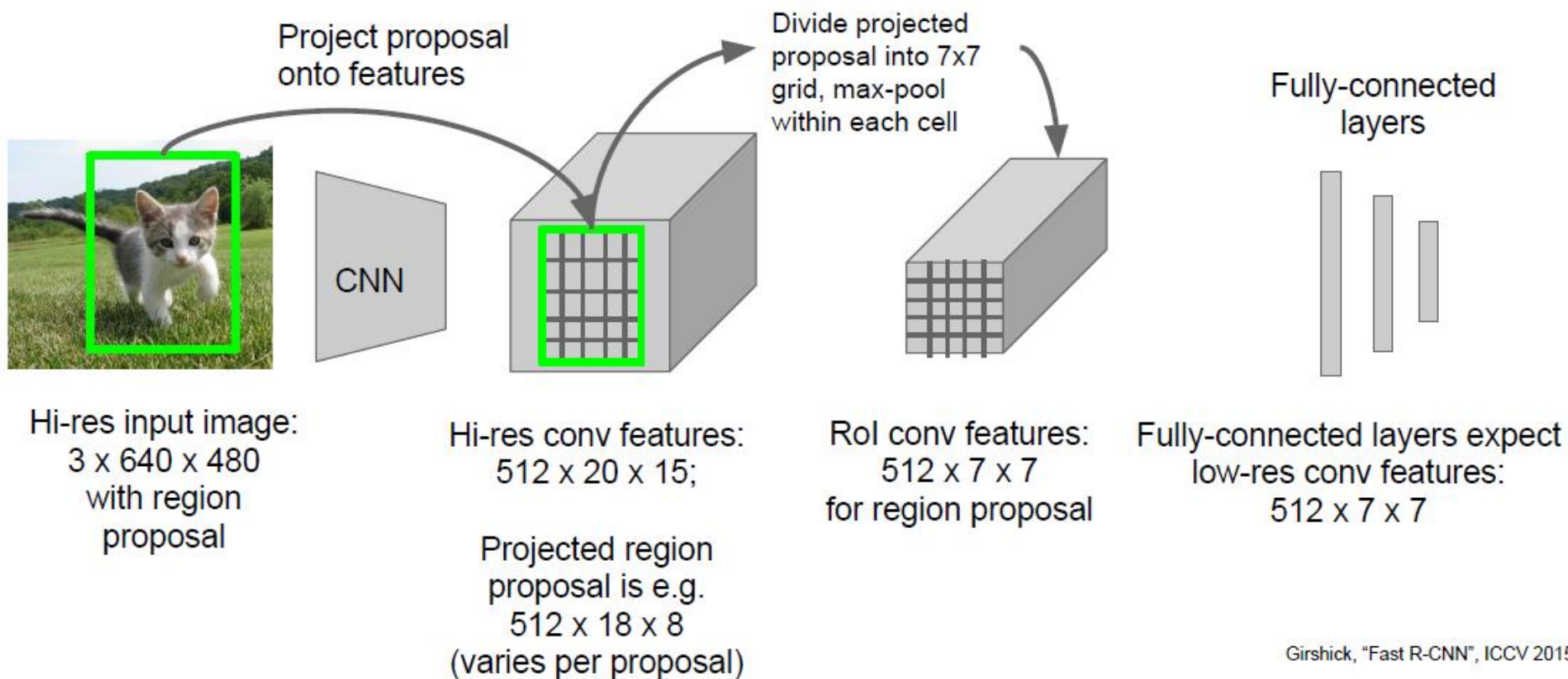
重用图像上的卷积结果

Fast R-CNN (Training)

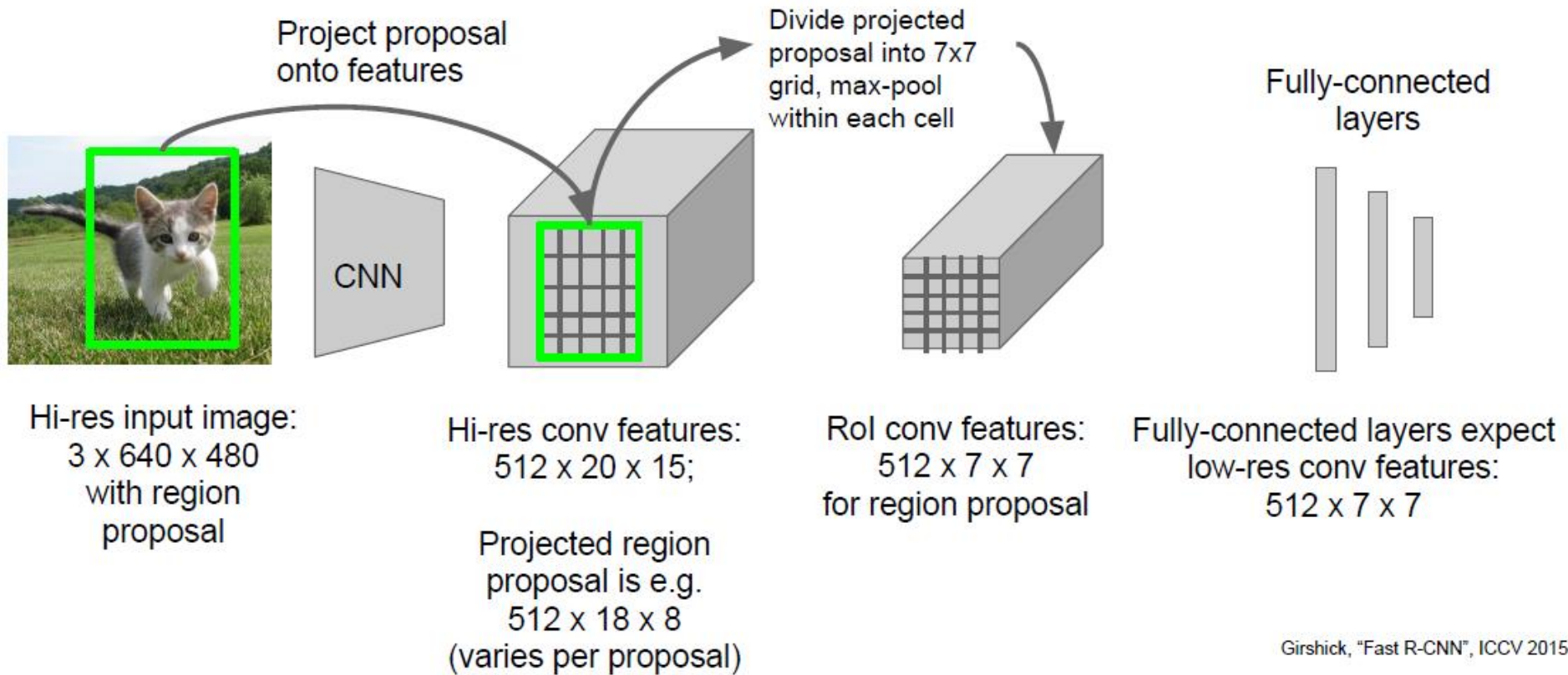


Girshick, "Fast R-CNN", ICCV 2015.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Cropping Features: RoI Pool

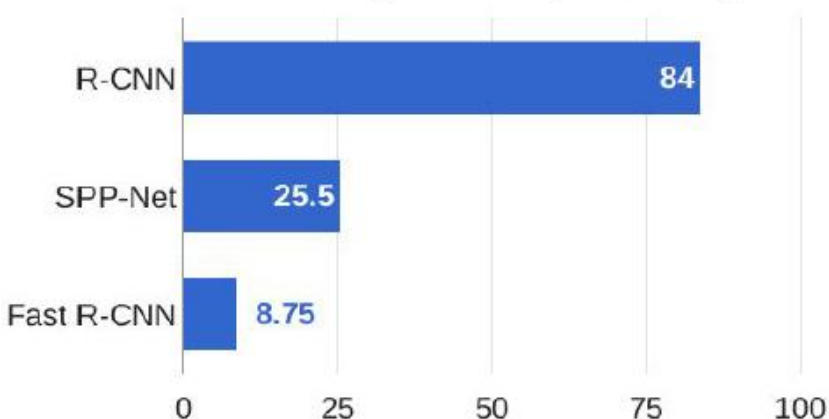


Fast R-CNN: RoI Pooling

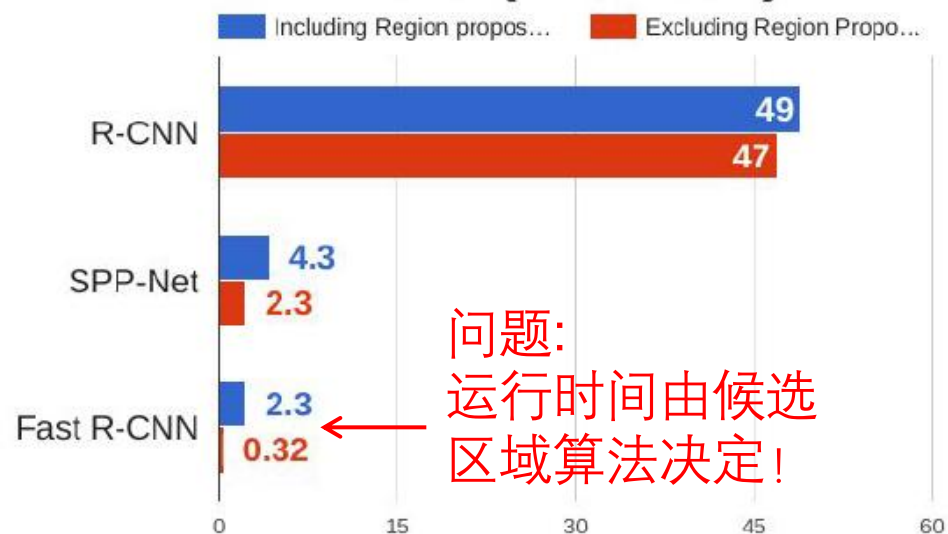


R-CNN vs SPP vs Fast R-CNN

Training time (Hours)



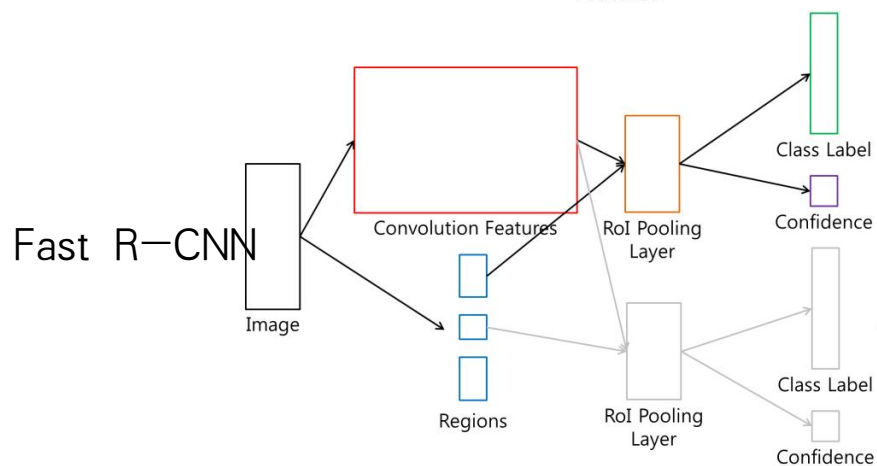
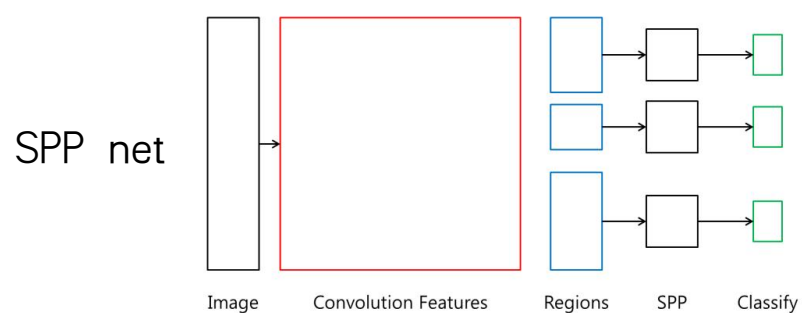
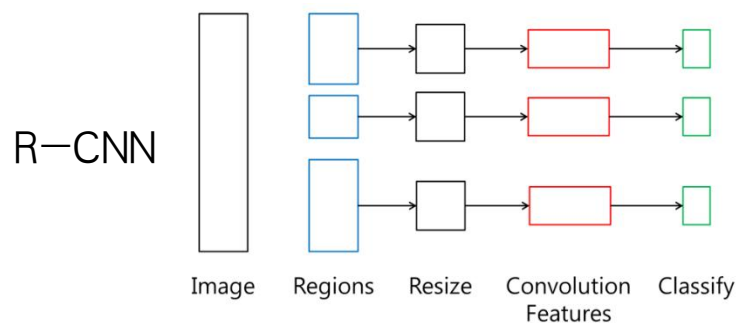
Test time (seconds)



问题:
运行时间由候选
区域算法决定!

几种方法消耗时间的对比，可以看到对于Fast R-CNN，时间大部分都花在了传统的候选区域算法上面。

R-CNN vs. SPP net vs. Fast R-CNN



RCNN



SPP-net



Fast RCNN



Faster RCNN

Too slow for multiple convolution

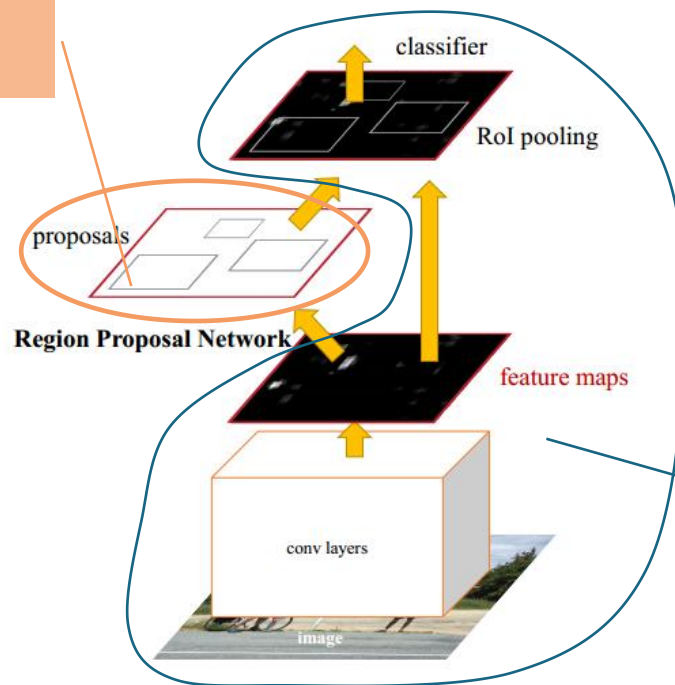
Multi stage training + No CNN fine-tuning

Object proposal sampling is still slow

Sample BBs with CNN!

Faster RCNN

This part is
New!



Region Proposal Net

+

Fast RCNN

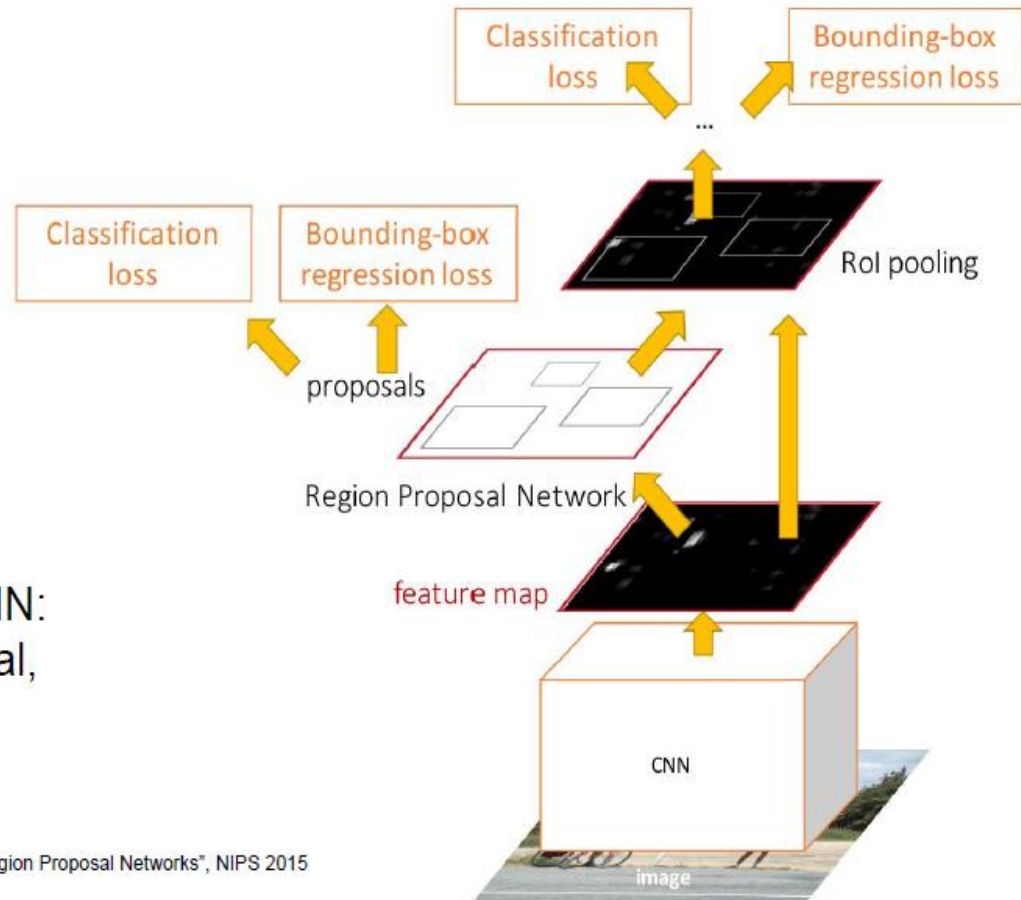
There are simple
Fast RCNN

Faster R-CNN

Make CNN do proposals!

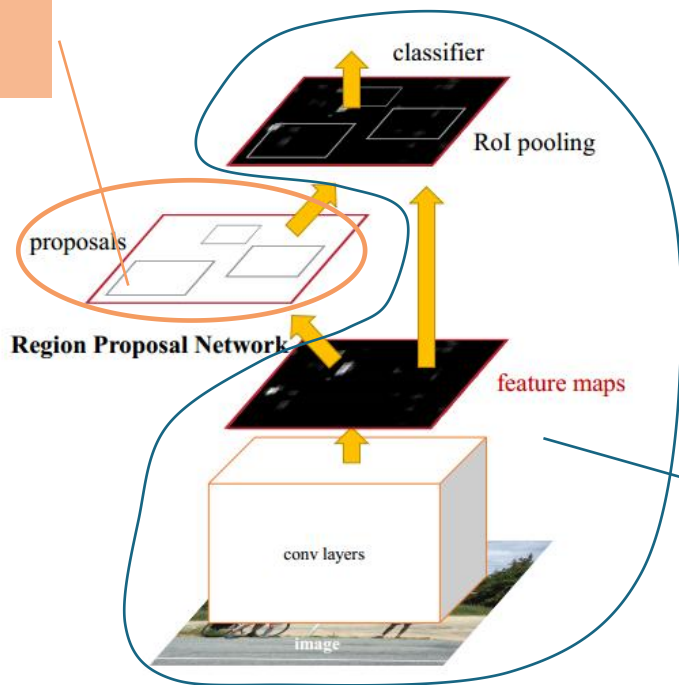
Insert **Region Proposal Network (RPN)** to predict proposals from features

Otherwise same as Fast R-CNN:
Crop features for each proposal,
classify each one



Faster RCNN

This part is New!



Region Proposal Net

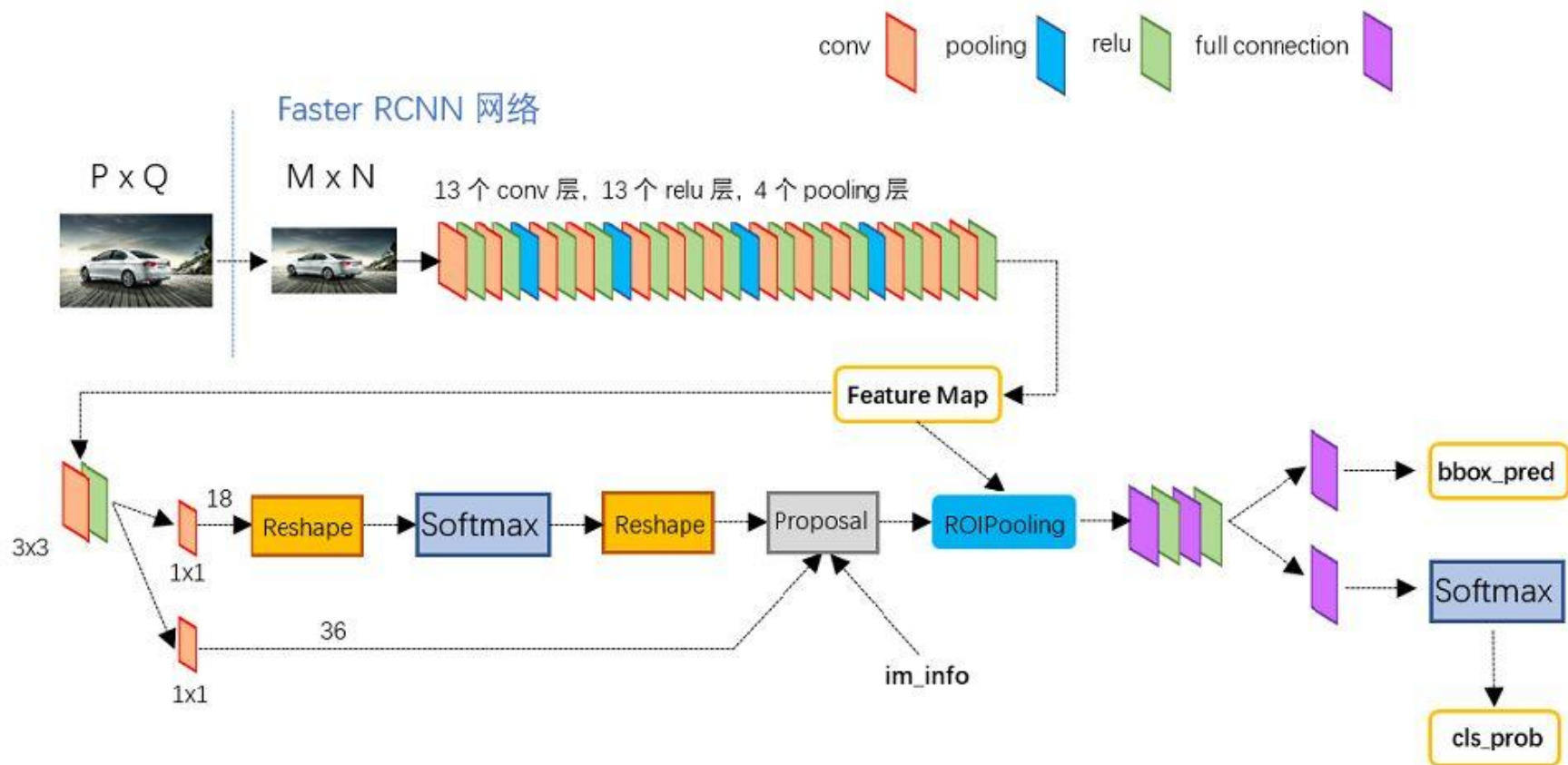
+

Fast RCNN

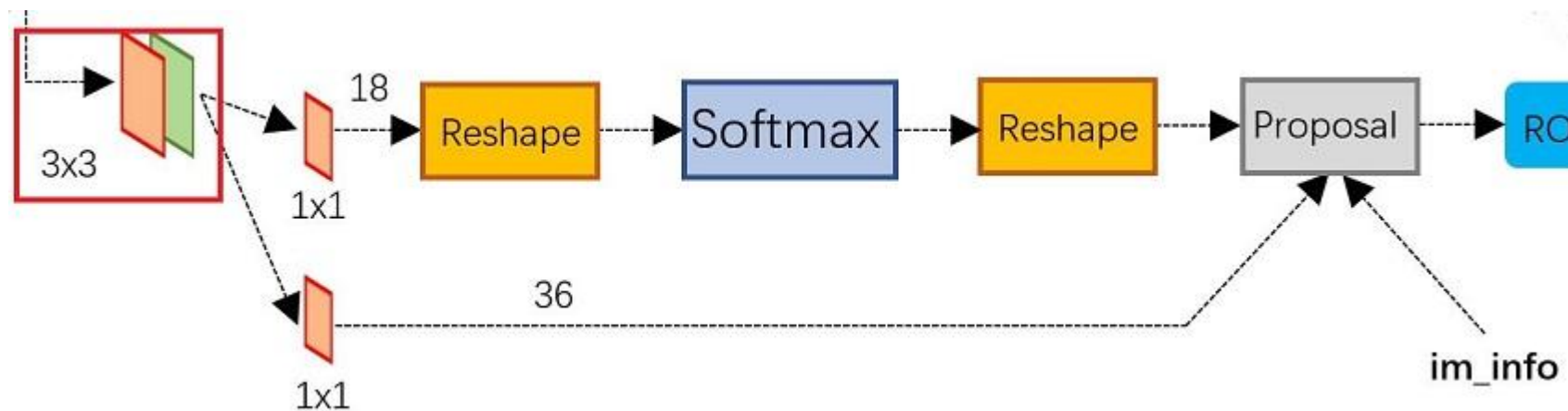
There are simple
Fast RCNN

Faster R-CNN算法流程可分为3个步骤

- 将图像输入网络得到相应的特征图
- 使用RPN结构生成候选框，将RPN生成的候选框投影到特征图上获得相应的特征矩阵
- 将每个特征矩阵通过ROI pooling层缩放到7x7大小的特征图，接着将特征图展平通过一系列全连接层得到预测结果

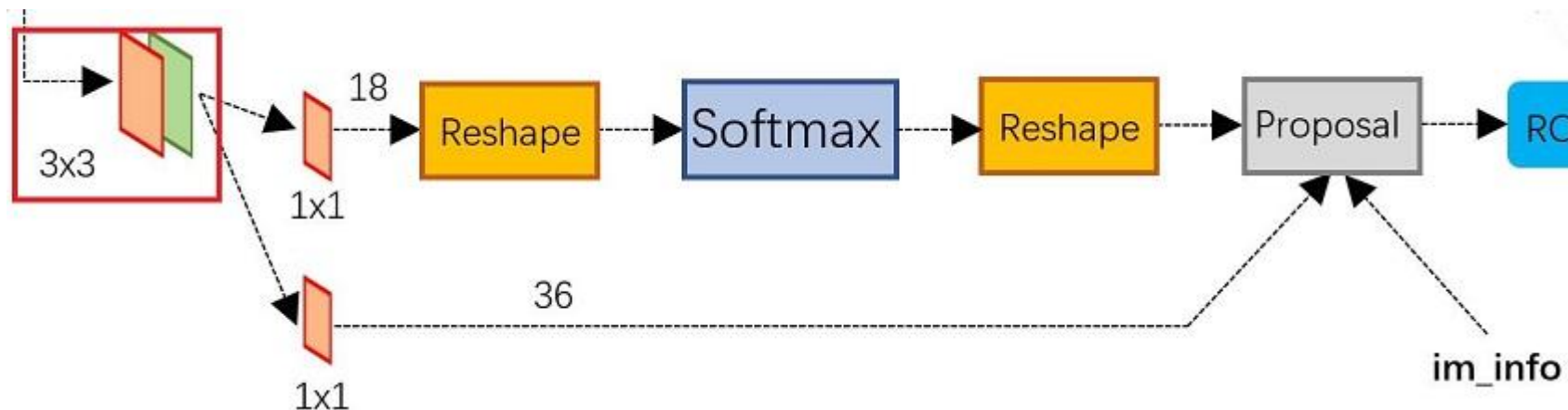


RPN



经典的检测方法生成检测框都非常耗时，如R-CNN使用SS(Selective Search)方法生成检测框。而Faster RCNN则抛弃了传统的滑动窗口和SS方法，直接使用RPN生成检测框。

RPN



RPN网络实际分为2条线，上面一条通过softmax分类anchors获得positive和negative分类，下面一条用于计算对于anchors的bounding box regression偏移量，以获得精确的proposal。而最后的Proposal层则负责综合positive anchors和对应bounding box regression偏移量获取proposals，同时剔除太小和超出边界的proposals。其实整个网络到了Proposal Layer这里，就完成了相当于目标定位的功能

Region Proposal Network



Input Image
(e.g. 3 x 640 x 480)

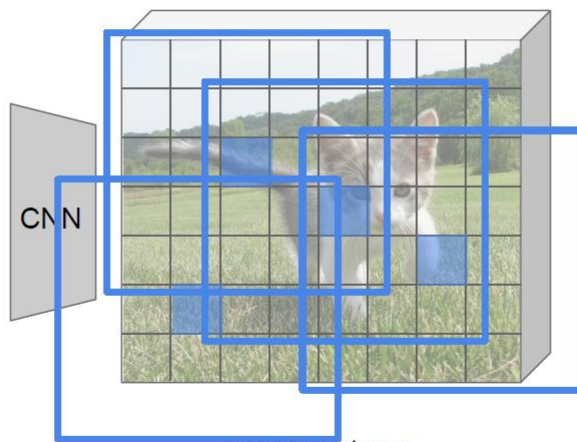


Image features
(e.g. 512 x 20 x 15)

Imagine an **anchor box**
of fixed size at each
point in the feature map

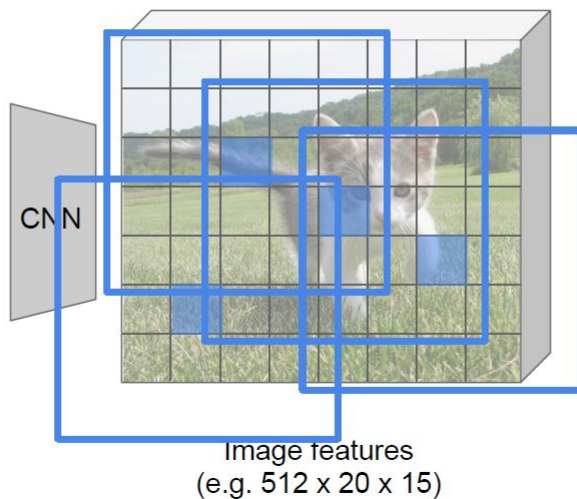
anchors, 实际上就是
一组矩形框

每行的4个值 表矩形左上和右下角点坐标。
9个矩形共有3种形状，长宽比为大约为三种

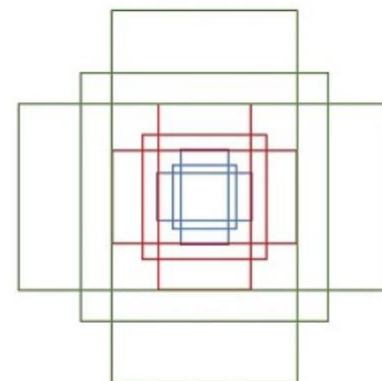
Region Proposal Network



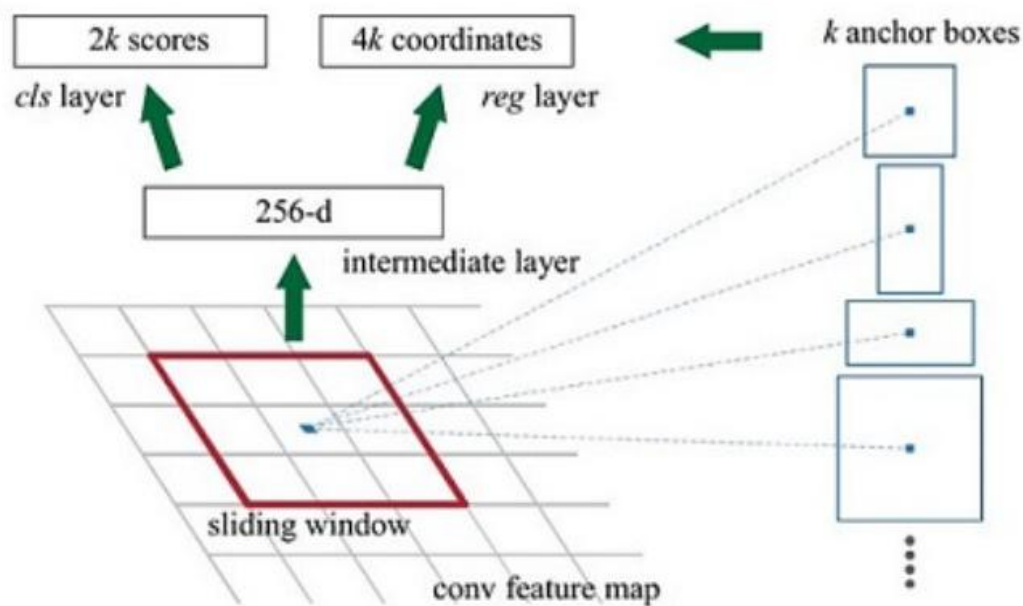
Input Image
(e.g. $3 \times 640 \times 480$)



Imagine an **anchor box**
of fixed size at each
point in the feature map



9个anchors是做什么的呢？遍历Conv layers计算获得的feature maps，为每一个点都配备这9种anchors作为初始的检测框。

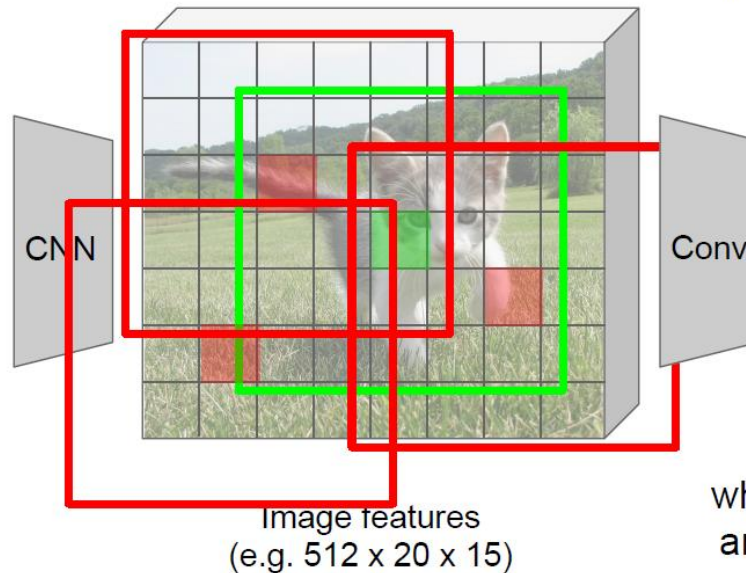


- ✓ 原文中，其Conv Layers中最后的conv5层num_output=256，对应生成256张特征图，所以相当于feature map每个点都是256-dimensions
- ✓ 在conv5之后，做了rpn_conv/3x3卷积且num_output=256，相当于每个点又融合了周围3x3的空间信息同时256-d不变（如图中的红框）
- ✓ 假设在conv5 feature map中每个点上有k个anchor（默认k=9），而每个anchor要分positive和negative，所以每个点由256d feature转化为cls=2k scores；而每个anchor都有(x, y, w, h)对应4个偏移量，所以reg=4k coordinates

Region Proposal Network



Input Image
(e.g. 3 x 640 x 480)



Imagine an **anchor box**
of fixed size at each
point in the feature map

Anchor is an object?
1 x 20 x 15

At each point, predict
whether the corresponding
anchor contains an object
(binary classification)

Region Proposal Network



Input Image
(e.g. 3 x 640 x 480)

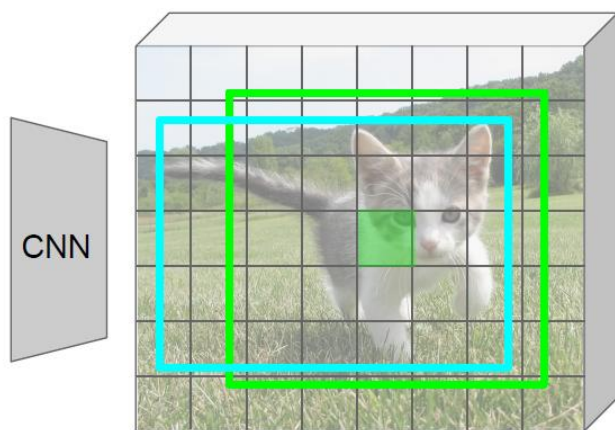
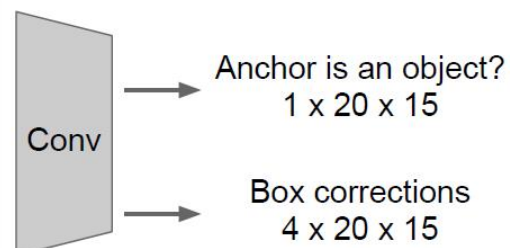


Image features
(e.g. 512 x 20 x 15)

Imagine an **anchor box**
of fixed size at each
point in the feature map



For positive boxes, also predict
a corrections from the anchor to
the ground-truth box (regress 4
numbers per pixel)

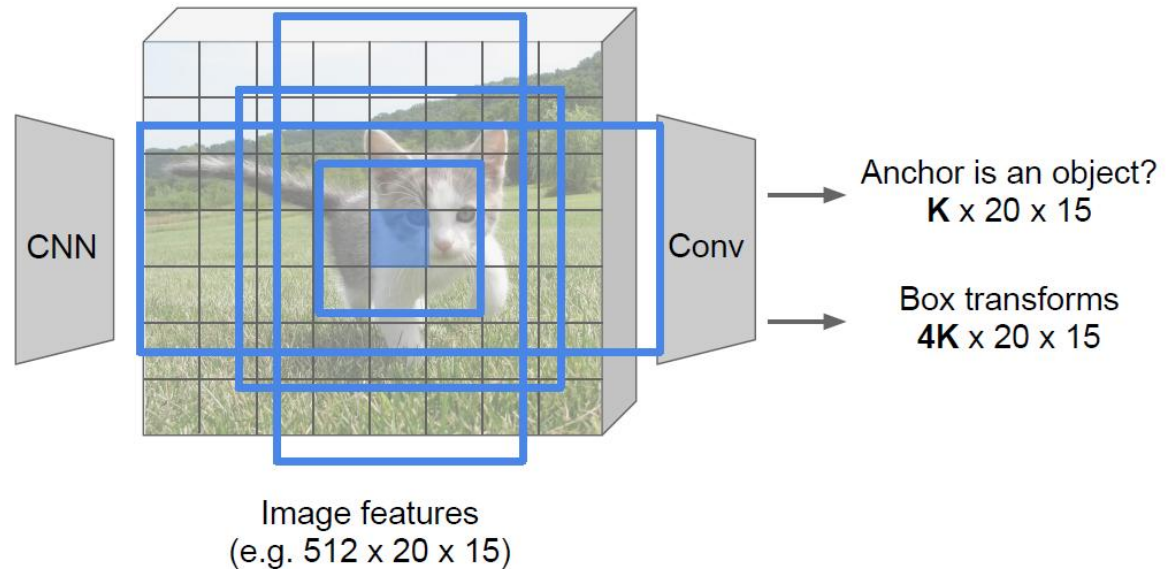
RPN最终就是在原图尺度上，设置了密密麻麻的候选Anchor。然后用cnn去判断哪些Anchor是里面有目标的positive anchor，哪些是没目标的negative anchor。

Region Proposal Network

In practice use K different anchor boxes of different size / scale at each point



Input Image
(e.g. $3 \times 640 \times 480$)



Region Proposal Network

In practice use K different anchor boxes of different size / scale at each point



Input Image
(e.g. $3 \times 640 \times 480$)

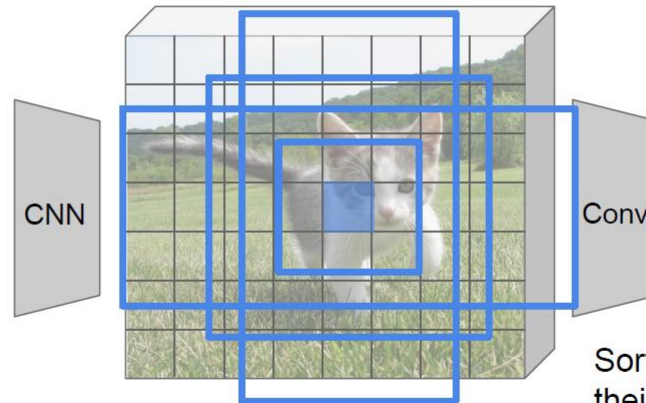
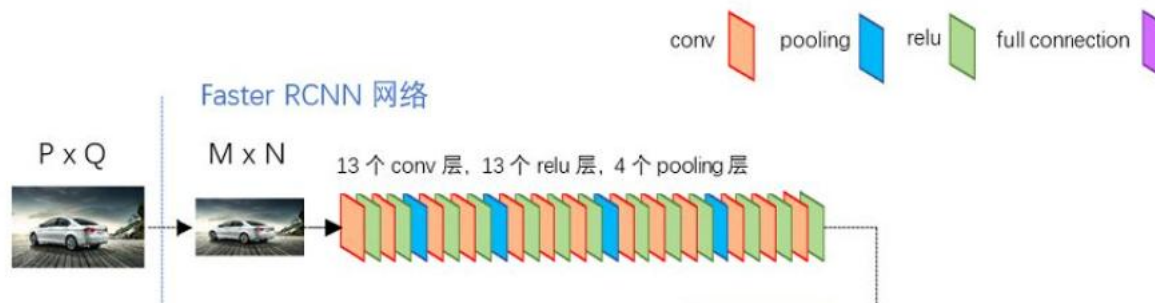


Image features
(e.g. $512 \times 20 \times 15$)

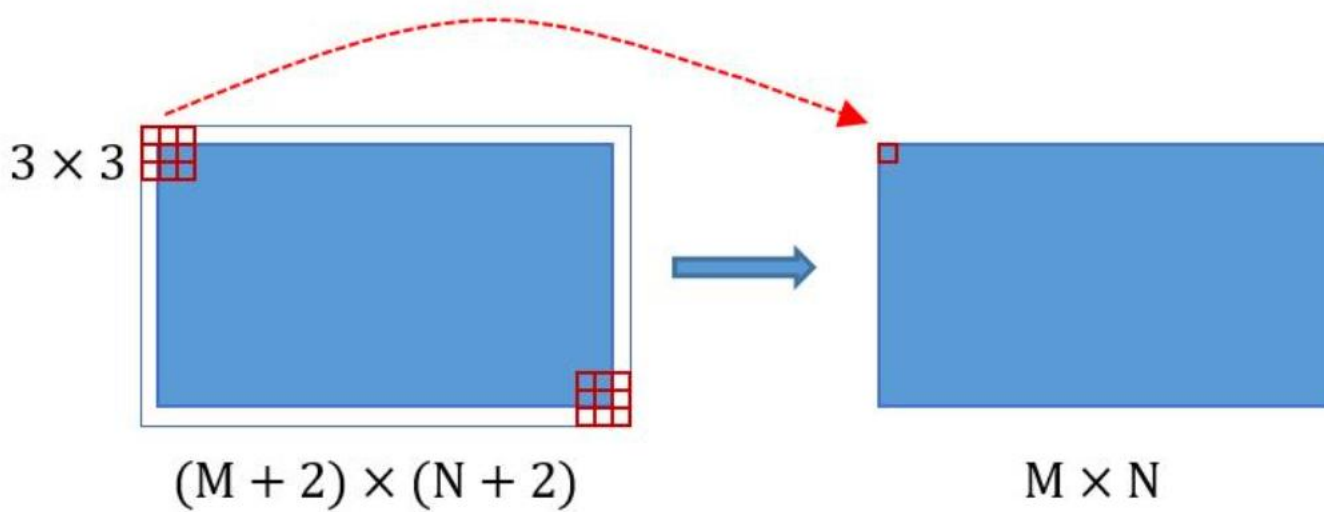
Anchor is an object?
 $K \times 20 \times 15$

Box transforms
 $4K \times 20 \times 15$

Sort the $K \times 20 \times 15$ boxes by their "objectness" score, take top ~ 300 as our proposals



一副 $M \times N$ 大小的矩阵送入 Faster RCNN 网络后，
到 RPN 网络变为 $(M/16) \times (N/16)$ ，不妨设
 $W = M/16$ ， $H = N/16$ 。

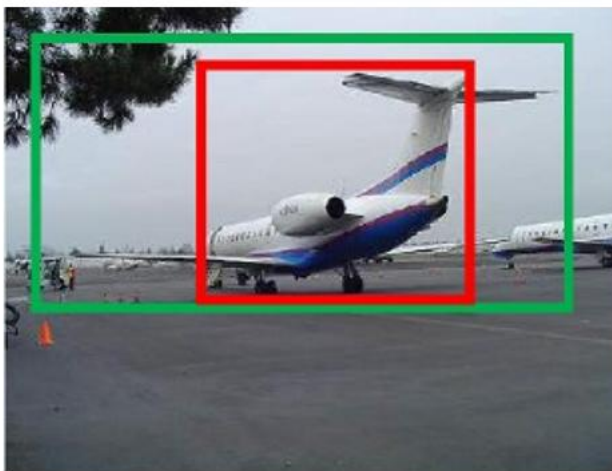


所有的conv层都是: kernel_size=3, pad=1, stride=1
所有的pooling层都是: kernel_size=2, pad=0, stride=2



$\text{num_output}=18$ ，经过 $1*1$ 卷积的输出图像为 $W \times H \times 18$ 大小

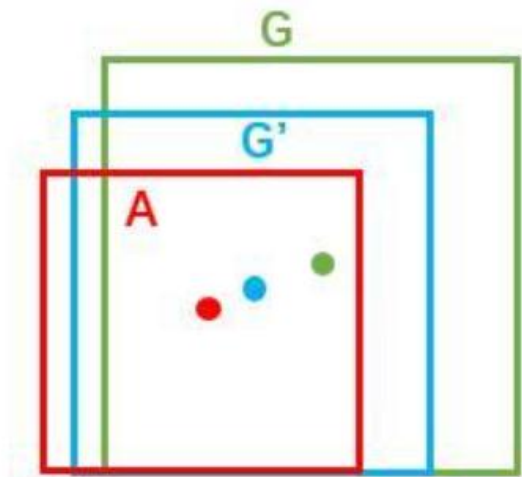
feature maps 每一个点都有9个anchors，同时每个anchors又有可能是positive和negative，所有这些信息都保存 $W \times H \times (9 \times 2)$ 大小的矩阵。



绿色框为飞机的Ground Truth(GT)，红色为提取的positive anchors，即便红色的框被分类器识别为飞机，但是由于红色的框定位不准，这张图相当于没有正确的检测出飞机。希望采用一种方法对红色的框进行微调，使得positive anchors和GT更加接近。

给定anchor $A = (A_x, A_y, A_w, A_h)$ 和 $GT = [G_x, G_y, G_w, G_h]$

寻找一种变换 $\mathbf{F}$, 使得: $F(A_x, A_y, A_w, A_h) = (G'_x, G'_y, G'_w, G'_h)$, 其中
 $(G'_x, G'_y, G'_w, G'_h) \approx (G_x, G_y, G_w, G_h)$



$$G'_x = A_w \cdot d_x(A) + A_x$$

$$G'_y = A_h \cdot d_y(A) + A_y$$

$$G'_w = A_w \cdot \exp(d_w(A))$$

$$G'_h = A_h \cdot \exp(d_h(A))$$

接下来的问题就是如何通过线性回归获得 $d_x(A), d_y(A), d_w(A), d_h(A)$ 了。线性回归就是给定输入的特征向量X, 学习一组参数W, 使得经过线性回归后的值跟真实值Y非常接近, 即 $Y = WX$ 。对于该问题, 输入X是cnn feature map, 定义为 Φ ; 同时还有训练传入A与GT之间的变换量, 即 (t_x, t_y, t_w, t_h) 。输出是 $d_x(A), d_y(A), d_w(A), d_h(A)$ 四个变换。那么目标函数可以表示为:

$$d_*(A) = W_*^T \cdot \phi(A)$$

其中 $\phi(A)$ 是对应anchor的feature map组成的特征向量, W_* 是需要学习的参数, $d_*(A)$ 是得到的预测值 (*表示 x, y, w, h, 也就是每一个变换对应一个上述目标函数)。为了让预测值 $d_*(A)$ 与真实值 t_* 差距最小, 设计L1损失函数:

$$\text{Loss} = \sum_i^N |t_*^i - W_*^T \cdot \phi(A^i)|$$

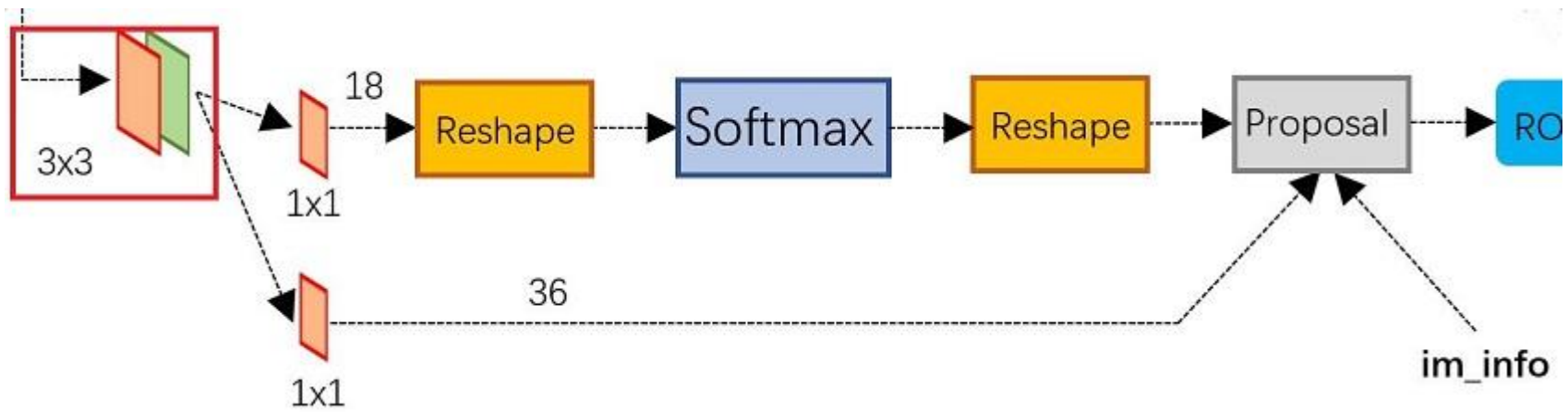
函数优化目标为:

$$\hat{W}_* = \operatorname{argmin}_{W_*} \sum_i^n |t_*^i - W_*^T \cdot \phi(A^i)| + \lambda ||W_*||$$



num_output=36, 即经过1*1卷积输出图像为WxHx36, 相当于feature maps每个点都有9个anchors, 每个anchors又都有4个用于回归的变换量

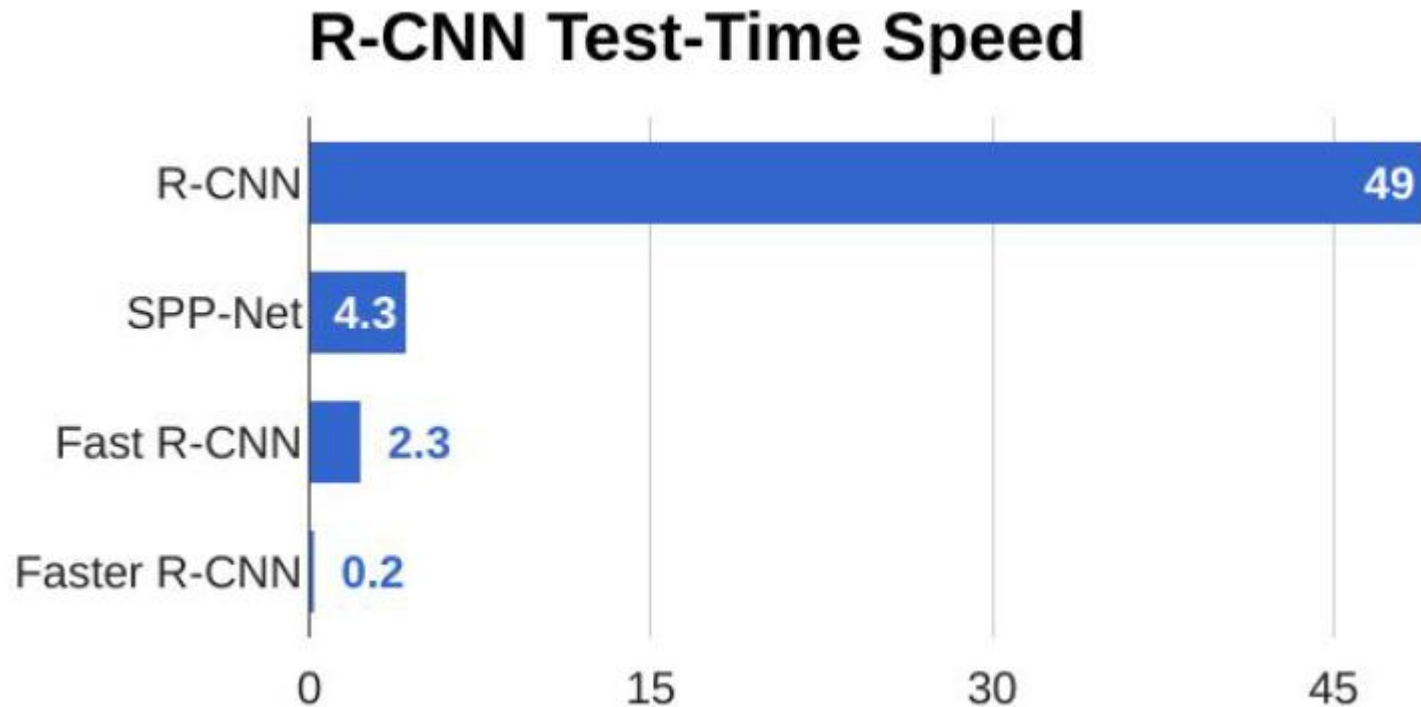
$$[d_x(A), d_y(A), d_w(A), d_h(A)]$$



生成anchors -> softmax分类器提取positive anchors -> bbox reg
归positive anchors -> Proposal Layer生成proposals

Faster R-CNN:

Make CNN do proposals!



Faster R-CNN:

Make CNN do proposals!

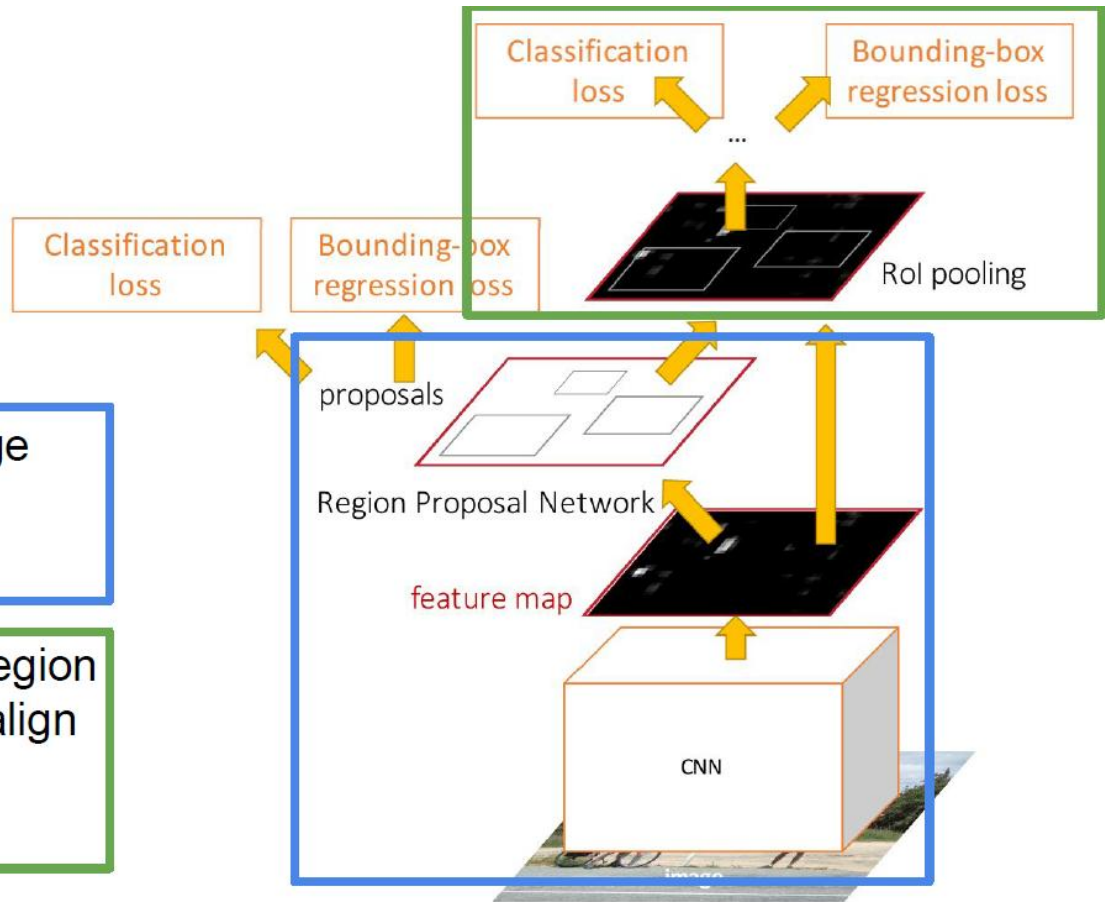
Faster R-CNN is a
Two-stage object detector

First stage: Run once per image

- Backbone network
- Region proposal network

Second stage: Run once per region

- Crop features: RoI pool / align
- Predict object class
- Prediction bbox offset



Faster R-CNN:

Make CNN do proposals!

Faster R-CNN is a
Two-stage object detector

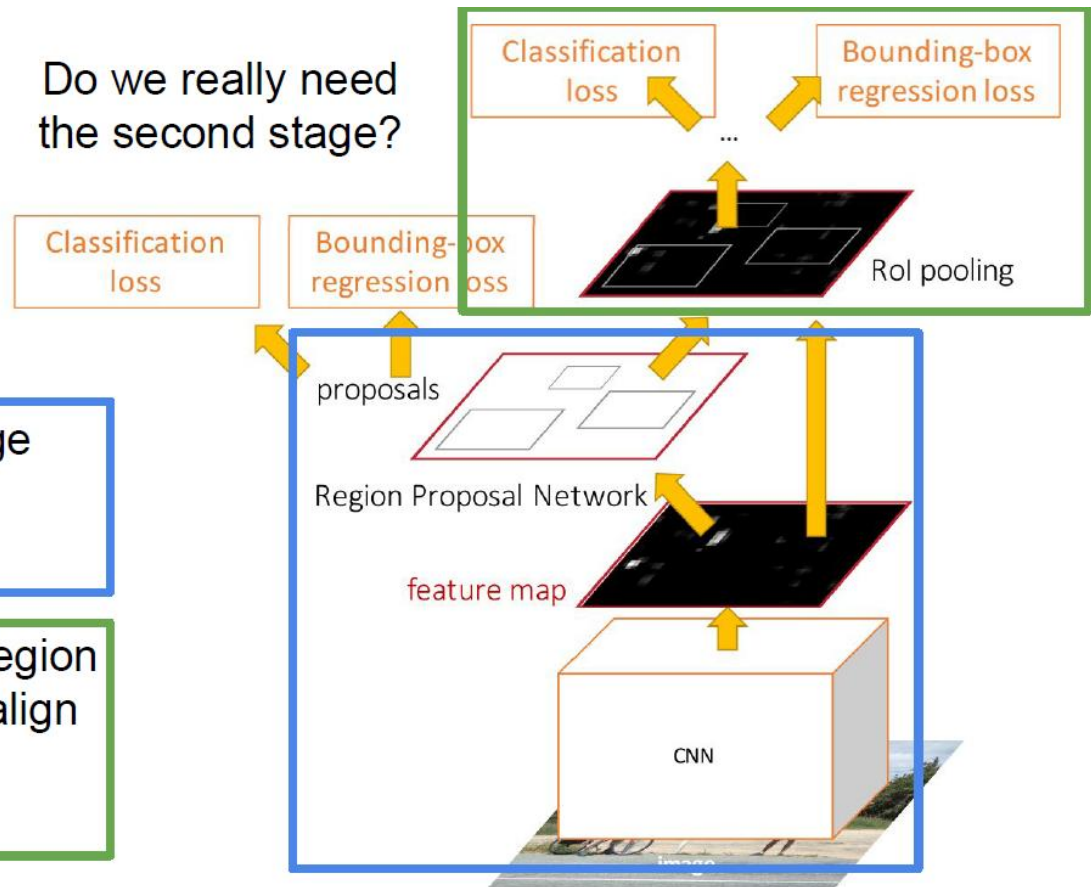
First stage: Run once per image

- Backbone network
- Region proposal network

Second stage: Run once per region

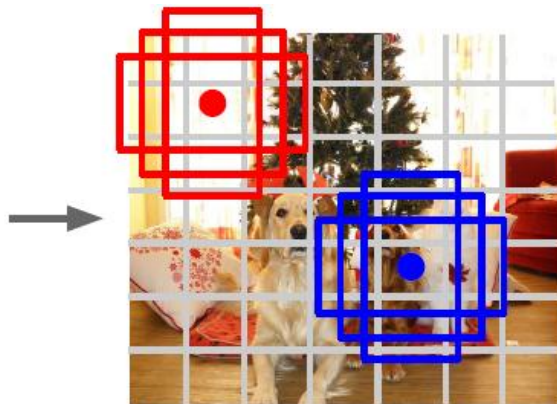
- Crop features: RoI pool / align
- Predict object class
- Prediction bbox offset

Do we really need
the second stage?



不Proposal的检测：YOLO / SSD

从输入图像到一个大的卷积网络的张量分数！



在每个网格单元格内：

- 从每个B基准盒到最后一个5个数字的盒子
(dx, dy, dh, dw, confidence)
- 预测每个C类的分数
(包括背景作为一个类)

Input image
 $3 \times H \times W$

将图像分成 7×7 格
在每个网格单元格中绘制一组基本框
这里 $B = 3$

Output:
 $7 \times 7 \times (5 * B + C)$

YOLO

将检测变为一个 regression problem, YOLO 从输入的图像, 仅仅经过一个 neural network, 直接得到 bounding boxes 以及每个 bounding box 所属类别的概率。正因为整个的检测过程仅仅有一个网络, 所以它可以直接 end-to-end 的优化。

YOLO

核心思想

输入：利用整张图作为网络的输入，

输出：直接在输出层回归bounding box的位置和bounding box所属的类别。

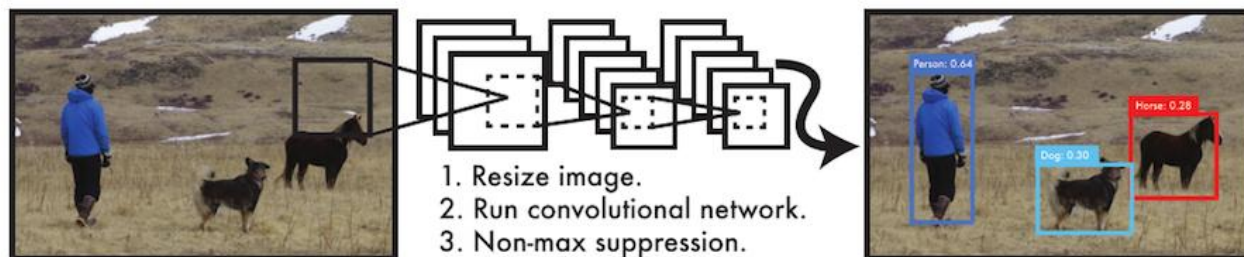
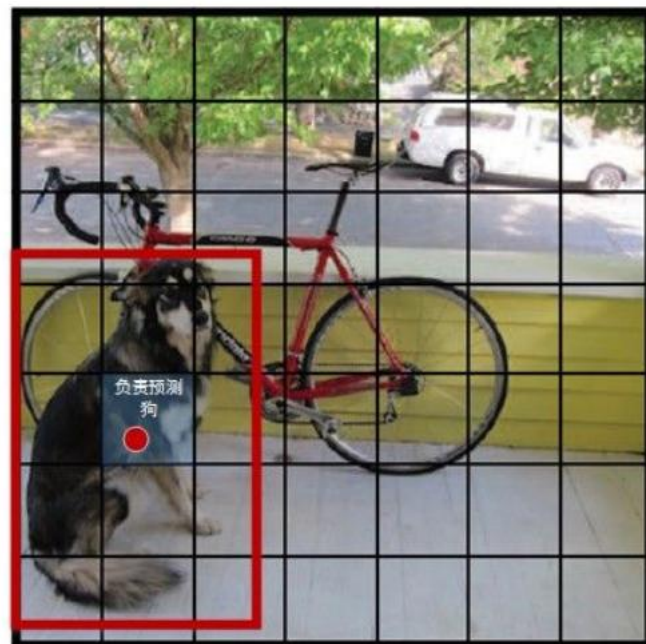


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

YOLO

核心思想

1. 将一幅图像分成SxS个网格(grid cell), 如果某个object的中心落在这个网格中, 则这个网格负责预测这个object。
2. 每个网格要预测B个bounding box, 每个bounding box除了要回归自身的位置之外, 还要附带预测一个confidence值, 即每个box要预测(x, y, w, h)和confidence共5个值。



3. 这个confidence代表了所预测的box中含有object的置信度和这个box预测的有多准两重信息

$$\text{Pr}(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

其中如果有object落在一个grid cell里, 第一项取1, 否则取0。第二项是预测的bounding box和实际的ground truth之间的IOU值。

YOLO

4. 每个bounding box要预测(x, y, w, h)和confidence共5个值，每个网格还要预测一个类别信息，记为C类。则SxS个网格，每个网格要预测B个bounding box还要预测C个categories。输出S x S x (5*B+C)的一个tensor。
注意：class信息是针对每个网格的，confidence信息是针对每个bounding box的。
5. 在test的时候，每个网格预测的class信息和bounding box预测confidence信息相乘，就得到每个bounding box的class-specific confidence score:

$$\Pr(\text{Class}_i | \text{Object}) * \Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

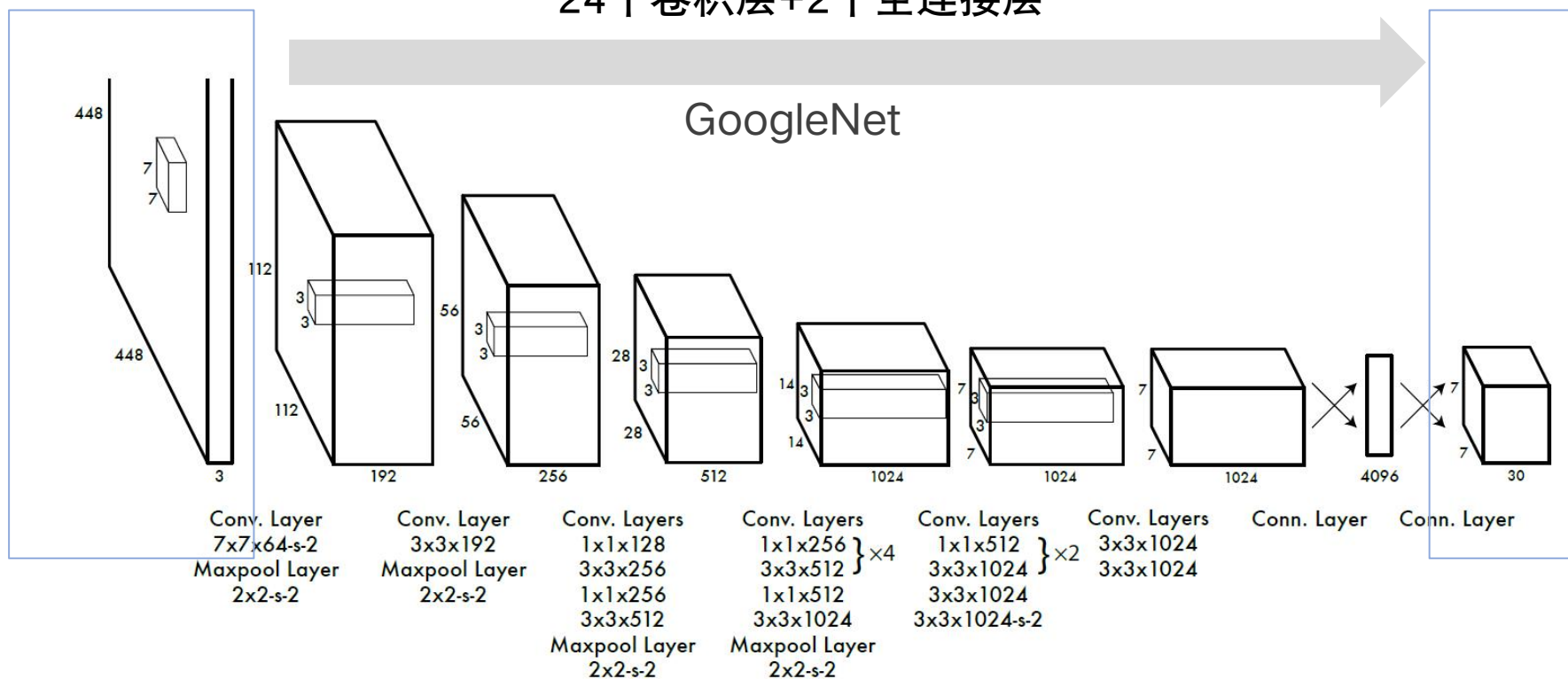
等式左边第一项就是每个网格预测的类别信息，第二三项就是每个bounding box预测的confidence。这个乘积即表示了预测的b-box属于某一类的概率，也有该box准确度的信息。

YOLO

输入 448×448 图像

24个卷积层+2个全连接层

输出 $7 \times 7 \times 30$ 张量



■ 损失函数

Output: $S \times S \times (5*B+C)$ 的一个tensor

B: (x, y, w, h, confidence)

C: $P_i(C)$ of Category

误差 = 定位误差 + 分类误差  不能等值加权

误差 = 定位误差  + 置信度误差 + 分类误差  置信度有两类

误差 = 定位误差  + 包含目标的框置信度误差
+ 不包含目标的框置信度误差  + 分类误差

■ 损失函数

采用平方根之差兼顾大框和小框的误差

$$\begin{aligned}
 & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \\
 & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \\
 & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
 & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
 & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
 \end{aligned}$$

判断第i个网格中的第j个box是否负责这个object

坐标预测

含object的box的confidence预测

不含object的box的confidence预测

判断是否有object中心落在网格i中

类别预测

bounding box的confidence预测误差。由于绝大部分网格中不包含目标，导致绝大部分box的confidence=0，所以在设计confidence误差时同等对待包含目标和不包含目标的box也是不合理的，否则会导致模型不稳定。作者在不含object的box的confidence预测误差中乘以惩罚权重 $\lambda_{\text{noobj}} = 0.5$ 。

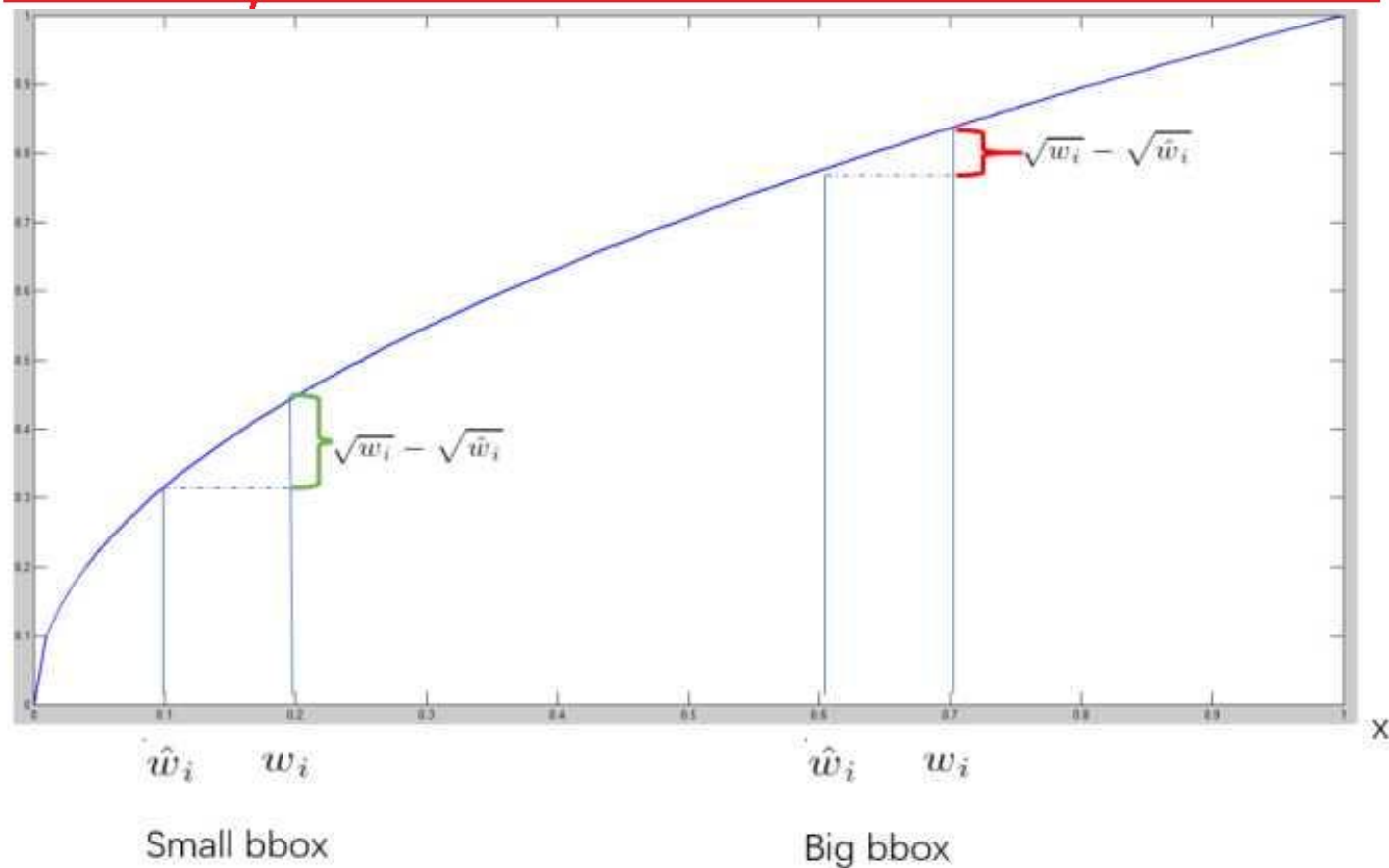
除此之外，同等对待4个值(x, y, w, h)的坐标预测误差与1个值的conference预测误差也不合理，所以作者在坐标预测误差之前乘以权重 $\lambda_{\text{coord}} = 5$

■ 损失函数

采用平方根之差兼顾
大框和小框的误差

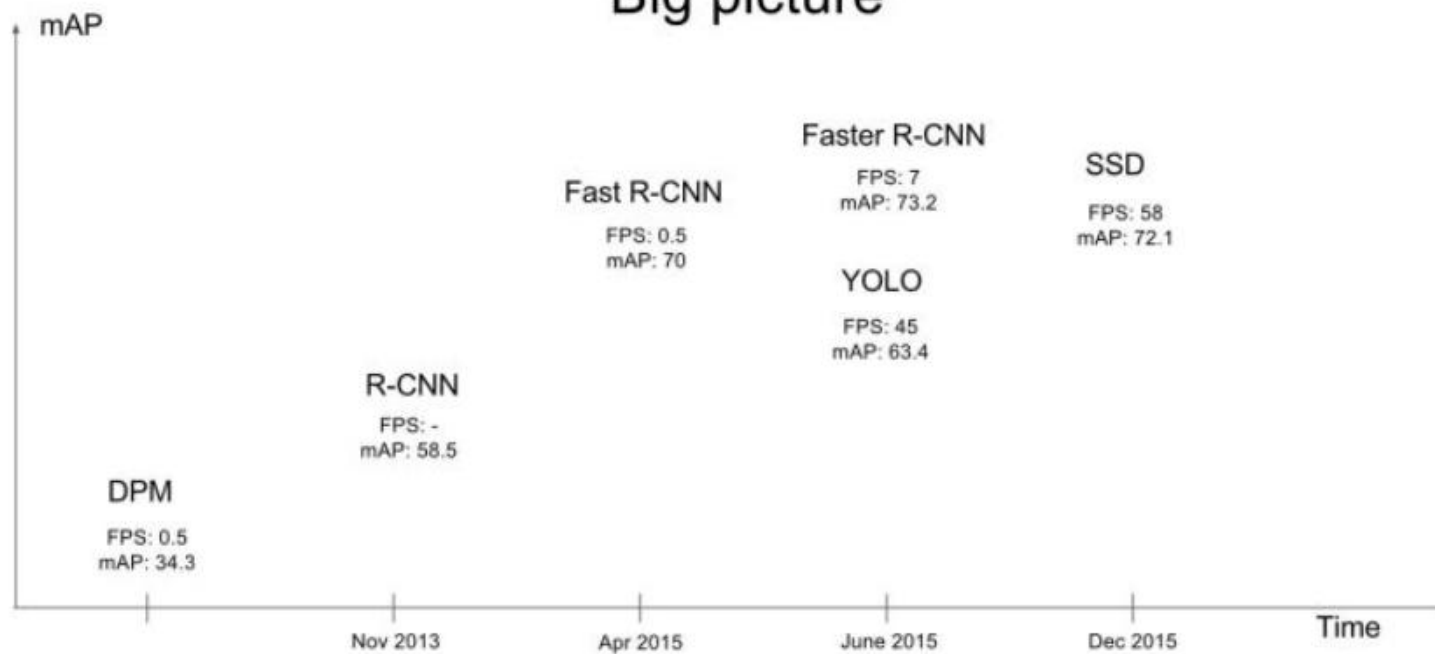
$$+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2$$

坐标预测



对不同大小的 bounding box 预测中，相比于大 box 大小预测偏一点，小 box 大小测偏一点肯定更不能被忍受。所以在 Loss 中同等对待大小不同的 box 是不合理的。先对 w 和 h 求平方根压缩数值范围，再进行回归。

Big picture



Object Detection: Lots of variables ...

Backbone

Network

VGG16

ResNet-101

Inception V2

Inception V3

Inception

ResNet

MobileNet

“Meta-Architecture”

Two-stage: Faster R-CNN

Single-stage: YOLO / SSD

Hybrid: R-FCN

Image Size

Region Proposals

...

Takeaways

Faster R-CNN is slower but more accurate

SSD is much faster but not as accurate

Bigger / Deeper backbones work better

Huang et al, “Speed/accuracy trade-offs for modern convolutional object detectors”, CVPR 2017

Zou et al, “Object Detection in 20 Years: A Survey”, arXiv 2019

R-FCN: Dai et al, “R-FCN: Object Detection via Region-based Fully Convolutional Networks”, NIPS 2016

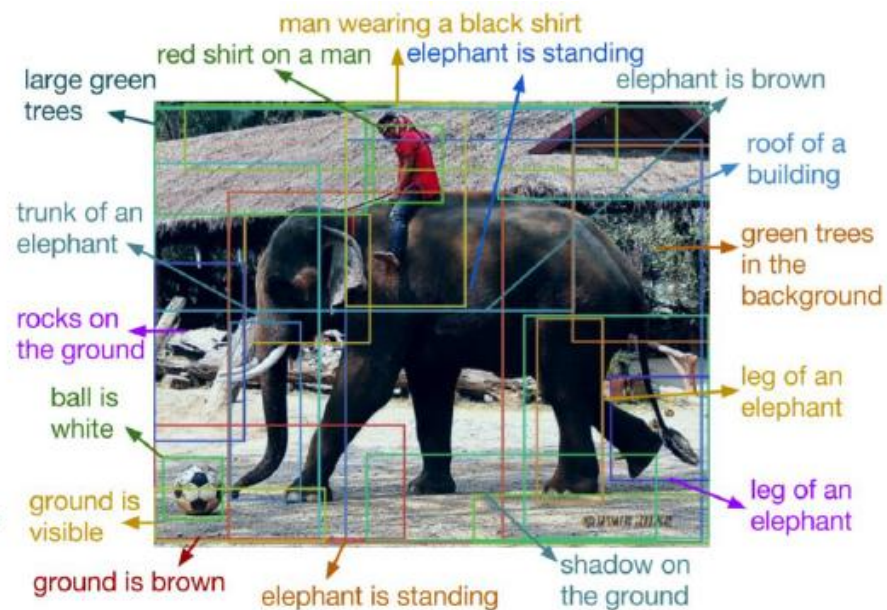
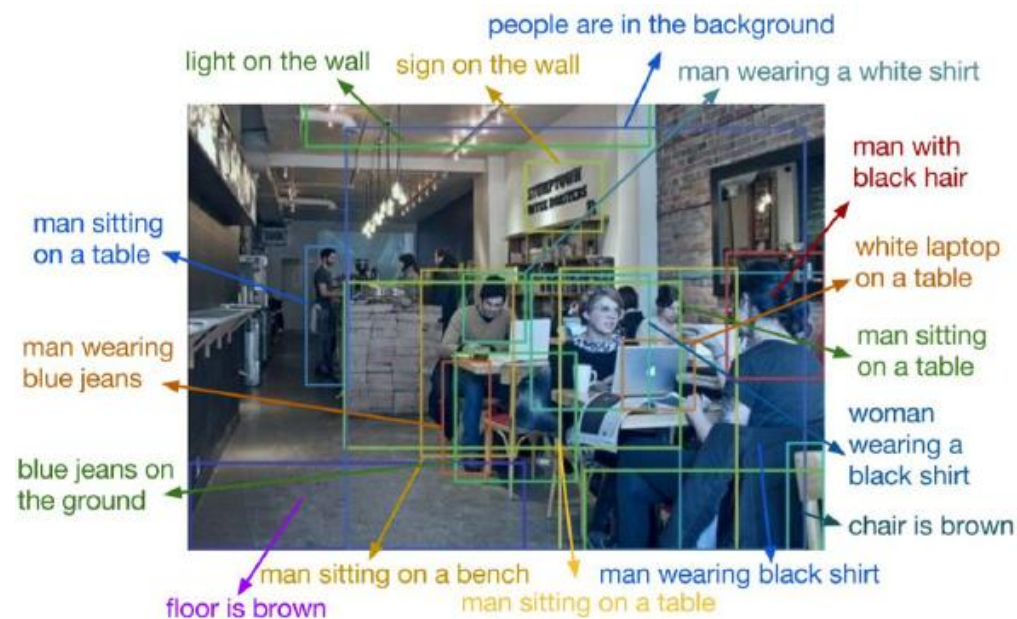
Inception-V2: Ioffe and Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”, ICML 2015

Inception V3: Szegedy et al, “Rethinking the Inception Architecture for Computer Vision”, arXiv 2016

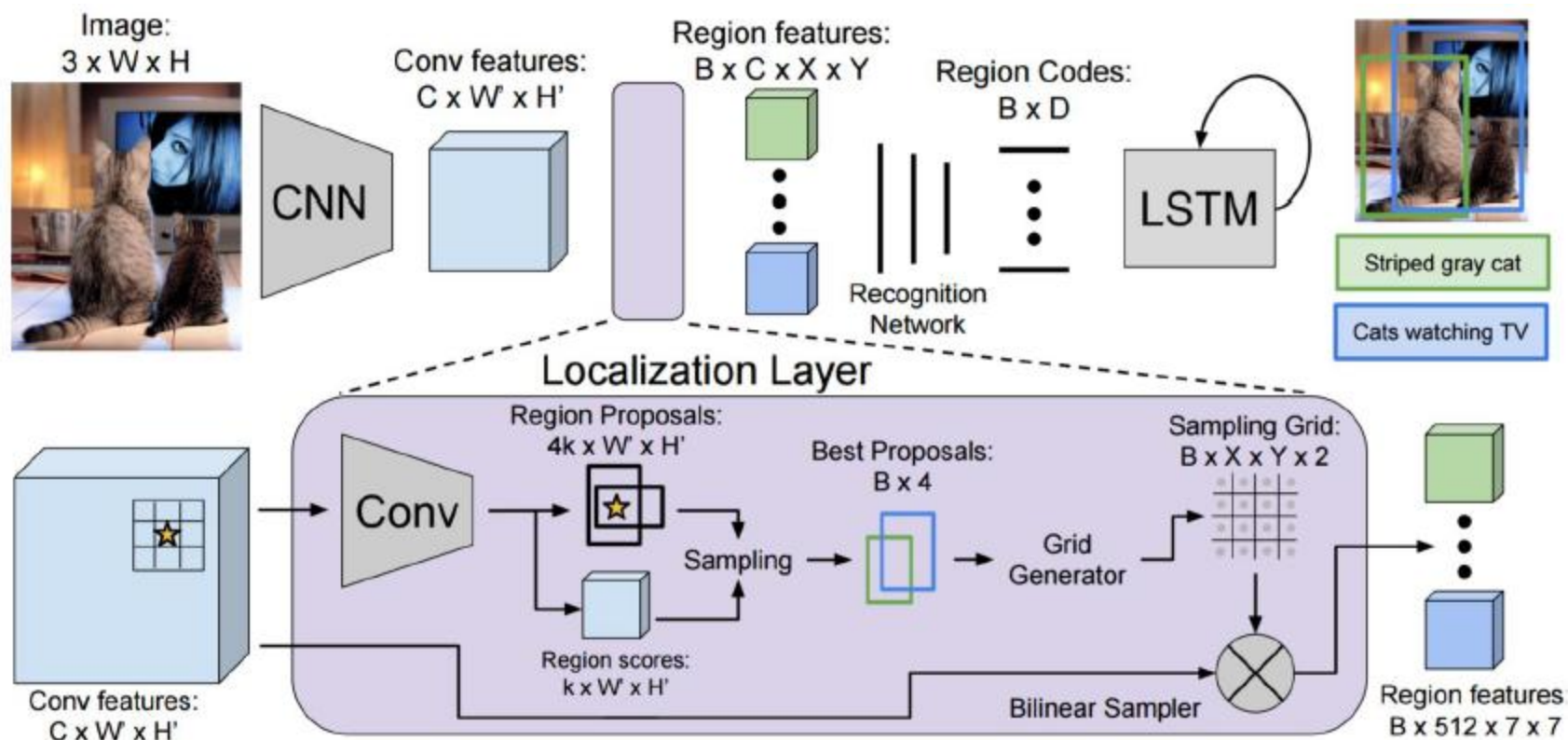
Inception ResNet: Szegedy et al, “Inception-V4, Inception-ResNet and the Impact of Residual Connections on Learning”, arXiv 2016

MobileNet: Howard et al, “Efficient Convolutional Neural Networks for Mobile Vision Applications”, arXiv 2017

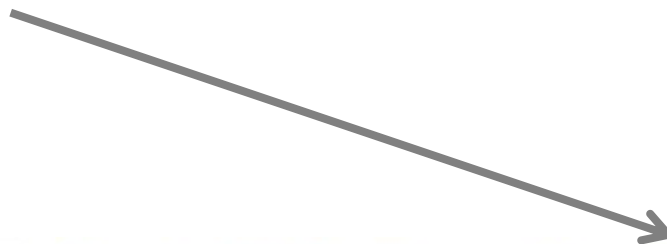
Aside: 目标检测+标题=密集标题



Aside: 目标检测+标题=密集标题



实例分割



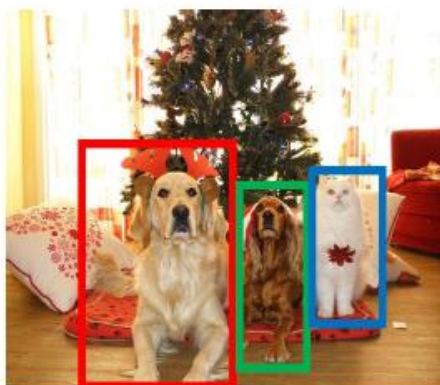
GRASS, CAT,
TREE, SKY

No objects, just pixels



CAT

Single Object



DOG, DOG, CAT

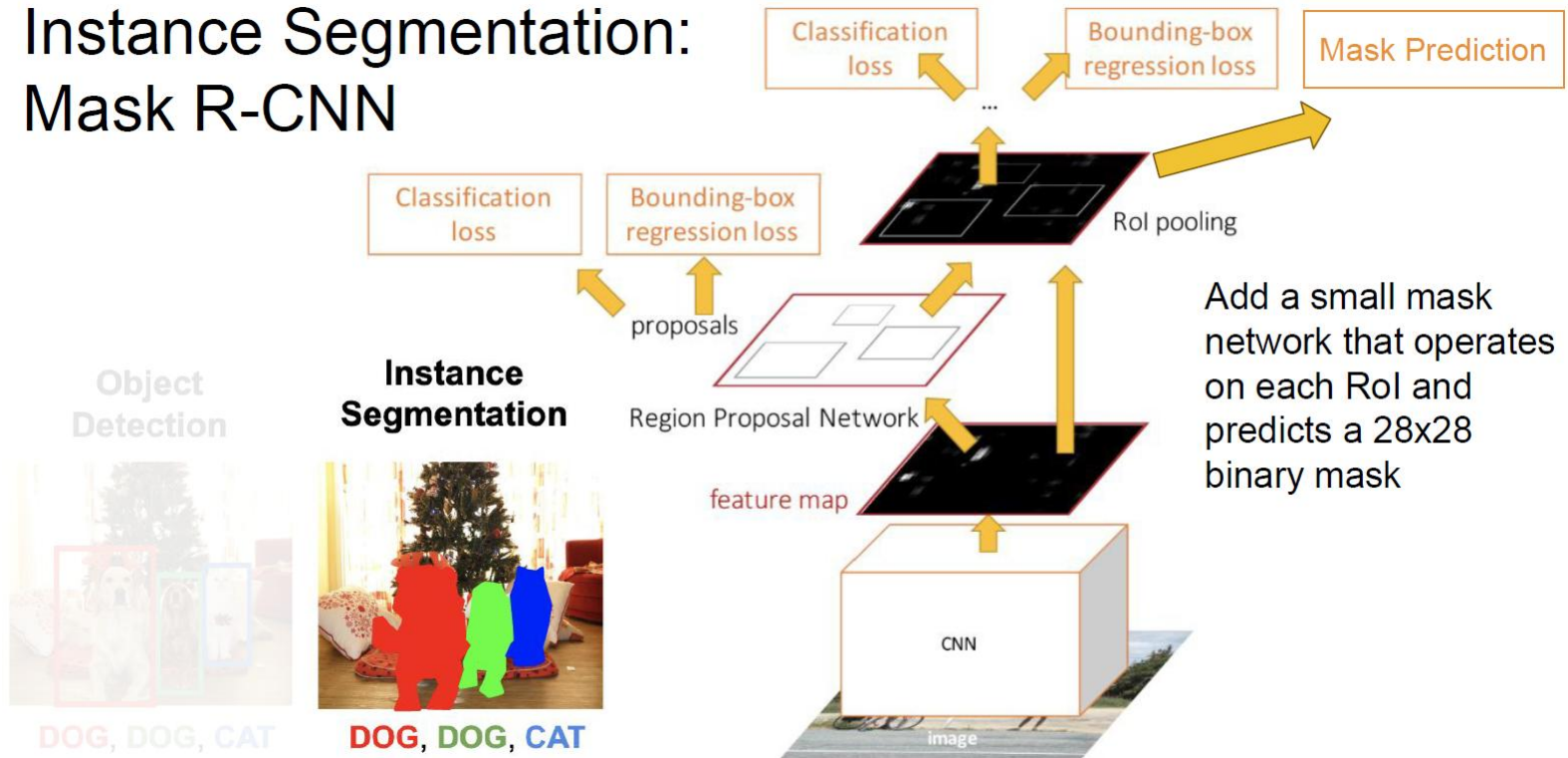
Multiple Object



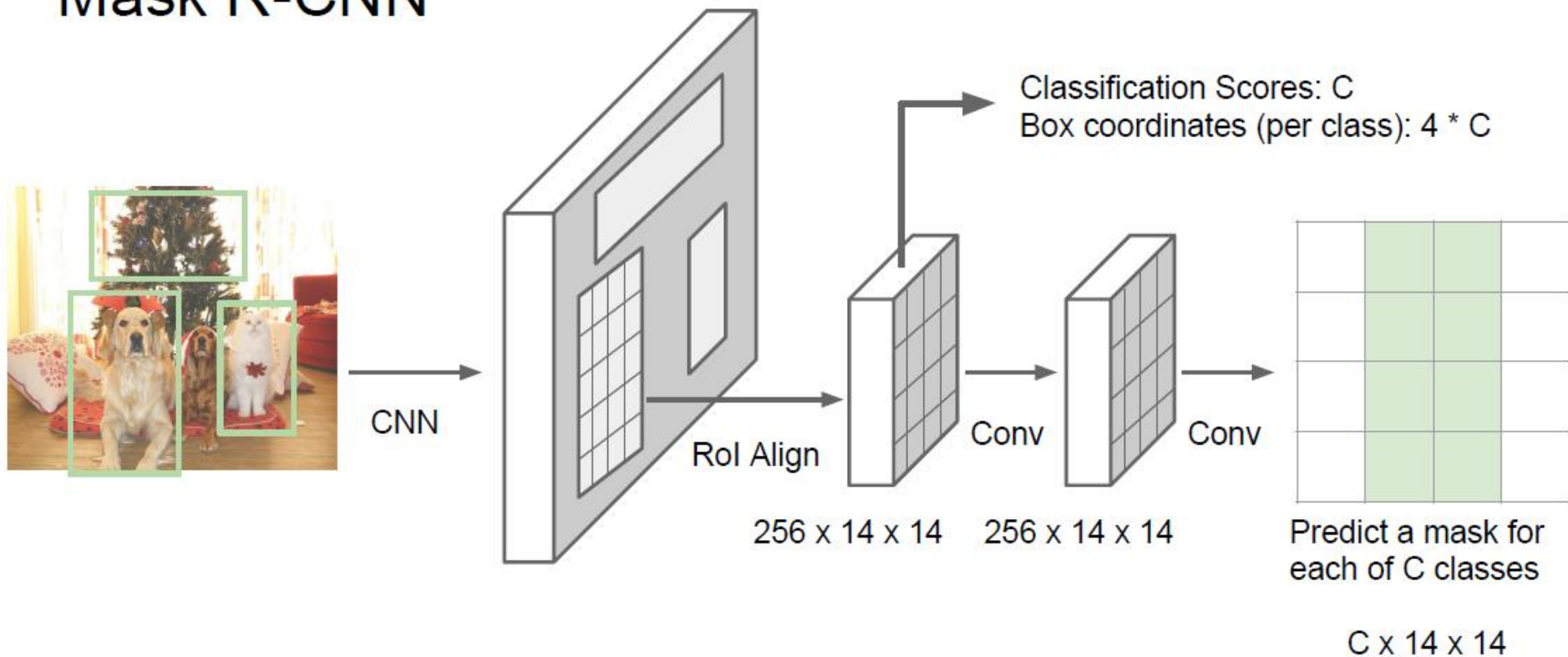
DOG, DOG, CAT

This image is CC0 public domain

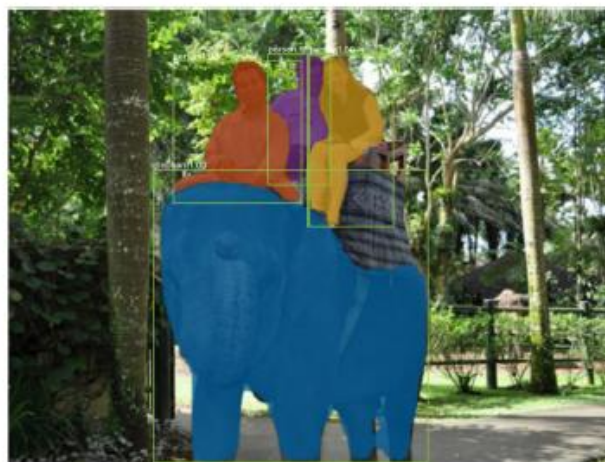
Instance Segmentation: Mask R-CNN



Mask R-CNN

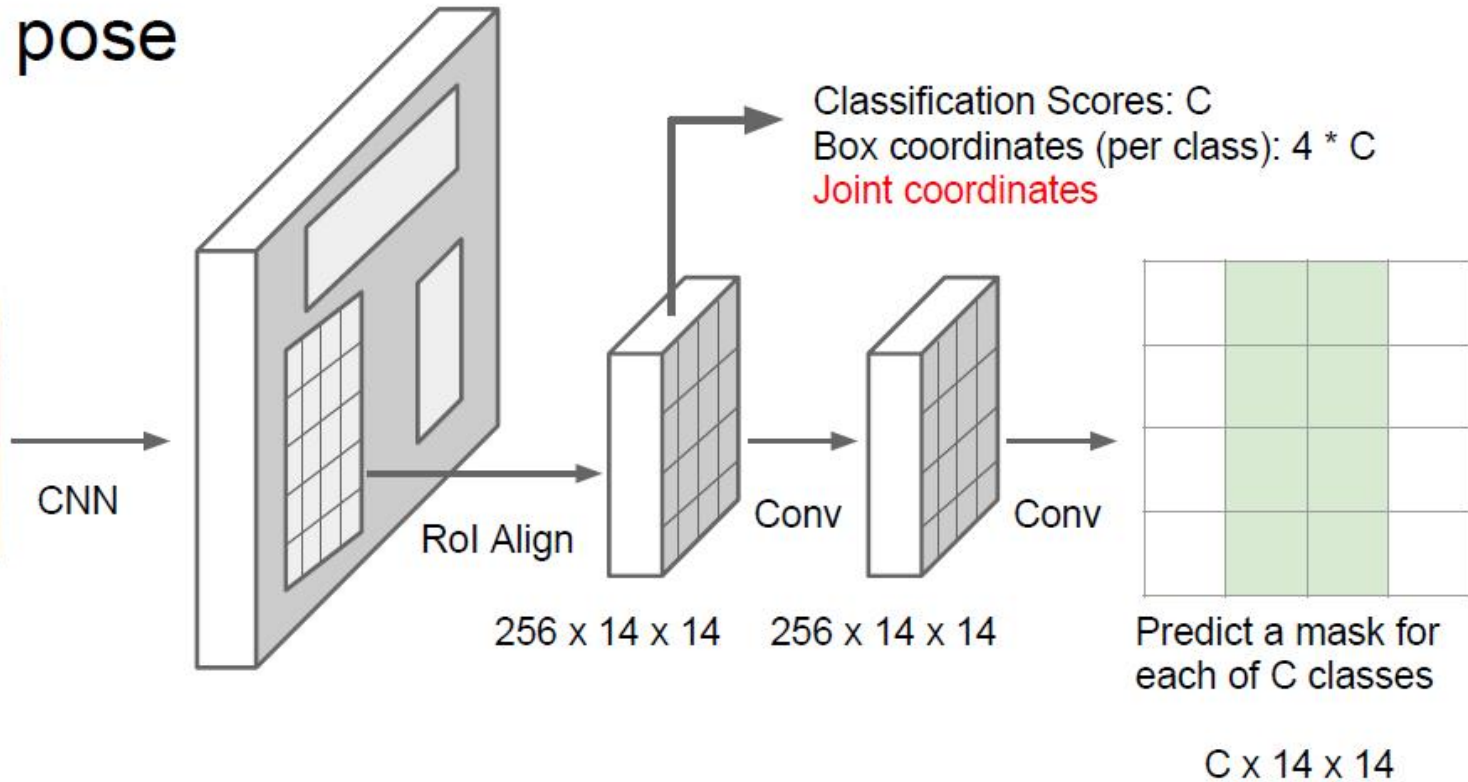


Mask R-CNN: 结果很好!



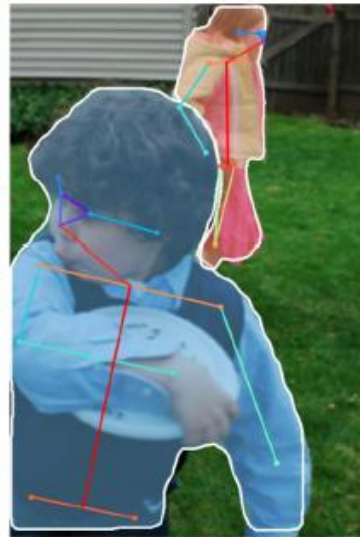
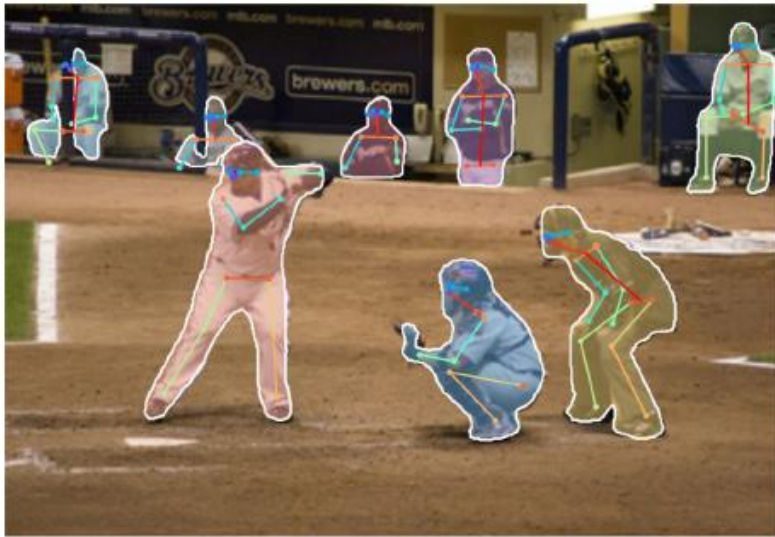
Mask R-CNN

Also does pose



Mask R-CNN

Also does pose



回顾

语义分割



GRASS, CAT,
TREE, SKY

No objects, just pixels

分类+ 局部化



CAT

Single Object

目标检测



DOG, DOG, CAT

Multiple Object

示例分割



DOG, DOG, CAT

This image is CC0 public domain